

ПРАКТИЧНА ДОВІДКА ДЛЯ ПОЧАТКІВЦІВ

Python через об'єкти

Практична довідка для початківців

Від першого .py-файла до об'єктів, файлів, помилок і тестів. Спочатку передбачаємо результат, потім запускаємо, простежуємо стан, поступово прибираємо підказки й переносимо знання у власну програму.

Перше офіційне видання · 2026

Попередня версія збережена на сайті як класичне видання.



Зміст

Як користуватися цією довідкою	12
Як побудований розділ	12
Три звички	13
.py-файл і перший об'єкт	14
Що зробимо	14
Що таке програма у файлі	14
Майстерня	17
Запуск через термінал	20
Файл, скрипт і інтерактивний режим	21
Типова помилка	22
Швидка перевірка	24
Самостійна робота	25
Підсумок	25
Стан об'єкта: рядки, числа й логічні значення	26
Що зробимо	26
Змінна зберігає значення	26
Майстерня	28
Коментарі пояснюють причину	32
Типова помилка	32
Швидка перевірка	34
Самостійна робота	35
Підсумок	35
Методи: параметри, аргументи й return	36
Що зробимо	36

Команді часто потрібні додаткові дані	36
Майстерня	37
return передає результат назовні	40
Передавання об'єкта методу	42
Типова помилка	42
Швидка перевірка	44
Самостійна робота	45
Підсумок	46
Рішення: порівняння, if, elif, else	47
Що зробимо	47
Порівняння створює логічне значення	47
Майстерня	49
Пов'язані гілки: if, elif, else	50
Кілька умов разом	51
Межі й порядок перевірок	52
Істинні та хибні значення	52
Перевірка стану без небезпечної зміни	53
Типова помилка	53
Швидка перевірка	56
Самостійна робота	57
Підсумок	57
Списки й кортежі: впорядковані колекції	58
Що зробимо	58
Список зберігає послідовність значень	58
Майстерня	60
Пошук, довжина й кількість	62

Сортування не тотожне порядку додавання	63
Копія й друге ім'я — не те саме	63
Кортеж фіксує послідовність	64
Типова помилка	65
Швидка перевірка	67
Самостійна робота	68
Підсумок	68

Повторення: for, range() і зрізи 69

Що зробимо	69
for бере елементи по одному	69
Майстерня	71
range() створює послідовність чисел	73
enumerate() дає і номер, і елемент	74
zip() поєднує послідовності попарно	74
Зріз бере частину послідовності	74
Числова статистика	75
Спискове включення створює новий список	75
Вкладені цикли	76
Типова помилка	76
Швидка перевірка	78
Самостійна робота	79
Підсумок	79

Діалог із програмою: input() і цикл while 80

Що зробимо	80
input() завжди повертає текст	80
while повторюється, доки умова істинна	82

Майстерня	83
break завершує найближчий цикл	85
continue переходить до наступної перевірки	86
Лічильник спроб	86
Нескінченний цикл і безпечне зупинення	87
Типова помилка	88
Швидка перевірка	89
Самостійна робота	90
Підсумок	90

Словники, вкладені дані й множини **91**

Що зробимо	91
Словник пов'язує ключ і значення	91
Майстерня	92
Перебирання словника	94
Вкладені структури	96
Коли клас, а коли словник	97
Безпечний параметр-словник	98
Множина зберігає унікальні значення	99
Типова помилка	100
Швидка перевірка	101
Самостійна робота	102
Підсумок	103

Функції й модулі: інструменти поза об'єктами **104**

Що зробимо	104
Функція має ім'я, параметри й результат	104
Майстерня	106

Параметри працюють так само, як у методах	107
Область видимості	108
Документаційний рядок	109
Модуль — звичайний .py-файл	110
Код для запуску й код для імпорту	111
Імпорт зі стандартної бібліотеки	112
Типова помилка	112
Швидка перевірка	113
Самостійна робота	114
Підсумок	114
Структура OOP-проєкту у VS Code	115
Що зробимо	115
Почни з маленької карти	115
Майстерня	116
Пакет і файл __init__.py	119
Абсолютні й відносні імпорти	120
Запускай з кореня проєкту	120
Поточна папка й шляхи до файлів	121
Віртуальне середовище	122
Кеші й службові файли	124
Циклічні залежності як сигнал структури	124
Типова помилка	125
Швидка перевірка	126
Самостійна робота	127
Підсумок	127
Пов'язані об'єкти: успадкування, композиція й властивості	128

Що зробимо	128
Успадкування передає спільний опис	128
Майстерня	130
Композиція: об'єкт складається з інших об'єктів	132
Підкреслення позначає внутрішню деталь	132
@property дає контрольоване читання	133
Атрибут класу спільний для всіх об'єктів	134
@classmethod як альтернативний конструктор	135
Поліморфізм: одна команда для різних об'єктів	135
Типова помилка	136
Швидка перевірка	137
Самостійна робота	138
Підсумок	138

Стандартна бібліотека й керована випадковість 140

Що зробимо	140
Три джерела імпортів	140
Майстерня	141
Основні інструменти random	142
Зерно не робить результат випадковим	144
Керована перевірка без вгадування	145
statistics: готові обчислення	145
collections.Counter: словник частот	146
Enum: названий набір станів	147
Дата й час	147
Як читати документацію модуля	148
Сторонній пакет: коли він справді потрібен	148
Типова помилка	149

Швидка перевірка	150
Самостійна робота	151
Підсумок	151

Текстові файли, UTF-8 і шляхи 153

Що зробимо	153
Path представляє шлях як об'єкт	153
Майстерня	155
UTF-8 робить текст передбачуваним	157
Відносний і абсолютний шлях	157
Створення папок і перевірка типу шляху	159
Читання великих файлів рядок за рядком	160
Двійкові файли — не текст	160
Різниця запису шляхів у системах	160
Типова помилка	162
Швидка перевірка	163
Самостійна робота	164
Підсумок	164

Виятки: передбачувані збої й корисні повідомлення 166

Що зробимо	166
Traceback показує шлях до винятку	166
try і точний except	167
Майстерня	168
else — шлях без винятку	170
finally виконується завжди	170
Кілька очікуваних типів	171
Ієрархія винятків і порядок except	171

Поширені винятки початкових програм	172
Перевірити наперед чи спробувати	172
Виняток або значення результату	173
Не використовуй assert для введення	173
Типова помилка	174
Швидка перевірка	175
Самостійна робота	176
Підсумок	177

JSON: збереження стану й рефакторинг 178

Що зробимо	178
JSON має прості типи	178
dumps() і loads() працюють із рядком	179
dump() і load() працюють із файловим об'єктом	180
Майстерня	181
Перевірка типів після JSON	184
Які Python-значення змінюються в JSON	184
Версія формату й зміна структури	185
Обережний запис через тимчасовий файл	186
Рефакторинг: змінюємо будову, зберігаємо поведінку	186
JSON не є безпечним лише тому, що текстовий	187
Типова помилка	188
Швидка перевірка	189
Самостійна робота	190
Підсумок	191

Автоматичні тести з pytest 192

Що зробимо	192
----------------------	-----

pytest — стабільний сторонній пакет	192
Як pytest знаходить тести	193
Перший тест: звичайний assert	194
Майстерня	195
Спочатку побач падіння	197
Перевірка винятку через pytest.raises	198
Fixture готує надійний контекст	199
Параметризація перевіряє таблицю випадків	199
tmp_path дає окрему тимчасову папку	200
Перевірка виведення через capsys	201
Що саме тестувати в об'єкті	201
Набір тестів і регресія	202
Конфігурація в pyproject.toml	202
Стандартний unittest	202
Типова помилка	203
Швидка перевірка	204
Самостійна робота	205
Підсумок	206
Підтримуваний код: стиль, налагодження й карта синтаксису	207
Що зробимо	207
Три великі види проблем	207
Майстерня	209
Як читати traceback	212
Спостереження без здогадок	212
Налагоджувач VS Code	213
Мінімальне відтворення	215
Журналювання для довшої роботи	215

Стиль — інтерфейс для читача	216
Анотації типів допомагають редактору	218
Маленькі методи й явний стан	219
Карта синтаксису	219
Контрольний маршрут перед завершенням зміни	221
Типова помилка	222
Швидка перевірка	223
Самостійна робота	224
Підсумок	225

ВСТУП

Як користуватися цією довідкою

Це не довгий курс, який треба обов'язково пройти від першої до останньої сторінки. Це карта основ Python. Якщо ти лише починаєш, рухайся послідовно. Якщо вже писав або писала код, відкрий потрібну тему через зміст чи пошук.

Це **перше офіційне видання** з активнішим маршрутом навчання. Попередня версія збережена на сайті як **класичне видання**: її можна читати окремо, але основним матеріалом є ця редакція.

РЕЗУЛЬТАТ

Після кожного розділу в тебе має залишитися не тільки нове слово, а програма, яку можна запустити, змінити й перевірити.

Як побудований розділ

- 1 **Фокус.** Знайди одну головну дію та орієнтир, до якого можна повернутися після відволікання.
- 2 **Передбачення.** Прочитай код і запиши очікуваний результат до натискання **Run**.
- 3 **Запуск.** Перевір прогноз на реальній програмі, а не лише подумки.
- 4 **Трасування.** Зістав стан «до», точні кроки виконання і стан «після».
- 5 **Пояснення.** Сформулюй нове правило власними словами.
- 6 **Поступове зняття підказок.** Доповни пропуск або склади перемішаний код, потім зміни готовий приклад.
- 7 **Навмисна помилка.** Побач повідомлення Python, знайди причину й віднови робочу версію.
- 8 **Згадування.** Закрий приклад і коротко відтвори головне без підглядання.
- 9 **Самостійне перенесення.** Застосуй той самий принцип до іншого об'єкта або ситуації.

Не кожна сторінка містить усі дев'ять кроків. Це маршрут цілого поняття або розділу, а не анкети після кожного абзацу.

ЩО ОЗНАЧАЄ «ЧЕРЕЗ ОБ'ЄКТИ»

Ми часто уявлятимемо програму як світ із конкретними учасниками. Робот має ім'я і заряд, герой отримує шкоду, інвентар зберігає предмети, а сховище записує дані у файл. У коді такі учасники стають об'єктами. Так легше бачити не окремі команди, а зв'язки між даними й діями.

Три звички

- Запускай код після кожної невеликої зміни.
- Читай останній рядок помилки, а потім піднімайся вище до свого файлу.
- Спочатку поясни собі очікуваний результат, а вже потім натискай **Run**.

КОДОВА ВСТАВКА ЧИ СКРІНШОТ

Код показано текстовою вставкою, коли важливі точні символи, відступи, логіка або рух значення. Так його можна прочитати, скопіювати й порівняти.

Обрізаний скріншот VS Code потрібен лише тоді, коли важливе **місце в інтерфейсі**: де створити файл, де натиснути запуск або де побачити результат. Послідовність із кількох кадрів використовується тільки для помітної зміни стану редактора.

Команди й шляхи, які відрізняються у Windows, macOS та Linux, показані текстом. Ми не вигадуємо скріншоти систем, яких не перевіряли; зображення VS Code демонструють лише спільні для платформ елементи.

ПЕРША САМОПЕРЕВІРКА

Обери будь-який предмет поруч. Запиши дві його властивості та дві дії, які він може виконувати або які можна виконати з ним. Це майже готова заготовка майбутнього класу.

РОЗДІЛ 01

.py-файл і перший об'єкт

У цьому розділі ми не встановлюємо редактор і Python. Вважаємо, що папка вже відкрита у VS Code, але окремо перевіримо два способи запуску: кнопкою редактора та командою у звичайному терміналі. Наше завдання — зрозуміти, **що саме ми запускаємо**, і відразу створити об'єкт, який щось робить.

Що зробимо

Наприкінці розділу в папці буде файл `robot.py`. У ньому ми опишемо робота, створимо конкретного робота й накажемо йому привітатися.

РЕЗУЛЬТАТ

Після запуску в терміналі має з'явитися повідомлення Привіт! Я Робі.. Ти знатимеш, де в коді опис об'єктів, де конкретний об'єкт, а де команда для нього.

ЗАРАЗ У ФОКУСІ

Одна дія: створи `robot.py`, запусти його й знайди рядок, який справді наказує об'єкту діяти.

Якщо відволікся, повернися до виклику `robot.say_hello()`: саме з нього починається видима поведінка.

Що таке програма у файлі

Відкрий у VS Code новий файл і збережи його як `robot.py`.

Звичайний `.py`-файл — це **текстовий файл**. Усередині немає прихованих деталей, спеціальних кнопок чи готової програми. Там лежать символи, які можна було б набрати навіть у найпростішому текстовому редакторі.

Розширення `.py` повідомляє людям і програмам: «цей текст написаний мовою Python». Завдяки цьому:

- VS Code підсвічує різні частини коду різними кольорами;
- редактор пропонує доречні підказки;
- операційна система може пов'язати файл з Python;
- іншим програмістам відразу зрозуміло, як файл запускати.

Сам код виконує не VS Code. **VS Code редагує текст**, а **інтерпретатор Python читає команди й виконує їх**. Кнопка запуску у VS Code лише передає відкритий файл вибраному інтерпретатору.

ІНТЕРПРЕТАТОР

Інтерпретатор — програма, яка читає код Python і виконує його. Якщо VS Code показує код, але команда запуску не працює, проблема може бути не у файлі, а у виборі інтерпретатора.

Маленький експеримент із .txt

ДОВЕДИ, ЩО РОЗШИРЕННЯ НЕ ЗАПУСКАЄ КОД

Створи файл `message.txt` із таким вмістом і натисни `Ctrl+S`. Біла крапка біля назви вкладки має зникнути — це означає, що зміни записані на диск.

```
message.txt

print("Цей текст виконав Python.")
```

До запуску скажи вголос, що тут робитиме VS Code, а що — інтерпретатор Python. Потім перевір припущення командою для своєї системи нижче.

Якщо явно передати цей файл інтерпретатору, він спробує виконати вміст навіть без розширення `.py`. У звичайному Windows PowerShell, відкритому в папці з файлом, спочатку перевір обидві умови:

```
powershell

Test-Path .\message.txt
python --version
python .\message.txt
```

На перевіреному Windows-комп'ютері результат виглядав так; номер версії на іншому комп'ютері може відрізнятися:

```
Результат

True
Python 3.13.2
Цей текст виконав Python.
```

Тут важливі всі три рядки:

- 1 True доводить, що PowerShell справді бачить збережений файл у поточній папці.
- 2 Python 3.13.2 доводить, що саме зовнішній PowerShell знайшов інтерпретатор. Сам факт, що Python працює у VS Code, цього ще не гарантує.
- 3 Лише після двох перевірок запускаємо `python .\message.txt`.

Якщо Test-Path показав False, не перемикай навання версії Python: спочатку збережи файл і перевір його назву та папку. Якщо файл знайдено, але `python --version` не працює або Windows пропонує відкрити Store, спробуй launcher-команду:

```
powershell  
  
py --version  
py .\message.txt
```

Якщо VS Code запускає код через проєктне середовище `.venv`, а зовнішній PowerShell не знаходить ані `python`, ані `py`, з кореня саме цього проєкту можна явно викликати той самий інтерпретатор:

```
powershell  
  
.\.venv\Scripts\python.exe .\message.txt
```

Цей останній шлях працює лише тоді, коли папка `.venv` справді є в поточному проєкті. Її не треба вигадувати або створювати заради цього експерименту.

У Command Prompt замість Test-Path використовуй:

```
bat  
  
dir message.txt  
python message.txt
```

На macOS або Linux спочатку перевір файл і команду так:

```
bash  
  
ls ./message.txt  
python3 --version  
python3 ./message.txt
```

Цей експеримент не означає, що варто називати програми `.txt`. Інтерпретатору важливий вміст, але редакторам, інструментам і людям потрібна зрозуміла домовленість. Для Python-скриптів такою домовленістю є `.py`.

Майстерня

Набери у `robot.py` цей код. Не вставляй нові команди, доки перша версія не запрацює.

СПОЧАТКУ ПЕРЕДБАЧ

Не запускай файл одразу. Визнач, чи надрукує щось сам опис класу `Robot`, скільки рядків з'явиться в терміналі та який рядок коду запустить друк.

`robot.py`

```
class Robot:
    def say_hello(self):
        print("Привіт! Я Робі.")

robot = Robot()
robot.say_hello()
```

ВІД ОПИСУ КЛАСУ ДО ПЕРШОЇ ДІЇ

ДО
Об'єкта `robot` ще немає, термінал порожній.

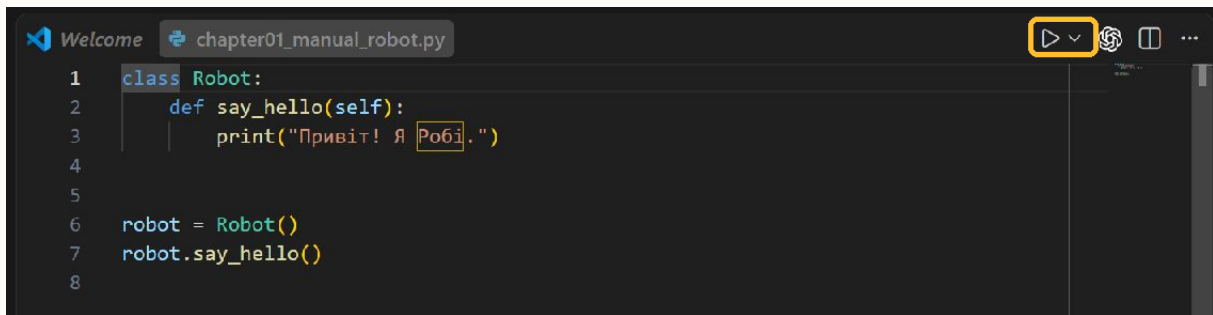
→

ПІСЛЯ
Об'єкт `robot` існує, у терміналі є
Привіт! Я Робі.

1	Опис	<code>class Robot:</code>	Python запам'ятовує, які дії матимуть роботи; тіло методу ще не виконується.
2	Створення	<code>robot = Robot()</code>	З'являється конкретний об'єкт, на який посилається ім'я <code>robot</code> .
3	Виклик	<code>robot.say_hello()</code>	Крапка обирає метод об'єкта, а Python передає цей об'єкт у <code>self</code> .
4	Видимий результат	<code>print("Привіт! Я Робі.")</code>	Команда всередині методу записує один рядок у термінал.

Що тут важливо: Клас описує можливість, об'єкт є конкретним учасником, а виклик методу запускає дію.

Натисни `Ctrl+S` і перевір, що біла крапка біля назви вкладки зникла. Потім запусти файл кнопкою **Run Python File** у правому верхньому куті VS Code. Редактор відкриє вбудований термінал і передасть збережений `robot.py` вибраному Python.

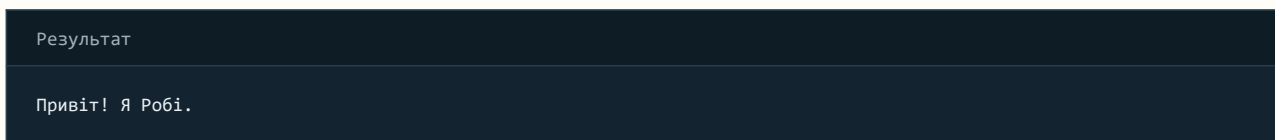


VS Code з відкритим robot.py; жовтою рамкою позначено кнопку Run Python File

Кадр зроблено у Windows. На macOS і Linux тема та службові кнопки можуть трохи відрізнятися, але виділений трикутник — команда самого VS Code. Відмінні команди запуску для кожної системи нижче показано текстом, а не вигаданими скріншотами.

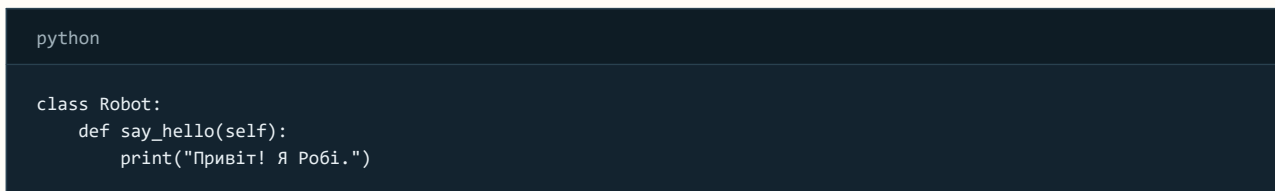
У нижній панелі VS Code видно, який інтерпретатор вибрано. Звичайний PowerShell поза VS Code може знайти інший Python або не знайти жодного — тому перевірки `python --version` і `py --version` не зайві.

Очікуваний результат:



Це вже об'єктна програма. Вона маленька, але в ній є три різні рівні.

1. Клас описує можливості



`class Robot:` починає опис роботів. Клас можна уявити як креслення або набір правил: кожен об'єкт, створений за цим описом, матиме передбачені можливості.

Назву класу пишемо з великої літери. Якщо слів декілька, кожне теж починаємо з великої: `GameCharacter`, `MusicPlayer`, `SecretDoor`.

Рядок `def say_hello(self):` описує дію. Дія, що належить об'єкту, називається **методом**. Назва `say_hello` читається як «скажи привіт».

Поки що сприймай `self` як слово «цей конкретний об'єкт». Воно дає методу доступ до робота, для якого викликали команду. У наступних розділах ми використаємо `self`, щоб кожен робот мав власне ім'я й заряд.

2. Відступ показує належність

```
python

class Robot:
    def say_hello(self):
        print("Привіт! Я Робі.")
```

У Python відступ — частина граматики. Він не просто робить текст красивим.

- `def say_hello(self):` має чотири пробіли, тому метод належить класу `Robot`.
- `print(...)` має ще чотири пробіли, тому команда належить методу `say_hello`.
- `robot = Robot()` стоїть без відступу, тому опис класу перед ним уже закінчився.

У VS Code клавіша `Tab` зазвичай додає потрібний рівень відступу. У Python прийнято використовувати чотири пробіли на рівень і не змішувати пробіли з табуляцією.

3. Об'єкт — конкретний учасник програми

```
python

robot = Robot()
robot.say_hello()
```

`Robot()` створює конкретний об'єкт за описом класу `Robot`. Ми даємо цьому об'єкту коротке ім'я `robot`, щоб звертатися до нього далі.

Крапка у `robot.say_hello()` читається так: «у об'єкта `robot` виконай метод `say_hello`». Порожні дужки означають, що зараз команді не передаємо додаткових даних.

ПОБАЧ ДВА РІЗНІ ОБ'ЄКТИ

Додай ще одного робота й виклич той самий метод для обох:

```
two_robots.py

class Robot:
    def say_hello(self):
        print("Привіт! Я Робі.")

first_robot = Robot()
second_robot = Robot()

first_robot.say_hello()
second_robot.say_hello()
```

Поки обидва поведуться однаково. Це нормально: ми вже створили два об'єкти, але ще не дали їм окремого стану. Саме це зробимо в наступному розділі.

Запуск через термінал

Кнопка **Run Python File** зручна, але вона не є єдиним способом запуску. Відкрий у VS Code меню **Terminal** → **New Terminal**. Новий термінал зазвичай починає роботу в корені відкритої проєктної папки.

КОМАНДИ ДЛЯ РІЗНИХ ОПЕРАЦІЙНИХ СИСТЕМ

Windows: PowerShell або Command Prompt

```
powershell  
  
python robot.py
```

Сучасна офіційна збірка Python для Windows рекомендує команду `python`. Якщо на конкретному комп'ютері вона не налаштована, часто працює `launcher`-команда:

```
powershell  
  
py robot.py
```

PowerShell — це оболонка Windows, тобто програма, яка приймає текстові команди. Рядок на кшталт `PS C:\Projects\Python>` є запрошенням оболонки; його не треба набирати.

macOS: Terminal із zsh

```
bash  
  
python3 robot.py
```

Стандартна програма для команд називається **Terminal**, а типовою оболонкою сучасної macOS є `zsh`. На Mac команда `python` може бути відсутня або вказувати не туди, тому поза активованим віртуальним середовищем використовуй `python3`.

Linux: Terminal із Bash або іншою оболонкою

```
bash  
  
python3 robot.py
```

У більшості дистрибутивів Linux інтерпретатор Python 3 також запускається командою `python3`. Символ `$` у прикладах документації часто позначає запрошення оболонки; його не треба копіювати.

Що однакове

Сам файл `robot.py` однаковий на всіх трьох системах. Зазвичай відрізняються команда запуску, вигляд шляху й назва оболонки. Після активації проєктного віртуального середовища команда часто стає однаковою — `python robot.py`; до цього повернемося перед зовнішніми пакетами.

Якщо термінал «не бачить» файл

Команда `python robot.py` шукає `robot.py` у поточній папці терміналу. Якщо файл лежить в іншій папці, Python повідомить, що не може його відкрити.

Перевір дві речі:

- 1 У лівій панелі VS Code файл справді має назву `robot.py`, а не `robot.py.txt`.
- 2 Рядок терміналу показує папку, у якій цей файл лежить.

Команда `pwd` показує поточну папку в PowerShell, macOS і більшості Linux-оболонки. Команда `dir` зручна у Windows, а `ls` — у macOS та Linux; PowerShell розуміє обидві форми.

Файл, скрипт і інтерактивний режим

Python можна використовувати двома близькими способами.

Скрипт — збережений файл із послідовністю команд. Його можна запускати багато разів, редагувати, надсилати іншій людині й зберігати в Git.

REPL — інтерактивний режим, у якому Python читає одну команду, виконує її та одразу показує результат. Назва складається з англійських слів *read, evaluate, print, loop*: прочитати, обчислити, показати, повторити.

Щоб відкрити REPL у терміналі, введи лише команду інтерпретатора:

```
powershell
python
```

або на macOS/Linux:

```
bash
python3
```

Побачиш запрошення `>>>`. Тут зручно швидко перевірити вираз:

```
python
>>> 2 + 3
5
>>> "ро" + "бот"
'робот'
```

Символи `>>>` не є частиною Python-коду. Вони лише показують, що інтерактивний режим чекає на команду. Для виходу введи `exit()`.

КОЛИ ЩО ВИКОРИСТОВУВАТИ

REPL зручний для маленького експерименту на один-два рядки. Усі приклади, які потрібно зберегти, розширити або перевірити повторно, пиши у `.py`-файлах.

ПОВЕРНИ ПРОГРАМУ В РОБОЧИЙ ПОРЯДОК

Рядки нижче перемішані. Перепиши їх у правильному порядку й віднови відступи. Спочатку має з'явитися опис, потім об'єкт, а вже після цього — команда об'єкту.

```
python

robot.say_hello()
    def say_hello(self):
robot = Robot()
    print("Привіт!")
class Robot:
```

Очікуваний результат після запуску: один рядок Привіт!.

Типова помилка

Приберемо двокрапку після назви класу й запустимо файл кнопкою **Run Python File**:

```
robot_broken.py

class Robot
    def say_hello(self):
        print("Привіт!")
```

У терміналі з'явиться справжній traceback. Початок шляху нижче скорочено до `...`, бо на кожному комп'ютері він інший:

```
Результат

File "...\\robot_broken.py", line 1
class Robot
    ^
SyntaxError: expected ':'
```

Розберемо його зверху вниз.

- 1 `File "...\\robot_broken.py", line 1` — Python називає файл і рядок, який не зміг прочитати.
- 2 `class Robot` — копія проблемного рядка з нашого файла.
- 3 Символ `^` показує місце, біля якого читання зламалося. Він допомагає шукати, але не завжди вказує точну причину помилки.

- 4 `SyntaxError` — тип помилки: код порушує правила запису Python.
- 5 `expected ':'` — коротка причина: після `class Robot` інтерпретатор очікував двокрапку.

Поверни двокрапку:

```
python

class Robot:
```

Інший частий збій — відсутній відступ перед методом:

```
robot_missing_indent.py

class Robot:
def say_hello(self):
    print("Привіт!")
```

Після запуску бачимо інше повідомлення:

```
Результат

File "...\\robot_missing_indent.py", line 2
def say_hello(self):
    ^^^
IndentationError: expected an indented block after class definition on line 1
```

Тут `line 2` і рядок `def say_hello(self):` показують місце, де Python помітив проблему. `^^^` підкреслює початок `def`, але останній рядок пояснює причину точніше: після опису класу в рядку 1 очікувався блок із відступом. Отже, додаємо чотири пробіли перед `def`, а рядок `print(...)` зсуваємо ще на чотири.

VS Code може підкреслити ці місця червоним ще до запуску. Проте завжди прочитай і повідомлення інтерпретатора в терміналі: воно показує, що сталося під час справжньої спроби виконати файл. Виправляй не весь код навімання, а перший власний рядок, на який вказує `traceback`, і перевіряй останній рядок повідомлення.

ТИПОВА ПОМИЛКА

Не називай файл `robot.py.py`, `robot.py.txt` або `my_robot.py`. Увімкни показ розширень файлів у системі, а для назв Python-модулів використовуй малі латинські літери та підкреслення: `robot.py`, `music_player.py`.

Швидка перевірка

Перед самостійною роботою поясни вголос кожен рядок:

```
door.py

class Door:
    def open(self):
        print("Двері відчиняються.")

door = Door()
door.open()
```

Перевір себе:

- Door — опис категорії об'єктів;
- door — один конкретний об'єкт;
- open — дія цього об'єкта;
- крапка обирає дію, що належить об'єкту;
- відступи показують, які рядки належать класу й методу.

ПЕРЕВІР СЕБЕ

Який рядок створює конкретний об'єкт за описом класу Robot?

- robot = Robot()
- class Robot:
- def robot(self):

Відповідь: robot = Robot()

Клас уже має бути описаний. Виклик Robot() створює його екземпляр, а ім'я robot зберігає посилання на цей об'єкт.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий код і назви чотири різні речі: файл, клас, об'єкт і метод. Потім одним реченням поясни, чому сам class Robot: нічого не друкує.

Самостійна робота

Виконуй завдання по одному. Після кожної зміни запускай файл і порівнюй результат зі своїм прогнозом.

САМОСТІЙНА РОБОТА

- Зміни текст привітання робота. Перед запуском запиши, який рядок має з'явитися в терміналі.
- Додай до `Robot` метод `wave()`, який виводить Робот махає рукою.. Виклич спочатку `say_hello()`, потім `wave()`.
- Створи два об'єкти `first_robot` і `second_robot`. Виклич обидва методи для кожного робота.
- Навмисно видали одну двокрапку, запусти код, знайди тип помилки й номер рядка, а потім віднови робочу версію.
- Скопіюй найменший робочий приклад у файл `robot.txt` і явно передай його інтерпретатору. Поясни, що залишилося робочим, а що VS Code перестав розпізнавати автоматично.
- Придумай власний клас із реального або вигаданого світу. Дай йому одну дію, створи об'єкт і виклич метод. Не додавай властивості — це завдання наступного розділу.

КОРОТКА ІСТОРИЧНА ПАУЗА

Назва Python з'явилася не через змію. Коли Гвідо ван Россум починав роботу над мовою, він читав сценарії британського комедійного шоу *Monty Python's Flying Circus* і шукав коротку, незвичну назву. Тому в навчальних прикладах Python часто трапляється легкий гумор.

Підсумок

- `.py` — звичайний текстовий файл із домовленим розширенням для Python-коду.
- VS Code допомагає писати й запускати, але код виконує інтерпретатор Python.
- Клас описує категорію об'єктів, а виклик на кшталт `Robot()` створює конкретний об'єкт.
- Метод — дія об'єкта; крапка в `robot.say_hello()` обирає цю дію.
- Відступи в Python передають структуру, тому вони впливають на виконання.
- На Windows типовою командою є `python`, додатково може працювати `py`; на macOS і Linux зазвичай використовують `python3`.
- `Traceback` — не покарання, а карта до місця, де інтерпретатор перестав розуміти програму.

Офіційні орієнтири: як [VS Code запускає Python-файл](#), як [Python працює у Windows](#), як [Python працює у macOS](#), як [Python працює на Unix-платформах](#).

РОЗДІЛ 02

Стан об'єкта: рядки, числа й логічні значення

У попередньому розділі два роботи поводитися однаково. Тепер кожен об'єкт отримає власні дані: ім'я, запас енергії та ознаку готовності до роботи. Такі дані називаються **станом об'єкта**.

Що зробимо

Створимо клас `Robot`, якому під час народження передають ім'я й початковий заряд. Робот зможе показати свій стан, витратити енергію та зарядитися.

РЕЗУЛЬТАТ

Після запуску два об'єкти одного класу покажуть різні імена й різний заряд. Зміна одного робота не зачепить другого.

ЗАРАЗ У ФОКУСІ

Одна дія: навчися читати стан як набір підписаних значень: ім'я атрибута відповідає на «що це?», а значення — на «який стан зараз?».

Якщо відволікся, знайди `self.energy`: це підписане число всередині конкретного об'єкта.

Змінна зберігає значення

Почнемо не з класу, а з трьох коротких рядків:

```
python

robot_name = "Робі"
energy = 80
is_ready = True
```

Зліва стоять **імена змінних**, справа — значення. Знак `=` тут означає не «дорівнює», а «прив'яжи ім'я до значення».

- `"Робі"` — рядок `str`, тобто текст;
- `80` — ціле число `int`;
- `True` — логічне значення `bool`, тобто «так»;
- число `2.5` мало б тип `float`, тобто число з дробовою частиною.

Python визначає тип за самим значенням. Перевірити припущення можна функцією `type()`:

СПОЧАТКУ ПЕРЕДБАЧ

До запуску підпиши тип кожного значення: "Робі", 80, True, 2.5. Запиши чотири назви типів у тому порядку, в якому вони мають з'явитися.

value_types.py

```
robot_name = "Робі"
energy = 80
is_ready = True
speed = 2.5

print(type(robot_name).__name__)
print(type(energy).__name__)
print(type(is_ready).__name__)
print(type(speed).__name__)
```

ЯК ЗНАЧЕННЯ ОТРИМУЮТЬ ІМЕНА Й ТИПИ

ДО

Імена `robot_name`, `energy`, `is_ready`, `speed` ще ні на що не посилаються.

→

ПІСЛЯ

Термінал показує `str`, `int`, `bool`, `float`; початкові значення не змінилися.

1	Рядок	<code>robot_name = "Робі"</code>	Ім'я посилається на текстове значення типу <code>str</code> .
2	Ціле число	<code>energy = 80</code>	Ім'я посилається на ціле число типу <code>int</code> .
3	Логічний стан	<code>is_ready = True</code>	Ім'я посилається на логічне значення типу <code>bool</code> .
4	Дробове число	<code>speed = 2.5</code>	Ім'я посилається на число типу <code>float</code> .
5	Перевірка	<code>type(value).__name__</code>	<code>type()</code> знаходить тип самого значення, а <code>__name__</code> дає його коротку назву.

Що тут важливо: Змінна або атрибут має ім'я, але тип належить значенню, на яке це ім'я зараз посилається.

`type()` корисна для дослідження, але в готовій програмі не треба друкувати тип кожної змінної. Краще давати іменам точний зміст: `energy` зрозуміліше за `number`, а `is_ready` — за `flag1`.

Правила імен

Ім'я змінної або атрибута може містити латинські літери, цифри й підкреслення, але не може починатися з цифри.

```
python

player_name = "Лея"
level_2_score = 150
```

Для звичайних змінних у Python використовують `snake_case`: слова пишуть малими літерами й розділяють підкресленням. Назви класів мають інший стиль — `PascalCase`: `Robot`, `GameHero`, `MusicPlayer`.

Не використовуй як власні імена слова, що вже мають роль у Python: `class`, `def`, `True`, `False`, `None`, `if`. VS Code зазвичай підсвічує їх як ключові слова.

Майстерня

Набери першу повну версію у файлі `robot_state.py`:

```
robot_state.py

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy
        self.is_ready = True

    def show_status(self):
        print(f"{self.name}: {self.energy}%")

    def move(self):
        self.energy -= 15

first_robot = Robot("Робі", 80)
second_robot = Robot("Бін", 35)

first_robot.show_status()
second_robot.show_status()

first_robot.move()

first_robot.show_status()
```

robot_state.py · продовження

```
second_robot.show_status()
```

Очікуваний результат:

Результат

```
Робі: 80%
Бін: 35%
Робі: 65%
Бін: 35%
```

Останні два рядки доводять важливу річ: після руху змінився тільки `first_robot`. Клас спільний, але стан у кожного об'єкта власний.

`__init__` готує новий об'єкт

python

```
def __init__(self, name, energy):
    self.name = name
    self.energy = energy
    self.is_ready = True
```

Метод `__init__` Python викликає автоматично після створення об'єкта. Його назву починають і завершують по два символи підкреслення. Такі спеціальні імена не вигадують навмання — вони передбачені мовою.

У виклику `Robot("Робі", 80)`:

- "Робі" потрапляє в параметр `name`;
- 80 потрапляє в параметр `energy`;
- сам новий об'єкт Python передає в `self` автоматично.

Рядок `self.name = name` читається так: «у цього об'єкта створи атрибут `name` і запиши туди отримане ім'я».

Атрибут — значення, яке належить об'єкту. До нього звертаються через крапку: `first_robot.name`, `second_robot.energy`.

ЧОМУ ІМЕНА ПОВТОРЮЮТЬСЯ

У `self.energy = energy` праве `energy` — тимчасовий параметр методу, а `self.energy` — довготривалий атрибут об'єкта. Після завершення `__init__` параметр більше не потрібен, а атрибут лишається разом з об'єктом.

Рядки й f-рядки

Текст у Python беруть в одинарні або подвійні лапки:

python

```
first_message = "Привіт"
second_message = 'Робот готовий'
```

Обидва варіанти рівноправні. Обирай той, з яким текст читається легше. Наприклад, апостроф зручно залишити всередині подвійних лапок:

```
python

message = "Об'єкт зберігає власний стан."
```

Літера **f** перед рядком дозволяє вставляти значення в фігурних дужках:

```
python

print(f"{self.name}: {self.energy}%")
```

Такий текст називається **f-рядком**. Python обчислює вміст `{ . . }` і перетворює результат на текст. Тому рядок та число можна показати разом без ручного перетворення.

Корисні методи рядків:

```
string_methods.py

name = "роби"
model = " Космо Бот  "

print(name.upper())
print(name.lower())
print(name.title())
print(model.strip())
```

- `upper()` створює версію великими літерами;
- `lower()` — малими;
- `title()` піднімає першу літеру кожного слова;
- `strip()` прибирає пробіли на початку й у кінці.

Рядок при цьому не змінюється сам. Метод повертає нове значення. Щоб зберегти його, потрібне присвоєння: `name = name.title()`.

Числа змінюють стан

Python підтримує звичні дії:

```
python

total = 10 + 5
left = 10 - 3
doubled = 7 * 2
half = 9 / 2
whole_part = 9 // 2
remainder = 9 % 2
power = 3 ** 2
```

`/` завжди дає `float`; `//` залишає цілу частину ділення; `%` дає остачу; `**` підносить до степеня.

У методі руху ми використали скорочене присвоєння:

```
python

self.energy -= 15
```

Це коротка форма `self.energy = self.energy - 15`. Так само працюють `+=`, `*=`, `/=`.

Дробові числа в комп'ютері іноді мають крихітну похибку:

```
float_example.py

result = 0.1 + 0.2

print(result)
print(round(result, 2))
```

Це не поломка Python: багато десяткових дробів неможливо точно записати обмеженою кількістю двійкових цифр. Для показу можна використати `round()`. Для грошей у серйозних програмах застосовують окремі правила або спеціальний тип `Decimal`.

Логічне значення відповідає «так» або «ні»

`True` і `False` пишуться з великої літери й без лапок:

```
python

self.is_ready = True
```

Якщо написати "True", це буде звичайний текст, а не логічне значення. Логічні атрибути зручно називати як коротке запитання: `is_ready`, `has_key`, `can_fly`.

Поки ми лише зберігаємо `is_ready`. У розділі про умови програма почне приймати рішення на основі цього значення.

ДОДАЙ ЗАРЯДЖАННЯ

Встав у клас метод:

```
python

def charge(self, amount):
    self.energy += amount
```

Після створення робота виконай `first_robot.charge(10)` і знову покажи стан. Перед запуском самостійно порахуй очікуваний заряд.

Коментарі пояснюють причину

Коментар починається із #. Python ігнорує текст після цього символу до кінця рядка.

```
python

# На один крок робот витрачає 15 одиниць енергії.
self.energy -= 15
```

Корисний коментар пояснює **чому** рішення саме таке. Коментар # Віднімаємо 15 лише повторює код і майже нічого не додає.

Коли під час дослідження хочеться тимчасово вимкнути рядок, можна поставити перед ним #. Але не накопичуй у готовому файлі великі шматки «мертвого» коду — Git краще зберігає історію змін.

ДОПОВНИ СТАН РОБОТА

Перепиши приклад і заміни пропуски так, щоб атрибути отримали аргументи конструктора.

```
python

class Robot:
    def __init__(self, name, energy, is_ready):
        self.name = ____
        self.energy = ____
        self.is_ready = ____

robot = Robot("Робі", 80, True)
```

Після створення об'єкта перевір окремими `print()`: ім'я, енергію та готовність.

Типова помилка

Атрибут створено не для того об'єкта

Такий метод не змінює заряд робота. Запустимо повний мінімальний приклад; у traceback початок шляху скорочено до ...:

```
missing_self.py

class Robot:
    def __init__(self):
        self.energy = 100

    def move(self):
        energy -= 15

robot = Robot()
robot.move()
```


Результат

```
Traceback (most recent call last):
  File "...\\missing_self.py", line 10, in <module>
    robot.move()
  File "...\\missing_self.py", line 6, in move
    energy -= 15
    ^^^^^
UnboundLocalError: cannot access local variable 'energy' where it is not associated with a value
```

Присвоєння `energy -= 15` робить `energy` локальним ім'ям методу, але початкового локального значення немає. Атрибут `self.energy` існує, проте це інше ім'я. Потрібно звернутися до стану поточного об'єкта:

python

```
def move(self):
    self.energy -= 15
```

Атрибут прочитано до створення

missing_attribute.py

```
class Robot:
    def __init__(self, name):
        self.name = name

    def show_status(self):
        print(self.energy)

robot = Robot("Робі")
robot.show_status()
```

Результат

```
Traceback (most recent call last):
  File "...\\missing_attribute.py", line 10, in <module>
    robot.show_status()
  File "...\\missing_attribute.py", line 6, in show_status
    print(self.energy)
    ^^^^^^^^^^^^^
AttributeError: 'Robot' object has no attribute 'energy'
```

Останній рядок називає відсутній атрибут, а кадр `show_status` показує читання, на якому це стало помітно. Причина виникла раніше: `energy` не створили в `__init__`. Найнадійніше готувати обов'язкові атрибути під час створення об'єкта.

ТИПОВА ПОМИЛКА

Не змінюй значення різних типів без розуміння. Після `self.energy = "вісімдесят"` віднімання `self.energy -= 15` закінчиться `TypeError`: текст і число не можна відняти одне від одного.

Швидка перевірка

Прочитай код, спрогнозуй обидва рядки, а тоді запусти:

```
pet_state.py

class Pet:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def play(self):
        self.energy -= 1

    def show(self):
        print(f"{self.name}: {self.energy} од. енергії.")

cat = Pet("Луна", 5)
dog = Pet("Топ", 9)

cat.play()
cat.show()
dog.show()
```

Перевір, чи можеш пояснити:

- чому `cat` і `dog` мають спільні методи;
- де зберігається ім'я конкретної тварини;
- чому `cat.play()` не змінює `dog.energy`;
- який тип мають `name` та `energy`.

ПЕРЕВІР СЕБЕ

Що робить рядок `self.energy = energy` всередині `__init__`?

- Порівнює атрибут із параметром.
- Записує отримане значення `energy` в атрибут поточного об'єкта.
- Створює одну спільну енергію для всіх об'єктів.

Відповідь: Записує отримане значення `energy` в атрибут поточного об'єкта.

Ліворуч `self.energy` означає атрибут конкретного об'єкта, а праворуч `energy` — параметр, переданий під час створення.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Не дивлячись у код, наведи по одному власному прикладу `str`, `int`, `float` і `bool`. Потім поясни, чим `energy` відрізняється від `robot.energy`.

Самостійна робота

САМОСТІЙНА РОБОТА

- Створи клас `Book` з атрибутами `title`, `pages` і `is_open`. Додай метод, який показує всі три значення зрозумілим реченням.
- Створи два об'єкти `GameHero` з різними іменами та запасом здоров'я. Зміни здоров'я лише першого й доведи виведенням, що другий не змінився.
- Додай роботу атрибут `distance = 0`. Нехай кожен виклик `move()` збільшує відстань на 3 і зменшує енергію на 15.
- Досліди методи `replace()` і `removeprefix()` для рядків. Збережи результат в окремій змінній і поясни, чи змінився початковий рядок.
- Створи клас `Temperature` з атрибутом `celsius`. Додай метод, що друкує температуру за Цельсієм і приблизне значення за Фаренгейтом за формулою $celsius * 9 / 5 + 32$.
- Навмисно звернися до неіснуючого атрибута, прочитай останній рядок `traceback`, а потім виправ клас так, щоб атрибут створювався в `__init__`.

КОРОТКА ІСТОРИЧНА ПАУЗА

Слово «об'єкт» у програмуванні не означає лише видиму річ. Об'єктом може бути рахунок, музичний трек, температура або правило гри — будь-що, для чого програмі зручно тримати дані й пов'язані з ними дії разом.

Підсумок

- Змінна прив'язує ім'я до значення, `a =` виконує присвоєння.
- Базові типи цього розділу: текст `str`, цілі числа `int`, дробові `float`, логічні значення `bool`.
- `__init__` готує новий об'єкт і зазвичай створює його обов'язкові атрибути.
- `self.attribute` звертається до стану конкретного об'єкта.
- Об'єкти одного класу мають спільний опис поведінки, але окремий стан.
- `f`-рядок вставляє значення в текст, а скорочення `+=` і `-=` оновлюють числа.
- Добре ім'я пояснює зміст, а добрий коментар — причину рішення.

РОЗДІЛ 03

Методи: параметри, аргументи й return

Об'єкт уже вміє зберігати стан. Тепер навчимо його приймати команди з додатковими даними, перевіряти результат своєї роботи та повертати значення тому коду, який викликав метод.

Що зробимо

Створимо героя, який отримує лікування й витрачає енергію на удар. Одні методи змінюватимуть стан, інші — обчислюватимуть і **повертатимуть** результат без друку.

РЕЗУЛЬТАТ

Після розділу ти зможеш простежити шлях значення: від аргументу у виклику — до параметра методу — до нового стану або результату return.

ЗАРАЗ У ФОКУСІ

Одна дія: щоразу показуй пальцем на два різні місця: значення у виклику та ім'я для нього у визначенні методу.

Якщо відволікся, повернися до пари `hero.heal(15)` і `def heal(self, amount):`.

Команді часто потрібні додаткові дані

Метод `robot.move()` може завжди рухати робота на однакову відстань. Але команда `hero.heal(15)` повинна знати, скільки саме здоров'я відновити.

Спершу знайди **два різні місця** у програмі.

Місце 1. Загальний опис методу

```
python

class Hero:
    def heal(self, amount):
        self.health += amount
```

`amount` стоїть у **визначенні** методу. Це **параметр** — ім'я, через яке метод працюватиме з майбутнім значенням. Тут ще не вирішено, буде це 5, 15 чи 100.

Місце 2. Один конкретний виклик

```
python
```

```
hero.heal(15)
```

15 стоїть у **виклику**. Це **аргумент** — конкретне значення саме для цього запуску методу.

Порівняй кілька викликів одного опису:

```
python
```

```
hero.heal(5)
hero.heal(15)
hero.heal(100)
```

Параметр у визначенні лишається один — `amount`. Аргумент у кожному виклику інший — 5, 15 або 100. Під час кожного виклику Python тимчасово робить відповідне зіставлення: `amount = 5`, потім `amount = 15`, потім `amount = 100`.

Коротко:

- **параметр** — ім'я в загальному описі методу;
- **аргумент** — значення або вираз у конкретному виклику;
- `self` теж параметр, але об'єкт для нього Python підставляє автоматично.

Майстерня

Створи файл `hero_actions.py`:

СПОЧАТКУ ПЕРЕДБАЧ

У Міри спочатку 70 здоров'я і 25 енергії. Не запускаючи код, визнач її стан після `heal(15)` та `strike(12)`, а також значення, яке поверне `strike()`.

```
hero_actions.py
```

```
class Hero:
    def __init__(self, name, health=100, energy=30):
        self.name = name
        self.health = health
        self.energy = energy

    def heal(self, amount):
        self.health += amount

    def strike(self, damage, energy_cost=8):
        self.energy -= energy_cost
        return damage

    def status_text(self):
        return f"{self.name}: {self.health} здоров'я, {self.energy} енергії."
```

hero_actions.py · продовження

```

hero = Hero("Міра", health=70, energy=25)

print(hero.status_text())
hero.heal(15)
dealt_damage = hero.strike(12)

print(f"Удар завдав {dealt_damage} шкоди.")
print(hero.status_text())

```

АРГУМЕНТ СТАЄ ЗНАЧЕННЯМ ПАРАМЕТРА

ДО
 hero.health = 70; ім'я amount **ще не**
має значення для цього виклику.

→

ПІСЛЯ
 hero.health = 85; після
завершення методу локальне ім'я
amount більше не потрібне.

1	Місце виклику	hero.heal(15)	15 — аргумент: конкретне значення, яке передає цей виклик.
2	Загальний опис	def heal(self, amount):	amount — параметр: ім'я, під яким метод прийматиме різні значення.
3	Зіставлення	amount = 15	Під час цього виклику Python пов'яже параметр amount з аргументом 15.
4	Зміна стану	self.health += amount	Для цього об'єкта виконується 70 + 15.

Що тут важливо: Аргумент живе у конкретному виклику, параметр — у загальному описі методу; під час виклику Python тимчасово з'єднує їх.

Тут є три різні види роботи:

- heal() змінює стан;
- strike() змінює стан і повертає число;
- status_text() не змінює стан, а складає й повертає текст.

Значення за замовчуванням

У заголовку __init__ два параметри мають готові значення:

```

python

def __init__(self, name, health=100, energy=30):

```

Тому всі ці виклики правильні:

```
python
```

```
first = Hero("Міпа")
second = Hero("Орест", 80)
third = Hero("Лея", 70, 50)
```

- first отримає health=100, energy=30;
- second отримає health=80, energy=30;
- third отримає всі передані значення.

Обов'язкові параметри ставлять перед параметрами зі значенням за замовчуванням. Такий заголовок був би помилковим: `def __init__(self, health=100, name):`.

СТАБІЛЬНЕ ЗНАЧЕННЯ ЗА ЗАМОВЧУВАННЯМ

Для числа, рядка, True, False або None значення за замовчуванням безпечне. Змінні списки й словники потребують іншого прийому; ми розберемо його після знайомства з колекціями.

Позиційні й іменовані аргументи

У виклику `Hero("Міпа", 70, 25)` Python зіставляє значення з параметрами за позицією. Це **позиційні аргументи**.

```
python
```

```
hero = Hero("Міпа", health=70, energy=25)
```

`health=70` та `energy=25` — **іменовані аргументи**. Вони довші, але не дають переплутати два схожих числа.

Позиційні аргументи мають стояти перед іменованими:

```
python
```

```
hero = Hero("Міпа", energy=25, health=70)
```

Порядок двох іменованих аргументів уже не важливий.

ПЕРЕВІР РІЗНІ ВИКЛИКИ

Створи трьох героїв:

```
hero_defaults.py

class Hero:
    def __init__(self, name, health=100, energy=30):
        self.name = name
        self.health = health
        self.energy = energy

    def short_status(self):
        return f"{self.name}: {self.health}/{self.energy}"

first = Hero("Ада")
second = Hero("Бор", 60)
third = Hero("Біпа", energy=45, health=90)

print(first.short_status())
print(second.short_status())
print(third.short_status())
```

Поясни, звідки взялося кожне число: з аргументу чи зі значення за замовчуванням.

return передає результат назовні

Порівняй два методи:

```
python

def print_status(self):
    print(f"Здоров'я: {self.health}")

def status_text(self):
    return f"Здоров'я: {self.health}"
```

Перший сам вирішує, що результат треба показати в терміналі. Другий лише готує текст і віддає його виклику. Тому з `status_text()` можна зробити більше:

```
python

message = hero.status_text()
print(message)
save_later = message
```

Пізніше цей текст можна буде показати у графічному інтерфейсі, записати у файл або перевірити тестом. Метод не прив'язаний до одного способу використання.

Рядок із `return` одразу завершує метод:

```
python

def double(self, number):
    return number * 2
    print("Цей рядок не виконається.")
```

Код після виконаного `return` у тому самому блоці недосяжний.

Метод без явного `return` повертає `None`

```
none_result.py

class Hero:
    def heal(self, amount):
        print("Лікування завершено.")

hero = Hero()
result = hero.heal(10)
print(result)
```

`None` означає «корисного значення тут немає». Це окремий об'єкт Python, а не нуль, не порожній рядок і не `False`.

Не треба додавати `return None` наприкінці кожного методу: Python робить це неявно. Явний `return` потрібен, коли ми справді хочемо передати результат або достроково завершити роботу.

Обчислення краще відділяти від виведення

```
travel_cost.py

class Journey:
    def __init__(self, price_per_step):
        self.price_per_step = price_per_step

    def calculate_cost(self, steps):
        return self.price_per_step * steps

journey = Journey(price_per_step=3)
cost = journey.calculate_cost(12)
print(f"Подорож коштує {cost} монет.")
```

Метод рахує, а зовнішній код вирішує, як використати число. Це простий приклад **поділу відповідальності**: кожна частина має одну зрозумілу роботу.

Передавання об'єкта методу

Аргументом може бути не лише число або рядок, а й інший об'єкт.

```
training.py

class Hero:
    def __init__(self, name):
        self.name = name
        self.experience = 0

    def train(self, student, points):
        student.experience += points
        return f"Тренування: {self.name} -> {student.name}."

    def experience_text(self):
        return f"{self.name} отримує {self.experience} досвіду."

mentor = Hero("Ніка")
student = Hero("Рей")

print(mentor.train(student, 3))
print(student.experience_text())
```

Усередині `train()` параметр `student` посилається на той самий об'єкт, що й зовнішнє ім'я `student`. Тому зміна `student.experience` зберігається після завершення методу.

Це потужна можливість, але користуйся нею зрозуміло: назва методу має підказувати, що стан іншого об'єкта зміниться.

РОЗДІЛИ ДВА МІСЦЯ НАВМИСНО

Заповни пропуски різними словами або числами. У першому пропуску має бути **ім'я параметра**, у другому — **аргумент конкретного виклику**.

```
python

class Hero:
    def heal(self, ____):
        self.health += ____

hero.heal(____)
```

Перевір себе: перші два пропуски мають збігатися, третій може бути будь-яким доречним числом.

Типова помилка

Виклик не відповідає параметрам

```
missing_argument.py
```

```
class Hero:
    def heal(self, amount):
        print(f"Відновлено: {amount}")

hero = Hero()
hero.heal()
```

Результат

```
Traceback (most recent call last):
  File "...\\missing_argument.py", line 7, in <module>
    hero.heal()
TypeError: Hero.heal() missing 1 required positional argument: 'amount'
```

Останній рядок називає метод і параметр `amount`, для якого не передали значення.

```
too_many_arguments.py
```

```
class Hero:
    def heal(self, amount):
        print(f"Відновлено: {amount}")

hero = Hero()
hero.heal(10, 20)
```

Результат

```
Traceback (most recent call last):
  File "...\\too_many_arguments.py", line 7, in <module>
    hero.heal(10, 20)
TypeError: Hero.heal() takes 2 positional arguments but 3 were given
```

У повідомленні враховано й прихований для виклику `self`: метод має два параметри `self` та `amount`, але отримав об'єкт плюс два числа — разом три аргументи.

Результат надруковано двічі

```
printed_none.py
```

```
class Hero:
    def status_text(self):
        print("Герой готовий.")

hero = Hero()
print(hero.status_text())
```

Результат

Герой готовий.
None

Метод спочатку друкує повідомлення, а потім неявно повертає None; зовнішній `print()` показує саме це повернене значення. Якщо зовнішній код має друкувати результат, метод повинен **повертати** рядок:

python

```
def status_text(self):  
    return "Герой готовий."
```

ТИПОВА ПОМИЛКА

Не пиши `return print(...)`, коли хочеш повернути текст. `print()` показує значення й повертає None. Спочатку склади рядок, поверни його через `return`, а друкуй у місці виклику.

Швидка перевірка

Спочатку визнач, які значення потраплять у параметри `minutes` і `intensity`, і лише тоді запускай:

music_player.py

```
class MusicPlayer:  
    def __init__(self, battery=60):  
        self.battery = battery  
  
    def play(self, minutes, intensity=1):  
        used = minutes * intensity  
        self.battery -= used  
        return used  
  
    def battery_text(self):  
        return f"Залишилося {self.battery} хв."  
  
player = MusicPlayer()  
spent = player.play(minutes=9, intensity=2)  
  
print(spent)  
print(player.battery_text())
```

ПЕРЕВІР СЕБЕ

Навіщо метод `status_text()` повертає рядок, а не одразу друкує його?

- Щоб атрибути об'єкта стали глобальними змінними.
- Щоб код, який викликав метод, сам вирішив, як використати готовий текст.
- Бо методи класу не можуть використовувати `print()`.

Відповідь: Щоб код, який викликав метод, сам вирішив, як використати готовий текст. `return` віддає значення назовні. Його можна надрукувати, зберегти, записати у файл або перевірити тестом.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий сторінку й закінчи дві фрази: «Параметр — це...» та «Аргумент — це...». Потім наведи власну пару з одного визначення методу й одного його виклику.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай герою метод `rest(minutes)`, який відновлює по 2 одиниці енергії за хвилину й повертає фактично додану кількість.
- Створи клас `Calculator` з методами `add(a, b)`, `subtract(a, b)` і `square(number)`. Усі методи мають повертати значення, а друк зроби поза класом.
- Створи клас `Ticket` з обов'язковою назвою події та ціною за замовчуванням 100. Створи три квитки різними поєднаннями позиційних та іменованих аргументів.
- Перероби метод, який одночасно рахує і друкує вартість, на два кроки: метод повертає число, зовнішній код формує повідомлення.
- Створи два об'єкти `Pet`. Нехай метод одного об'єкта `share_food(other, amount)` зменшує його запас їжі й збільшує запас іншого.
- Навмисно виклич метод без обов'язкового аргументу. Знайди в останньому рядку `traceback` назву методу та опис відсутнього параметра, після чого виправ виклик.

КОРОТКА ІСТОРИЧНА ПАУЗА

Слово `return` буквально означає «повернути». Значення ніби проходить усередину методу через аргументи, опрацьовується, а потім повертається до місця виклику. Ця схема існувала в мовах задовго до Python і лишається одним з головних способів складати великі програми з малих частин.

Підсумок

- Параметр описує місце для значення, аргумент передає конкретне значення під час виклику.
- `self` Python передає автоматично, решту обов'язкових аргументів передає програміст.
- Значення за замовчуванням робить аргумент необов'язковим.
- Іменовані аргументи пояснюють зміст і не залежать від взаємного порядку.
- `return` передає результат коду, який викликав метод, і завершує метод.
- Метод без явного `return` повертає `None`.
- Обчислення, зміна стану й показ результату — різні відповідальності; їх корисно бачити окремо.
- Аргументом може бути інший об'єкт, і метод може взаємодіяти з його станом.

РОЗДІЛ 04

Рішення: порівняння, if, elif, else

До цього методи виконували однакові дії за будь-якого стану. Але двері не мають відчинятися без ключа, герой не може витратити більше енергії, ніж має, а завершене завдання не треба завершувати вдруге. Для цього програма вчиться ставити запитання й обирати гілку дій.

Що зробимо

Створимо розумні двері. Вони знатимуть правильний код, кількість невдалих спроб і свій стан. Метод `try_open()` повертатиме різні повідомлення залежно від введеного коду.

РЕЗУЛЬТАТ

Один виклик методу зможе піти різними шляхами. Ти навчишся читати умову як запитання, результатом якого є лише `True` або `False`.

ЗАРАЗ У ФОКУСІ

Одна дія: перед кожною умовою сформулюй питання, на яке Python відповість лише `True` або `False`.

Якщо відволікся, повернися до поточного значення `energy` і прочитай порівняння як звичайне запитання.

Порівняння створює логічне значення

СПОЧАТКУ ПЕРЕДБАЧ

Для `energy = 20` прочитай чотири порівняння зверху вниз і запиши чотири відповіді `True` або `False`. Лише після цього запускай файл.

```
comparisons.py
```

```
energy = 20

print(energy == 20)
print(energy == 10)
print(energy > 5)
print(energy != 0)
```

ПОРІВНЯННЯ ЧИТАЮТЬ СТАН, АЛЕ НЕ ЗМІНЮЮТЬ ЙОГО

ДО
energy = 20; у терміналі ще немає
відповідей.

→

ПІСЛЯ
energy досі дорівнює 20; термінал
містить чотири логічні відповіді.

1	Рівність	energy == 20	Питання «20 дорівнює 20?» дає True.
2	Інша рівність	energy == 10	Питання «20 дорівнює 10?» дає False.
3	Більше	energy > 5	Питання «20 більше 5?» дає True.
4	Нерівність	energy != 0	Питання «20 не дорівнює 0?» дає True.

Що тут важливо: Порівняння обчислює нове логічне значення, але саме по собі не переприводить стан.

Кожен вираз повертає True або False.

- == — дорівнює;
- != — не дорівнює;
- > — більше;
- < — менше;
- >= — більше або дорівнює;
- <= — менше або дорівнює.

Один знак = **записує** значення. Два знаки == **порівнюють**. Це різні операції.

Рядки теж можна порівнювати:

```
string_compare.py

entered_code = "MOON"

print(entered_code == "MOON")
print(entered_code == "moon")
```

Великі й малі літери різняться. Якщо регістр не повинен мати значення, нормалізуй обидві сторони:

```
python

entered_code.strip().lower() == "moon"
```


Майстерня

Створи `smart_door.py`:

```
smart_door.py

class SmartDoor:
    def __init__(self, secret_code):
        self.secret_code = secret_code
        self.is_open = False
        self.failed_attempts = 0

    def try_open(self, entered_code):
        if self.is_open:
            return "Двері вже відчинені."

        if entered_code == self.secret_code:
            self.is_open = True
            return "Двері відчинено."

        self.failed_attempts += 1
        return f"Невірний код. Спроба {self.failed_attempts} з 3."

door = SmartDoor("MOON")

print(door.try_open("STAR"))
print(door.try_open("MOON"))
```

smart_door.py · продовження

```
print(door.try_open("MOON"))
```

Прочитай метод зверху вниз:

- 1 Якщо двері вже відкриті, повертаємо перше повідомлення й виходимо.
- 2 Інакше перевіряємо код.
- 3 Якщо код правильний, змінюємо стан і завершуємо метод.
- 4 Якщо жоден попередній `return` не спрацював, збільшуємо лічильник помилок.

Тут два окремі `if`, бо кожен правильний шлях завершується через `return`. Нижче розберемо конструкцію, де гілки з'єднані явно.

Блок `if`

```
python

if entered_code == self.secret_code:
    self.is_open = True
    return "Двері відчинено."
```

Після `if` стоїть логічний вираз, потім двокрапка. Вкладені рядки виконуються лише тоді, коли вираз дав `True`.

Відступ визначає межу гілки:

```
python

if self.is_open:
    print("Двері відкриті.")
    print("Можна проходити.")

print("Перевірку завершено.")
```

Останній `print()` не має відступу, тому виконається незалежно від умови.

Логічний атрибут уже є умовою

```
python

if self.is_open:
```

Не обов'язково писати `if self.is_open == True:`. Атрибут уже містить логічне значення. Для протилежного запитання використовуй `not`:

```
python

if not self.is_open:
    print("Двері зачинені.")
```

Так код читається майже як коротке англійське речення: «якщо не відкрито».

Пов'язані гілки: `if`, `elif`, `else`

Коли треба вибрати рівно одну категорію, гілки пов'язують:

```
energy_level.py

class Battery:
    def __init__(self, charge):
        self.charge = charge

    def level_text(self):
        if self.charge >= 70:
            return "Запас енергії високий."
        elif self.charge >= 30:
            return "Запас енергії середній."
        else:
            return "Запас енергії низький."

battery = Battery(55)
print(battery.level_text())
```

Python перевіряє умови згори вниз і виконує першу придатну гілку. Решту пропускає.

Порядок важливий. Якби умова `charge >= 30` стояла першою, заряд 80 потрапив би до «середнього» рівня, бо перша умова вже була б істинною.

`elif` можна мати декілька, а `else` не має умови: це шлях «в усіх інших випадках».

ДОДАЙ БЛОКУВАННЯ ДВЕРЕЙ

Розшир метод `try_open()` так, щоб після трьох невдалих спроб двері повертали Доступ заблоковано.. Для цього зручно перевірити `self.failed_attempts >= 3` на самому початку методу.

Кілька умов разом

and: обидві умови мають бути істинними

```
python

if self.has_key and self.energy > 0:
    return "Герой може відчинити браму."
```

or: достатньо хоча б однієї істинної умови

```
python

if self.is_admin or self.has_pass:
    return "Доступ дозволено."
```

not: перетворює істину на хибу й навпаки

```
python

if not self.is_active:
    return "Об'єкт вимкнений."
```

Коли вираз довгий, дужки роблять задум видимим:

```
python

if (self.has_key or self.knows_code) and self.energy > 0:
    return "Прохід відкрито."
```

Спочатку перевіряється дужка: є ключ **або** відомий код. Потім результат поєднується з вимогою мати енергію.

КОРОТКЕ ОБЧИСЛЕННЯ

Python перевіряє логічний вираз лише настільки далеко, наскільки потрібно. У `False and друга_умова` друга частина не змінить результат і не виконується. У `True or друга_умова` — так само. Це називають коротким обчисленням.

Межі й порядок перевірок

Умова має точно описувати межу.

```
python

if age >= 10:
    print("Доступ до рівня відкрито.")
```

Дитина віком рівно 10 років проходить. З умовою `age > 10` — ні. Перед написанням знака скажи правило звичайними словами й перевір значення **на межі**, трохи нижче та трохи вище.

```
level_access.py

class LevelGate:
    def can_enter(self, score):
        return score >= 100

gate = LevelGate()
print("Так" if gate.can_enter(99) else "Ні")
print("Так" if gate.can_enter(100) else "Ні")
print("Так" if gate.can_enter(101) else "Ні")
```

Останні три рядки використовують короткий умовний вираз. Його зручно читати так: «надрукуй Так, якщо умова істинна, інакше Ні». Для складних дій звичайний блок `if` читабельніший.

Істинні та хибні значення

В умові Python може оцінити не лише `bool`.

Хибними вважаються, зокрема:

- `0` і `0.0`;
- порожній рядок `""`;
- порожні колекції, з якими познайомимось далі;
- `None`;
- саме значення `False`.

Інші числа й непорожні рядки зазвичай істинні:

```
truthy_values.py

name = "Робі"
energy = 0

if name:
    print("Є ім'я.")

if not energy:
    print("Енергії немає.")
```

На початку навчання явне порівняння іноді зрозуміліше: `if energy == 0`. Скорочуй лише тоді, коли сенс не губиться.

Перевірка стану без небезпечної зміни

Розглянь героя, який може атакувати лише за достатньої енергії:

```
safe_attack.py

class Hero:
    def __init__(self, energy):
        self.energy = energy

    def attack(self, cost):
        if cost > self.energy:
            return "Не вистачає енергії."

        self.energy -= cost
        return f"Атаку виконано. Залишилося {self.energy}."

hero = Hero(10)
print(hero.attack(12))
print(hero.attack(8))
```

Спочатку метод перевіряє небезпечний випадок і завершується. Лише після цього змінює стан. Такий ранній вихід часто робить метод пласким і читабельним.

СКЛАДИ ОДИН ВЗАЄМОВИКЛЮЧНИЙ ВИБІР

Розташуй гілки так, щоб Python перевіряв найнебезпечніший стан першим і друкував рівно одне повідомлення.

```
python

else:
    print("Енергії достатньо")
if energy <= 0:
    print("Робот вимкнений")
elif energy < 10:
    print("Потрібна зарядка")
```

Перевір з `energy = 0, 5 і 20`.

Типова помилка

Присвоєння замість порівняння

assignment_condition.py

```
entered_code = "1234"
secret_code = "1234"

if entered_code = secret_code:
    print("Відкрито")
```

Результат

```
File "...assignment_condition.py", line 4
    if entered_code = secret_code:
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Рядок із File і номером веде до умови, підкреслення охоплює неправильний вираз, а останній рядок навіть пропонує ==. Тут потрібне порівняння, не присвоєння.

Два незалежні if замість одного вибору

two_if.py

```
score = 120

if score >= 50:
    print("Рівень пройдено")

if score >= 100:
    print("Новий рекорд")
```

Результат

```
Рівень пройдено
Новий рекорд
```

Програма не падає: для score = 120 обидві умови істинні, тому виконуються **обидва** блоки. Це може бути саме тим, що потрібно. Але якщо потрібна лише одна категорія, використовуй if/elif/else.

Стан змінюється до перевірки

negative_energy_bug.py

```
class Robot:
    def __init__(self, energy):
        self.energy = energy

    def use(self, cost):
        self.energy -= cost
        if self.energy < 0:
            return "Не вистачає енергії."
        return "Готово."

robot = Robot(10)
print(robot.use(15))
print(robot.energy)
```

Результат

```
Не вистачає енергії.
-5
```

Це логічний дефект: повідомлення правильне, винятку немає, але другий рядок доводить, що стан уже зіпсований. Спочатку перевіряє можливість, потім змінює енергію.

ТИПОВА ПОМИЛКА

Не порівнюй логічний результат із текстом: `self.is_open == "True"`. Якщо атрибут містить `True` або `False`, перевіряй `if self.is_open:` або `if not self.is_open:`.

Швидка перевірка

Перед запуском визнач результат для кожного учня:

```
lesson_access.py

class Lesson:
    def __init__(self, free_places):
        self.free_places = free_places

    def access_text(self, name, has_permission):
        if self.free_places <= 0:
            return f"{name}: немає місця"
        if not has_permission:
            return f"{name}: потрібна згода"
        return f"{name}: доступ"

lesson = Lesson(1)
print(lesson.access_text("Аня", True))

lesson.free_places = 0
print(lesson.access_text("Богдан", True))

lesson.free_places = 2
print(lesson.access_text("Віра", False))
```

ПЕРЕВІР СЕБЕ

Яка умова дозволяє атаку лише тоді, коли енергії не менше за вартість?

- if self.energy >= cost:
- if self.energy = cost:
- if self.energy < cost:

Відповідь: if self.energy >= cost:

Оператор >= охоплює і більший запас, і точну рівність. Один знак = виконує присвоєння, а < описує протилежний випадок.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без підглядання поясни різницю між =, == і !=. Потім назви випадок, коли потрібні пов'язані if / elif / else, а не два незалежні if.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай SmartDoor метод `close()`. Якщо двері вже зачинені, він має повернути окреме повідомлення й не змінювати інші атрибути.
- Створи клас Player з очками. Метод `rank_text()` має повертати один із чотирьох рангів за діапазонами, які ти визначиш сам. Перевір точні межі.
- Створи клас Lamp з атрибутами `is_on` і `battery`. Дозволяй увімкнення лише за умови, що лампа вимкнена і батарея більша за нуль.
- Напиши метод `can_join(age, has_permission)`, який дозволяє участь повнолітній людині або дитині з дозволом. Перевір щонайменше чотири комбінації.
- Навмисно постав перевірку широкого діапазону перед вузьким у ланцюжку `if/elif`. Побач неправильну категорію, а потім виправ порядок.
- Створи метод, який спершу змінює стан, а потім виявляє заборонений випадок. Перероби його так, щоб перевірка відбувалася до зміни.

КОРОТКА ІСТОРИЧНА ПАУЗА

Слово `elif` — коротка форма англійського «else if», тобто «інакше, якщо». Python навмисно має окреме коротке ключове слово: менше зайвих вкладень — легше побачити всі взаємовиключні варіанти.

Підсумок

- Порівняння повертає `True` або `False`; `=` присвоює, а `==` порівнює.
- `if` виконує вкладений блок лише за істинної умови.
- `elif` додає наступну умову, `else` ловить усі решта випадків.
- Пов'язані гілки обирають перший придатний шлях; окремі `if` можуть виконатися разом.
- `and`, `or` і `not` поєднують або заперечують логічні умови.
- Межі треба перевіряти точним значенням, значенням нижче та значенням вище.
- Ранній `return` зупиняє заборонений сценарій до зміни стану.
- Логічний атрибут можна перевіряти без `== True`.

РОЗДІЛ 05

Списки й кортежі: впорядковані колекції

Окремий атрибут зручно зберігає одне значення. Але наплічник героя містить багато предметів, черга — багато учасників, а координата — кілька пов'язаних чисел. Для таких даних Python має колекції.

Що зробимо

Створимо Васкраск, який зберігає предмети у списку. Навчимося додавати, знаходити, замінювати й забирати елементи, не відкриваючи внутрішню будову об'єкта всьому коду.

РЕЗУЛЬТАТ

Ти зможеш пояснити різницю між порядком, індексом і значенням, а також обрати список для змінних даних або кортеж для фіксованого набору.

ЗАРАЗ У ФОКУСІ

Одна дія: уявляй список як впорядковану полицю, де кожне місце має індекс, а сам список може змінюватися.

Якщо відволікся, повернися до `items[0]` і знайди перший елемент зліва.

Список зберігає послідовність значень

```
python
```

```
items = ["ліхтар", "карта", "ключ"]
```

Квадратні дужки позначають **список** `list`. Елементи зберігають порядок. У цьому списку "ліхтар" стоїть першим, "карта" — другим, "ключ" — третім.

Список може бути порожнім:

```
python
```

```
items = []
```

або містити значення різних типів:

```
python

mixed = ["Робі", 80, True]
```

Python це дозволяє, але однорідна колекція зазвичай зрозуміліша: список предметів, список чисел, список об'єктів. Пов'язані різнотипні дані одного учасника краще тримати в об'єкті.

Індекс починається з нуля

СПОЧАТКУ ПЕРЕДБАЧ

Не запускаючи файл, випиши значення для індексів 0, 1 і -1. Окремо виріши, чи змінять ці три читання сам список `items`.

```
list_indexes.py

items = ["ліхтар", "карта", "ключ"]

print(items[0])
print(items[1])
print(items[-1])
```

ЯК ІНДЕКС ОБИРАЄ ЕЛЕМЕНТ

ДО
`items = ["ліхтар", "карта", "ключ"]`; список має три позиції.

→

ПІСЛЯ
Надруковано три значення; порядок і вміст `items` не змінилися.

1	Перша позиція	<code>items[0]</code>	Відлік зліва починається з нуля, тому результат — "ліхтар".
2	Друга позиція	<code>items[1]</code>	Індекс 1 обирає "карта".
3	Остання позиція	<code>items[-1]</code>	Від'ємний індекс рахує справа, тому результат — "ключ".

Що тут важливо: Індекс указує на позицію у впорядкованій колекції; читання позиції не є зміною списку.

`items[0]` — перший елемент. Відлік із нуля поширений у програмуванні: індекс показує, на скільки кроків елемент віддалений від початку.

Від'ємні індекси рахують із кінця:

- `items[-1]` — останній;
- `items[-2]` — передостанній.

Майстерня

Створи `backpack.py`:

```
backpack.py

class Backpack:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def use(self, item):
        if item not in self.items:
            return None
        self.items.remove(item)
        return item

    def contents_text(self):
        if not self.items:
            return "Наплічник порожній."
        return f"У наплічнику: {' ', ' '.join(self.items)}."

backpack = Backpack()
backpack.add("карта")
backpack.add("ключ")
```

```
backpack.py · продовження

print(backpack.contents_text())
used_item = backpack.use("ключ")
print(f"Використано: {used_item}.")
print(backpack.contents_text())
```

Атрибут `items` належить конкретному наплічнику. Зовнішній код просить об'єкт `add()` або `use()`, а правила зміни списку залишаються всередині класу.

Додавання

```
python

self.items.append(item)
```

`append()` додає один елемент у кінець.

```
python

self.items.insert(0, "компас")
```

`insert(index, value)` вставляє значення у вказане місце й пересуває наступні елементи праворуч. Часті вставки на початок великого списку можуть бути повільними, але для маленьких навчальних колекцій це не проблема.

Щоб додати одразу елементи іншого списку, використовуй `extend()`:

```
python

self.items.extend(["мотузка", "вода"])
```

`append(["мотузка", "вода"])` додав би **один вкладений список**, а `extend(...)` додає два окремі елементи.

Заміна за індексом

```
replace_item.py

items = ["ліхтар", "стара карта", "ключ"]
items[1] = "нова карта"

print(items)
```

Список **змінний**: після створення можна замінювати, додавати й видаляти елементи.

Видалення різними способами

Спосіб залежить від того, що відомо програмі.

```
python

del items[0]
```

`del` видаляє за індексом і не повертає значення.

```
python

items.remove("ключ")
```

`remove()` видаляє перше входження відомого значення. Якщо такого значення немає, виникне `ValueError`, тому в майстерні ми спершу перевірили `item not in self.items`.

```
python

last_item = items.pop()
first_item = items.pop(0)
```

`pop()` видаляє і **повертає** елемент. Без аргументу — останній, з індексом — елемент у вказаному місці. Це зручно, коли предмет не просто зникає, а переходить у наступну дію.

ПЕРЕДАЙ ПРЕДМЕТ

Додай метод, який забирає останній предмет:

```
python

def take_last(self):
    if not self.items:
        return None
    return self.items.pop()
```

Перевір його на наповненому й порожньому напличнику. Чому перевірка стоїть до `pop()`?

Пошук, довжина й кількість

Оператори `in` і `not in` відповідають, чи є значення в колекції:

```
list_search.py

items = ["ключ", "карта", "ключ"]

print("карта" in items)
print("вода" in items)
print(len(items))
print(items.count("ключ"))
```

- `len(items)` повертає кількість елементів;
- `items.count(value)` — кількість входжень значення;
- `items.index(value)` — індекс першого входження, але дає `ValueError`, якщо значення відсутнє.

Для перевірки на порожність пиши `if not self.items:`. Це природний спосіб запитати, чи список порожній.

Перетворення списку рядків на один рядок

```
python

", ".join(self.items)
```

Метод `join()` належить рядку-роздільнику. Він бере послідовність рядків і склеює їх. Усі елементи мають бути текстом; список чисел спочатку треба перетворити.

Сортування не тотожне порядку додавання

```
sorting.py

items = ["ключ", "аптечка", "вода"]

alphabetical = sorted(items)
print(alphabetical)
print(items)
```

`sorted(items)` повертає новий відсортований список, не змінюючи початковий.

```
python

items.sort()
```

`sort()` змінює сам список і повертає `None`.

```
python

items.sort(reverse=True)
items.reverse()
```

- `sort(reverse=True)` сортує у зворотному напрямку;
- `reverse()` просто перевертає поточний порядок, не сортуючи значення.

У списку рядків регістр впливає на порядок. Для нечутливого до регістру сортування:

```
python

sorted_names = sorted(names, key=str.lower)
```

Параметр `key` отримує дію, за результатом якої порівнюють елементи. Ми повернемося до передавання функцій у розділі про функції та модулі.

Копія й друге ім'я — не те саме

```
list_alias.py

original = ["карта"]
alias = original

alias.append("ключ")

print(original)
print(alias)
```

`alias = original` не створює другого списку. Два імені посилаються на одну колекцію.

Для поверхневої копії:

```
list_copy.py

original = ["карта"]
copy_items = original.copy()

copy_items.append("ключ")

print(original)
print(copy_items)
```

Так само працює зріз `original[:]`, з яким познайомимося в наступному розділі.

Поверхнева копія створює новий зовнішній список, але вкладені змінні об'єкти все ще можуть бути спільними. Для початку достатньо запам'ятати: перевіряй, чи хочеш спільну колекцію, чи незалежну копію.

Кортеж фіксує послідовність

Кортеж `tuple` записують круглими дужками:

```
tuple_position.py

position = (12, 7)

print(position[0])
print(position[1])

position = (12, 9)
print(position)
```

Елементи кортежу не можна замінити через індекс. Але ім'я `position` можна прив'язати до нового кортежу цілком.

Кортеж із одним елементом потребує коми:

```
python

single_value = (5,)
```

Без коми `(5)` — просто число в дужках.

Розпакування

```
tuple_unpack.py

position = (12, 7)
x, y = position

print(f"x={x}, y={y}")
```

Кількість імен має відповідати кількості елементів. Розпакування працює і зі списками.

Кортеж доречний, коли кількість і зміст позицій стабільні: координата (x, y), колір (red, green, blue), межі області. Якщо набір має часто зростати або скорочуватися, обирай список.

НЕЗМІННІСТЬ НЕ РОБИТЬ ЗНАЧЕННЯ «КОНСТАНТОЮ»

Кортеж не дозволяє замінити власні елементи, але змінний об'єкт усередині кортежу все одно може змінюватися. Незмінність описує сам контейнер, а не магічно заморожує все, до чого він веде.

ДОПОВНИ ТРИ РІЗНІ ОПЕРАЦІЇ

Заповни пропуски так, щоб прочитати другий елемент, додати новий у кінець і видалити останній.

```
python

items = ["ліхтар", "карта", "ключ"]

print(items[____])
items.____("мотузка")
removed = items.____()
```

Перед запуском запиши фінальний список і значення removed.

Типова помилка

Індекс за межами

```
list_index.py

items = ["карта", "ключ"]
print(items[2])
```

Результат

```
Traceback (most recent call last):
  File "...list_index.py", line 2, in <module>
    print(items[2])
    ~~~~~^^^
IndexError: list index out of range
```

Допустимі індекси тут — 0 і 1. Вираз ~~~~~^^^ виділяє звернення, а останній рядок називає вихід за межі списку.

Результат `sort()` записано замість списку

```
sort_returns_none.py
```

```
items = ["ключ", "карта"]
items = items.sort()
print(items)
```

Результат

None

Це не виняток, а видимий неправильний результат. `sort()` змінив початковий список на місці й повернув `None`; присвоєння замінило посилання в `items`. Або викликай `items.sort()` окремо, або використовуй `items = sorted(items)`.

Спільний список для всіх об'єктів

Побачимо проблему на двох об'єктах:

```
shared_backpack.py
```

```
class Backpack:
    items = []

first = Backpack()
second = Backpack()
first.items.append("ключ")
print(second.items)
```

Результат

['ключ']

Ключ додали лише через `first`, але він з'явився й у `second`, бо список належить класу. Особистий список створюй у `__init__` через `self.items = []`.

ТИПОВА ПОМИЛКА

Не видаляй значення через `remove()` без перевірки, якщо його може не бути. Для безпечного сценарію спочатку використай `if item in items:` або спроектуй метод, який повертає зрозумілий результат.

Швидка перевірка

Спрогнозуй вміст обох коробок:

boxes.py

```
class Box:
    def __init__(self, items):
        self.items = items.copy()

    def add(self, item):
        self.items.append(item)

starter_items = ["олівець"]
first = Box(starter_items)
second = Box(starter_items)

second.add("стрикер")

print(f"Перша: {first.items}")
print(f"Друга: {second.items}")
```

Чому в `__init__` є `.copy()`? Що змінилося б без нього?

ПЕРЕВІР СЕБЕ

Який вираз забере останній елемент списку й водночас поверне його?

- `del items[-1]`
- `items.pop()`
- `items.remove()`

Відповідь: `items.pop()`

`pop()` без індексу видаляє останній елемент і повертає його. `remove()` потребує значення, а `del` нічого не повертає.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Намалюй список із трьох власних значень і підпиши індекси 0, 1, 2, а також -1. Потім поясни, чим копія списку відрізняється від другого імені того самого списку.

Самостійна робота

САМОСТІЙНА РОБОТА

- Розшир `Backpack`: додай обмеження місткості й метод `add()`, який повертає `False`, якщо місця вже немає.
- Створи клас `Playlist` зі списком назв. Додай методи для додавання, вилучення за назвою, показу першого й останнього треку та алфавітної копії списку.
- Досліди різницю між `append(other_list)` і `extend(other_list)` на двох коротких прикладах. Поясни форму результату.
- Створи початковий список предметів і два наплічники: спершу без копіювання, потім із `.copy()`. Покажи, коли зміна «протікає» в інший об'єкт.
- Збережи межі прямокутника як кортеж (`width`, `height`), розпакуй їх і обчисли площу. Потім заміни весь кортеж новими межами.
- Навмисно отримай `IndexError`, `ValueError` від `remove()` і `ValueError` від `index()`. Для кожного випадку поясни, яку передумову треба було перевірити.

КОРОТКА ІСТОРИЧНА ПАУЗА

Відлік з нуля не є примхою Python. Індекс історично зручно тлумачити як зміщення від початку області пам'яті: перший елемент віддалений від початку на нуль позицій. Сьогодні важливіше не походження, а послідовність: перший — `[0]`, останній — `[-1]`.

Підсумок

- Список — впорядкована змінна колекція; індекси починаються з нуля.
- `append()`, `insert()` і `extend()` додають елементи різними способами.
- `del`, `remove()` і `pop()` відрізняються тим, що відомо програмі й чи потрібне видалене значення.
- `in`, `not in`, `len()`, `count()` та `index()` допомагають досліджувати колекцію.
- `sorted()` створює новий список, `sort()` змінює наявний, `reverse()` перевертає порядок.
- Присвоєння другого імені не копіює список; `.copy()` створює поверхневу копію.
- Кортеж — впорядкована послідовність, елементи якої не можна замінювати.
- Колекцію варто ховати за зрозумілими методами об'єкта, якщо зміна має правила.

РОЗДІЛ 06

Повторення: for, range() і зрізи

Список лише зберігає багато значень. Щоб кожен учасник команди привітався або кожен датчик передав показник, програмі потрібне повторення. Цикл дозволяє описати дію один раз і застосувати її до всієї послідовності.

Що зробимо

Створимо команду з кількох об'єктів Member. Клас Team зможе представити всіх учасників, нарахувати кожному досвід, обрати частину складу й порахувати спільний результат.

РЕЗУЛЬТАТ

Ти навчишся відрізняти саму колекцію від одного поточного елемента, бачити межі тіла циклу за відступом і створювати нові списки без ручного повторення рядків.

ЗАРАЗ У ФОКУСІ

Одна дія: простеж один повний оберт циклу: взяти елемент, дати йому тимчасове ім'я, виконати блок, перейти до наступного.

Якщо відволікся, знайди рядок `for name in names:` і поточне значення `name`.

for бере елементи по одному

СПОЧАТКУ ПЕРЕДБАЧ

Випиши всі рядки майбутнього виведення та порахуй, скільки разів виконається `print()`. Не запускай код, доки не маєш конкретної відповіді.

```
first_loop.py
```

```
names = ["Ада", "Бор", "Віра"]

for name in names:
    print(f"Привіт, {name}!")
```

ТРИ ОБЕРТИ ОДНОГО ЦИКЛУ

ДО
`names = ["Ада", "Бор", "Віра"];`
жодного привітання ще немає.

→

ПІСЛЯ
Надруковано три рядки в порядку списку; names не змінився.

1	Перший оберт	<code>name = "Ада"</code>	Блок циклу друкує Привіт, Ада!.
2	Другий оберт	<code>name = "Бор"</code>	Той самий блок працює з наступним елементом.
3	Третій оберт	<code>name = "Віра"</code>	Блок друкує третє привітання.
4	Завершення	Кінець списку	Нового елемента немає, тому Python виходить із циклу.

Що тут важливо: Цикл не копіює блок тричі в коді — він повторно виконує один блок з іншим поточним елементом.

Рядок `for name in names:` читається: «для кожного `name` зі списку `names`». На кожному повторенні ім'я `name` посилається на наступний елемент.

Вкладений блок є **тілом циклу**:

```
python

for name in names:
    print(f"Привіт, {name}!")
    print("Реєстрацію завершено.")

print("Усі учасники готові.")
```

Два рядки з відступом повторяться для кожного імені. Останній рядок без відступу виконається один раз після циклу.

Однина й множина в іменах допомагають читати код: `for member in members`, `for item in items`, `for score in scores`.

Майстерня

Створи team.py:

```
team.py

class Member:
    def __init__(self, name, role):
        self.name = name
        self.role = role
        self.experience = 0

    def train(self, points):
        self.experience += points

    def description(self):
        return f"{self.name}, роль: {self.role}, досвід: {self.experience}."

class Team:
    def __init__(self, members):
        self.members = members.copy()

    def train_everyone(self, points):
        for member in self.members:
            member.train(points)

    def show_everyone(self):
```

```
team.py · продовження

        for member in self.members:
            print(member.description())

    def total_experience(self):
        return sum(member.experience for member in self.members)

members = [
    Member("Ада", "навігатор"),
    Member("Бор", "механік"),
    Member("Віра", "дослідниця"),
]

team = Team(members)
team.show_everyone()
team.train_everyone(5)
print(f"Спільний досвід: {team.total_experience()}.")
```

Список `members` містить не рядки, а три повноцінні об'єкти. Під час циклу `member` по черзі посилається на кожен із них, тому можна викликати `member.train()` і `member.description()`.

Створення об'єктів у списку

Формат із кожним елементом на окремому рядку зручно читати й змінювати:

```
python

members = [
    Member("Ада", "навігатор"),
    Member("Бор", "механік"),
    Member("Віра", "дослідниця"),
]
```

Завершальна кома після останнього елемента дозволена. Вона спрощує додавання нового рядка й робить зміни в Git чистішими.

Обчислення через генераторний вираз

```
python

sum(member.experience for member in self.members)
```

Внутрішня частина по черзі дістає experience кожного учасника, а `sum()` додає числа. Це **генераторний вираз**: значення надходять по одному й не потребують окремого проміжного списку.

На першому знайомстві можна написати довше:

```
python

total = 0
for member in self.members:
    total += member.experience
return total
```

Обидві версії правильні. Обирай коротку лише тоді, коли вона лишається зрозумілою.

ДАЙ БОНУС ЗА РОЛЬ

Додай метод, який нараховує додатковий бал механікам:

```
python

def reward_mechanics(self):
    for member in self.members:
        if member.role == "механік":
            member.train(1)
```

Цикл і умова виконують різну роботу: цикл перебирає учасників, умова обирає потрібних.

range() створює послідовність чисел

range_examples.py

```
for number in range(3):
    print(number, end=" ")
print()

for number in range(2, 5):
    print(number, end=" ")
print()

for number in range(2, 10, 3):
    print(number, end=" ")
```

range(start, stop, step) починає зі start, але **не включає** stop.

- range(3) дає 0, 1, 2;
- range(2, 5) дає 2, 3, 4;
- range(2, 10, 3) додає по 3: 2, 5, 8;
- range(5, 0, -1) дає зворотний відлік 5, 4, 3, 2, 1.

Сам об'єкт range не є списком, але його можна перетворити:

range_list.py

```
levels = list(range(1, 6))
print(levels)
```

Не створюй великий список без потреби. Для простого циклу `for number in range(...)` достатньо самого range.

Повторити дію відому кількість разів

python

```
for _ in range(3):
    robot.move()
```

Ім'я `_` традиційно показує: поточне число циклу не потрібне, важлива лише кількість повторень.

Якщо номер потрібен людині, часто додають 1:

round_numbers.py

```
for index in range(3):
    print(f"Раунд {index + 1}")
```

enumerate() дає і номер, і елемент

```
enumerate_team.py
```

```
names = ["Ада", "Бор", "Віра"]

for number, name in enumerate(names, start=1):
    print(f"{number}. {name}")
```

`enumerate()` повертає пари «номер, елемент». Розпакування одразу дає два зрозумілі імені. `start=1` змінює лише показуваний номер, не індекси самого списку.

Не використовуй `range(len(names))`, якщо тобі потрібен лише елемент. Прямий `for name in names` простіший. Індекс потрібен, коли треба замінити елемент у конкретній позиції або зіставити паралельні дані.

zip() поєднує послідовності попарно

```
zip_roles.py
```

```
names = ["Ада", "Бор"]
roles = ["навігатор", "механік"]

for name, role in zip(names, roles):
    print(f"{name} – {role}")
```

`zip()` зупиняється разом із найкоротшою послідовністю. Якщо різна довжина означає помилку даних, перевір її окремо або зберігай пов'язані значення в об'єктах — як у `Member`.

Зріз бере частину послідовності

```
slices.py
```

```
names = ["Ада", "Бор", "Віра", "Ганна"]

print(names[1:3])
print(names[:2])
print(names[2:])
print(names[::-2])
```

Зріз має форму `[start:stop:step]` і, як `range()`, не включає межу `stop`.

- `names[1:3]` — індекси 1 і 2;
- `names[:2]` — від початку до індексу 2, не включаючи його;
- `names[2:]` — від індексу 2 до кінця;
- `names[::-2]` — кожен другий елемент;
- `names[::-1]` — перевернута копія.

Зріз повертає новий список. `names[:]` — короткий спосіб зробити поверхневу копію всього списку.

ТИМЧАСОВА ГРУПА

Додай у Team метод:

```
python

def first_members(self, amount):
    return self.members[:amount]
```

Метод повертає список перших учасників. Зміна самого поверненого зовнішнього списку не змінить `self.members`, але об'єкти `Member` всередині обох списків ті самі.

Числова статистика

Для непорожньої числової колекції корисні:

```
score_stats.py

scores = [12, 6, 18, 9]

print(min(scores))
print(max(scores))
print(sum(scores))
print(sum(scores) / len(scores))
```

Перед діленням на `len(scores)` переконайся, що список не порожній. `min([])` і `max([])` теж не мають результату й спричиняють `ValueError`.

Метод об'єкта може безпечно обробити порожній список:

```
python

def average_score(self):
    if not self.scores:
        return None
    return sum(self.scores) / len(self.scores)
```

Спискове включення створює новий список

```
list_comprehension.py

squares = [number ** 2 for number in range(1, 6)]
print(squares)
```

Це **спискове включення** (*list comprehension*). Воно читається: «створи список із квадрата кожного `number` у діапазоні».

Можна додати просту умову:

even_squares.py

```
even_squares = [number ** 2 for number in range(1, 11) if number % 2 == 0]
print(even_squares)
```

Символ % дає остачу від ділення. Парне число має остачу 0 під час ділення на 2.

Коли перетворення має кілька кроків, змінює об'єкти або потребує складних умов, звичайний цикл читабельніший.

Вкладені цикли

Цикл може містити інший цикл:

grid.py

```
for row in range(2):
    for column in range(2):
        print((row, column), end=" ")
```

Для кожного row внутрішній цикл проходить усі column. Загальна кількість повторень тут $2 * 2 = 4$. Зі збільшенням меж вкладені цикли швидко створюють багато роботи, тому завжди оцінюй розмір.

ВІДНОВИ ЦИКЛ І ВІДСТУП

Перепиши фрагмент у правильному порядку. Рядок усередині циклу обов'язково має отримати один рівень відступу.

python

```
robot.charge(5)
for robot in robots:
    robots = [first_robot, second_robot]
print("Зарядження завершено")
```

Останній print() має виконатися один раз уже після циклу.

Типова помилка

Забутий відступ

loop_indent.py

```
for member in members:
    print(member.name)
```

Результат

```
File "...loop_indent.py", line 2
    print(member.name)
    ^^^^^
IndentationError: expected an indented block after 'for' statement on line 1
```

line 2 показує місце, де Python побачив print без очікуваного відступу; останній рядок посилається на цикл у рядку 1.

Зайвий відступ після циклу

repeated_summary.py

```
members = ["Ада", "Бор"]

for member in members:
    print(member)
    print("Усі учасники показані.")
```

Результат

```
Ада
Усі учасники показані.
Бор
Усі учасники показані.
```

Другий і четвертий рядки доводять, що підсумкове повідомлення повторилося для кожного учасника. Якщо воно стосується всього циклу, прибері один рівень відступу.

Зміна довжини списку під час перебору

remove_during_loop.py

```
items = ["цілий", "зламаний", "зламаний", "заряд"]

for item in items:
    if item == "зламаний":
        items.remove(item)

print(items)
```

Результат

```
['цілий', 'зламаний', 'заряд']
```

Один зламаний елемент залишився. Після першого видалення сусіднє значення зсунулося, а цикл перейшов далі. Перебирай копію `for item in items.copy():` або створи новий список потрібних елементів.

ТИПОВА ПОМИЛКА

Не вигадуй індекс, коли маєш сам елемент. `for member in members` майже завжди ясніше за `for index in range(len(members))`. Індекс додавай лише коли він справді потрібен.

Швидка перевірка

Спрогнозуй результат і поясни, чому третій учасник не отримав бонус:

```
team_bonus.py

class Member:
    def __init__(self, name):
        self.name = name
        self.points = 0

members = [Member("Ада"), Member("Бор"), Member("Біра")]

for member in members[:2]:
    member.points += 2

for member in members:
    print(f"{member.name}: {member.points}")
```

ПЕРЕВІР СЕБЕ

Які числа створює `range(2, 8, 2)`?

- 2, 4, 6, 8
- 0, 2, 4, 6
- 2, 4, 6

Відповідь: 2, 4, 6

`range` починає з 2, додає крок 2 і зупиняється перед межею 8.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без коду опиши один оберт `for` трьома дієсловами. Потім поясни різницю між `range(3)` і списком із трьох об'єктів.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай команді метод `train_role(role, points)`, який нараховує досвід лише учасникам із потрібною роллю.
- Створи 10 об'єктів `Sensor` у циклі. Нехай кожен отримує номер від 1 до 10, а потім усі друкують короткий опис.
- Використай `enumerate()` для нумерованого списку предметів наплічника. Нумерація для людини має починатися з 1.
- Створи список рівнів від 5 до 50 з кроком 5. Виведи мінімум, максимум, суму й середнє значення.
- Зроби три зрізи складу: перші троє, останні двоє та кожен другий учасник. Для кожного поясни межі зрізу.
- Перероби цикл, який додає квадрати чисел у порожній список, на спискове включення. Потім поверни довгу форму й порівняй читабельність.
- Створи таблицю координат 3×4 вкладеними циклами. Перед запуском обчисли кількість надрукованих пар.
- Продемонструй помилку зміни списку під час циклу, а потім виправ її перебором копії або створенням нового списку.

КОРОТКА ІСТОРИЧНА ПАУЗА

Цикл `for` у Python не означає буквально «збільшуй лічильник», як у деяких старіших мовах. Він просить у будь-якого перебраного об'єкта наступне значення. Тому однаково природно перебирає список об'єктів, рядок, `range` або інші послідовності.

Підсумок

- `for element in collection` бере елементи по одному, а відступ визначає тіло циклу.
- У списку можуть зберігатися об'єкти, і цикл може викликати їхні методи.
- `range()` описує числову послідовність і не включає кінцеву межу.
- `enumerate()` дає номер та елемент, `zip()` поєднує послідовності попарно.
- Зріз `[start:stop:step]` повертає нову частину послідовності без `stop`.
- `min()`, `max()`, `sum()` і `len()` допомагають рахувати статистику, але порожні дані треба врахувати.
- Спискове включення компактно створює новий список із простого перетворення.
- Не змінюй довжину того самого списку під час прямого перебору без усвідомленої стратегії.

РОЗДІЛ 07

Діалог із програмою: `input()` і цикл `while`

Досі всі значення були записані в коді заздалегідь. Інтерактивна консольна програма має чекати на людину, читати команду, відповідати й знову чекати — доки користувач не попросить завершити роботу.

Що зробимо

Створимо `RobotConsole`: об'єкт, який розуміє команди `status`, `move N`, `charge N` та `exit`. Так ми поєднаємо `input()`, перетворення тексту, цикл `while`, прапорець стану, `break` і `continue`.

РЕЗУЛЬТАТ

Програма працюватиме невідому наперед кількість кроків, але матиме чітку умову завершення. Уся поведінка команд лишиться в об'єкті, а цикл відповідатиме за діалог.

ЗАРАЗ У ФОКУСІ

Одна дія: розділяй текст, який прийшов від користувача, і значення потрібного програмі типу.

Якщо відволікся, повернися до переходу `answer` → `int(answer)`.

`input()` завжди повертає текст

```
python
```

```
name = input("Як тебе звати? ")  
print(f"Привіт, {name}!")
```

Рядок у дужках — **запрошення** (*prompt*). Курсор лишається після пробілу, користувач вводить текст і натискає Enter. Результат має тип `str`.

Навіть якщо людина вводить 12, Python отримує рядок "12":

СПОЧАТКУ ПЕРЕДБАЧ

Уяви, що користувач вводить 12. Запиши тип `answer`, числове значення `number` і останній рядок виведення до запуску програми.

`input_type.py`

```
answer = input("Введи число: ")

print(type(answer).__name__)
number = int(answer)
print(f"Наступне число: {number + 1}")
```

ВВЕДЕНИЙ ТЕКСТ СТАЄ ЧИСЛОМ

ДО

Користувач набрав символи 12;
змінних `answer` і `number` ще немає.

→

ПІСЛЯ

`answer == "12"`, `number == 12`,
надруковано Наступне число: 13.

1	Читання	<code>answer = input(...)</code>	<code>input()</code> завжди повертає рядок, тому <code>answer</code> отримує "12".
2	Перевірка типу	<code>type(answer).__name__</code>	Термінал показує <code>str</code> .
3	Перетворення	<code>number = int(answer)</code>	<code>int()</code> створює ціле число 12; початковий рядок не змінюється.
4	Обчислення	<code>number + 1</code>	Числова операція дає 13.

Що тут важливо: Текстове введення треба явно перевірити або перетворити перед числовими обчисленнями.

У автоматичній перевірці введення 12 передається програмі так, ніби його набрала людина.

Перетворення типів

- `int("12")` створює ціле число 12;
- `float("2.5")` створює дробове число 2.5;
- `str(12)` створює текст "12";
- `bool(value)` оцінює істинність значення, але не розуміє людські відповіді автоматично.

Зокрема, `bool("False")` дорівнює `True`, бо це непорожній рядок. Для відповідей «так/ні» краще нормалізувати текст і порівняти:

```
python

answer = input("Продовжити? так/ні: ").strip().lower()
is_confirmed = answer == "так"
```

Якщо `int()` отримає "дванадцять" або "12.5", виникне `ValueError`. У розділі про винятки ми навчимо програму відновлюватися після такого введення. Поки явно просимо ціле число й перевіряємо правильні приклади.

while повторюється, доки умова істинна

```
countdown.py

seconds = 3

while seconds > 0:
    print(seconds)
    seconds -= 1

print("Старт!")
```

Перед кожним повторенням Python перевіряє `seconds > 0`. Коли значення стає нульовим, умова дає `False`, і цикл завершується.

На відміну від `for`, цикл `while` не перебирає готову послідовність. Він доречний, коли кількість повторень наперед невідома: чекати правильну команду, працювати до `exit`, рухатися, доки є енергія.

Майстерня

Створи robot_console.py:

robot_console.py

```
# 1. Об'єкт, стан якого змінюють команди
class Robot:
    def __init__(self, name, energy=10):
        self.name = name
        self.energy = energy
        self.distance = 0

    def status_text(self):
        return f"{self.name}: енергія {self.energy}, шлях {self.distance}."

    def move(self, steps):
        if steps <= 0:
            return "Кількість кроків має бути додатною."
        if steps > self.energy:
            return "Не вистачає енергії."

        self.distance += steps
        self.energy -= steps
        return f"Робот пройшов {steps} кроків."

    def charge(self, amount):
        if amount <= 0:
```

robot_console.py · продовження

```
        return "Заряд має бути додатним."
        self.energy += amount
        return f"Додано {amount} енергії."

# 2. Об'єкт, який читає та розбирає команди
class RobotConsole:
    def __init__(self, robot):
        self.robot = robot
        self.is_running = True

    def handle(self, raw_command):
        parts = raw_command.strip().lower().split()

        if not parts:
            return "Введи команду."

        command = parts[0]

        if command == "status":
            return self.robot.status_text()
        if command == "exit":
            self.is_running = False
```

robot_console.py · продовження

```

        return "Роботу консолі завершено."
    if command in ("move", "charge"):
        if len(parts) != 2 or not parts[1].isdigit():
            return f"Формат: {command} ЦІЛЕ_ЧИСЛО"
        amount = int(parts[1])
        if command == "move":
            return self.robot.move(amount)
        return self.robot.charge(amount)

    return "Невідома команда."

def run(self):
    while self.is_running:
        command = input("> ")
        print(self.handle(command))

# 3. Створення об'єктів і запуск циклу
robot = Robot("Робі")
console = RobotConsole(robot)
console.run()
```

Спробуй вручну послідовність:

```

text

status
move 3
status
charge 5
move 20
exit
```

Один цикл — одна відповідальність

```

python

def run(self):
    while self.is_running:
        command = input("> ")
        print(self.handle(command))
```

Метод `run()` лише організовує діалог: прочитав, передав, показав. Він не містить правил заряду чи руху.

Метод `handle()` отримує звичайний рядок і повертає звичайний рядок. Тому його легко перевіряти без живого введення:

```

python

message = console.handle("move 3")
print(message)
```

Це важливий прийом для інтерактивних програм: тонкий зовнішній цикл, перевірювана логіка всередині об'єкта.

Прапорець керує циклом

`self.is_running` — логічний **прапорець**. Поки він `True`, цикл працює. Команда `exit` змінює його на `False`, і наступна перевірка `while` завершує цикл.

Прапорець доречний, коли завершення може залежати від кількох подій. Якщо є лише одна проста умова, можна перевіряти її прямо.

Розбір команди

```
python

parts = raw_command.strip().lower().split()
```

Ланцюжок методів виконується зліва направо:

- 1 `strip()` прибирає пробіли на краях;
- 2 `lower()` переводить літери в нижній регістр;
- 3 `split()` ділить рядок за пробілами й повертає список частин.

Для " MOVE 3 " результат буде ["move", "3"].

Метод `isdigit()` відповідає, чи складається непорожній рядок із цифр. Він зручний для додатних цілих, але "-3" і "2.5" повернуть `False`. Повна обробка чисел через винятки з'явиться пізніше.

ДОДАЙ КОМАНДУ `RENAME`

Підтримай `rename MOVE_IM`'я. Ім'я може складатися лише з одного слова на цьому етапі. Метод повинен змінити `robot.name` і повернути підтвердження.

break завершує найближчий цикл

Іноді окремий прапорець не потрібен:

```
break_loop.py

while True:
    command = input("> ").strip().lower()

    if command == "exit":
        print("До зустрічі!")
        break

    print(f"Отримано: {command}")
```

`while True` сам ніколи не стане хибним. Тому всередині обов'язково має бути досяжний `break` або інший спосіб завершити програму.

`break` зупиняє лише **найближчий** цикл. У вкладених циклах зовнішній продовжить роботу.

continue переходить до наступної перевірки

```
continue_loop.py

number = 0

while number < 5:
    number += 1
    if number % 2 == 0:
        continue
    print(number)
```

Якщо число парне, `continue` пропускає решту тіла й повертає виконання до умови `while`.

Уважно оновлюй стан **до** `continue`. Інакше цикл може назавжди лишитися на тому самому значенні.

Остача від ділення

Оператор `%` часто допомагає в інтерактивних задачах:

```
modulo.py

first = 14
second = 17

if first % 2 == 0:
    print(f"{first} – парне.")

print(f"{second} залишає остачу {second % 5} при діленні на 5.")
```

Так можна чергувати ходи, розкласти предмети по групах або перевіряти кратність.

Лічильник спроб

Цикл часто має обмеження, навіть якщо користувач не ввів `exit`:

```
pin_attempts.py

secret = "2468"
attempts_left = 3

while attempts_left > 0:
    entered = input("PIN: ")
    if entered == secret:
        print("Доступ дозволено.")
        break

    attempts_left -= 1
    print(f"Невірно. Залишилося спроб: {attempts_left}.")
else:
    print("Доступ заблоковано.")
```

Блок `else` після `while` виконується, якщо цикл завершився через хибну умову, але не через `break`. Це не найчастіша конструкція, проте тут вона точно описує: усі спроби вичерпано без успішного виходу.

Нескінченний цикл і безпечне зупинення

```
infinite_energy.py
```

```
energy = 3

while energy > 0:
    print(energy)
```

Перші рядки терміналу повторюються без кінця. Після ручного Ctrl+C результат виглядає приблизно так:

Результат

```
3
3
3
...
^C
Traceback (most recent call last):
  File "...infinite_energy.py", line 4, in <module>
    print(energy)
KeyboardInterrupt
```

energy ніколи не змінюється, отже умова завжди істинна. У терміналі VS Code програму можна перервати клавішами Ctrl+C. На macOS це також Control+C, не Command+C.

Це надсилає сигнал переривання, і Python зазвичай показує KeyboardInterrupt. Після зупинки виправ причину, а не просто запускай той самий код ще раз.

ПЕРЕРИВАННЯ У ТЕРМІНАЛІ

- Windows PowerShell і Command Prompt: Ctrl+C.
- Linux-термінали: Ctrl+C.
- macOS Terminal: Control+C.

Кнопка з кошиком у панелі терміналу VS Code завершує всю термінальну сесію. Для звичайної завислої програми спершу достатньо переривання клавішами.

ДАЙ ЦИКЛУ ШЛЯХ ДО ЗАВЕРШЕННЯ

Заповни пропуски так, щоб цикл надрукував 1, 2, 3 і зупинився.

```
python

counter = 1

while counter <= ____:
    print(counter)
    counter ____ 1
```

Перед запуском поясни, який рядок змінює умову майбутньої перевірки.

Типова помилка

Число лишилося рядком

```
input_type.py
```

```
age = input("Вік: ")
print(age + 1)
```

Результат

```
Вік: 12
Traceback (most recent call last):
  File "...\\input_type.py", line 2, in <module>
    print(age + 1)
    ~~~~~^~
TypeError: can only concatenate str (not "int") to str
```

`input()` повернув рядок "12", а число 1 має тип `int`; підкреслення показує несумісне додавання. Після перевірки формату перетвори введення: `age = int(input("Вік: "))`.

Умова ніколи не зміниться

```
infinite_running.py
```

```
is_running = True

while is_running:
    print("Працюю")
```

Результат

```
Працюю
Працюю
Працюю
...
^C
Traceback (most recent call last):
  File "...\\infinite_running.py", line 4, in <module>
    print("Працюю")
KeyboardInterrupt
```

Це не «самостійна помилка Python»: ми зупинили нескінченну поведінку ззовні. У самій програмі потрібен шлях, який змінить `is_running`, виконає `break` або завершить цикл іншим усвідомленим способом.

Порожню команду читають до перевірки

Після `parts = raw_command.split()` список може бути порожнім. Звернення `parts[0]` без `if not parts`: спричинить `IndexError`.

ТИПОВА ПОМИЛКА

Не змішуй увесь діалог, розбір тексту й зміну стану в одному великому циклі. Винеси правила у метод на кшталт `handle()`: так кожену команду можна перевірити окремим викликом.

Швидка перевірка

Тут живий цикл не потрібен: ми перевіряємо обробник заготовленою послідовністю команд.

```
counter_commands.py

class Counter:
    def __init__(self):
        self.value = 0

    def handle(self, command):
        if command == "skip":
            return "Пропущено"
        if command == "+":
            self.value += 1
        elif command == "-":
            self.value -= 1
        return f"Значення: {self.value}"

counter = Counter()
commands = ["show", "+", "+", "skip", "-"]

for command in commands:
    print(counter.handle(command))
```

ПЕРЕВІР СЕБЕ

Чому значення з `input()` треба перетворити через `int()`, перш ніж додавати до нього число?

- `int()` потрібна лише для від'ємних чисел.
- `input()` завжди повертає рядок, навіть якщо користувач увів лише цифри.
- `input()` повертає `None` до завершення циклу.

Відповідь: `input()` завжди повертає рядок, навіть якщо користувач увів лише цифри. Текст "12" і число 12 — різні типи. `int()` створює ціле число з правильного текстового запису.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий приклади й поясни, чому `input()` повертає текст навіть після введення цифр. Потім назви три частини безпечного `while`: початковий стан, умова і зміна стану.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай до RobotConsole команди `rename` ІМ'Я та `reset`, яка обнуляє пройдену відстань, але не заряд.
- Створи клас `QuizRound`, який читає відповіді, доки користувач не дасть правильну або не витратить три спроби. Логіку перевірки винеси в окремий метод.
- Напиши цикл, який просить додатне ціле число. Поки що використовуй `isdigit()`; поясни, які правильні числові записи цей метод не прийме.
- Реалізуй меню 1 – статус, 2 – зарядити, 0 – вихід. Відділи перетворення команди від методів об'єкта.
- Виведи числа від 1 до 20, пропускаючи кратні 3 через `continue`. Потім заверши цикл достроково на першому числі, більшому за 14.
- Створи обмежений цикл входу з трьома спробами й `while ... else`. Перевір окремо успішний `break` і вичерпання спроб.
- Навмисно створи нескінченний цикл, у якому лічильник не змінюється, зупини його через `Ctrl+C/Control+C`, а потім виправ умову або оновлення стану.

КОРОТКА ІСТОРИЧНА ПАУЗА

Слово `while` означає «поки». Це корисна перевірка для читання: якщо рядок `while` не можна природно продовжити фразою «поки ця умова істинна», можливо, умова або назви змінних потребують спрощення.

Підсумок

- `input()` показує запрошення й завжди повертає рядок.
- `int()`, `float()` і `str()` створюють значення іншого типу з придатного вхідного значення.
- `while` повторює блок, поки умова дає `True`; стан умови має змінюватися.
- Прапорець зручно керує циклом із кількома можливими причинами завершення.
- `break` завершує найближчий цикл, `continue` переходить до наступної перевірки.
- `%` повертає остачу й допомагає перевіряти парність та кратність.
- `Ctrl+C` або `Control+C` перериває завислу консольну програму.
- Інтерактивний цикл краще залишати тонким, а обробку команди переносити у метод, який можна викликати без `input()`.

РОЗДІЛ 08

Словники, вкладені дані й множини

Список знаходить елемент за позицією: нульовий, перший, останній. Але налаштування зручніше шукати за змістом: `volume`, `language`, `difficulty`. **Словник** пов'язує ключ із відповідним значенням.

Що зробимо

Створимо профіль гравця з налаштуваннями, досягненнями й статистикою. Побачимо, коли словник є частиною стану класу, а коли сам клас уже зайвий або, навпаки, необхідний.

РЕЗУЛЬТАТ

Ти зможеш безпечно читати й змінювати значення за ключем, перебирати пари, будувати вкладені дані та відрізнати словник даних від об'єкта з поведінкою.

ЗАРАЗ У ФОКУСІ

Одна дія: читай словник як відповідність «ключ → значення», а метод об'єкта — як контрольований спосіб змінити цю відповідність.

Якщо відволікся, знайди ключ `"volume"` і його значення до та після виклику.

Словник пов'язує ключ і значення

```
python
```

```
settings = {  
    "volume": 70,  
    "language": "uk",  
    "show_hints": True,  
}
```

Фігурні дужки позначають словник `dict`. Кожна пара має форму `key: value`.

- ключ `"volume"` веде до числа 70;
- ключ `"language"` — до рядка `"uk"`;
- ключ `"show_hints"` — до логічного значення.

Ключі в одному словнику мають бути унікальними. Якщо записати той самий ключ повторно, пізніше значення замінить попереднє.

З Python 3.7 порядок вставлення ключів є гарантованою властивістю мови. Але звертатися до словника все одно треба за змістовним ключем, а не будувати логіку на «другій парі».

Майстерня

Створи `player_profile.py`:

СПОЧАТКУ ПЕРЕДБАЧ

Перед запуском знайди початкову гучність, значення після `change_setting("volume", 40)` і кількість різних досліджених кімнат. Запиши три числа.

player_profile.py

```
class PlayerProfile:
    def __init__(self, name):
        self.name = name
        self.settings = {
            "volume": 70,
            "language": "uk",
            "show_hints": True,
        }
        self.statistics = {}
        self.achievements = []

    def change_setting(self, key, value):
        if key not in self.settings:
            return f"Невідоме налаштування: {key}."
        self.settings[key] = value
        return f"Налаштування {key} змінено."
```

player_profile.py · продовження

```
    def add_stat(self, key, amount=1):
        current = self.statistics.get(key, 0)
        self.statistics[key] = current + amount

    def unlock(self, achievement):
        if achievement in self.achievements:
            return "Досягнення вже було відкрито."
        self.achievements.append(achievement)
        return f"Досягнення відкрито: {achievement}."

    def settings_text(self):
        hints = "так" if self.settings["show_hints"] else "ні"
        return (
            f"Мова: {self.settings['language']}, "
            f"гучність: {self.settings['volume']}, "
            f"підказки: {hints}."
        )
```

```
profile = PlayerProfile("Mipa")
print(profile.settings_text())
```

player_profile.py · продовження

```
print(profile.unlock("Перший крок"))

profile.change_setting("volume", 40)
profile.add_stat("rooms_explored", 3)

print(profile.settings_text())
print(f"Досліджено кімнат: {profile.statistics['rooms_explored']}.")
```

МЕТОД ЗМІНЮЄ ОДНЕ ЗНАЧЕННЯ ЗА КЛЮЧЕМ

ДО
`self.settings["volume"] == 70.`

→

ПІСЛЯ
`self.settings["volume"] == 40;`
інші налаштування лишилися без змін.

1	Виклик	<code>profile.change_setting("volume", 40)</code>	Аргументи передають ключ "volume" і нове значення 40.
2	Захисна перевірка	<code>key not in self.settings</code>	Метод спершу переконується, що такий ключ дозволений.
3	Вибір комірки	<code>self.settings[key]</code>	Значення key дорівнює "volume", тому обрано саме налаштування гучності.
4	Заміна	<code>self.settings[key] = value</code>	Старе значення 70 замінюється на 40.

Що тут важливо: Ключ називає потрібну комірку словника, а метод обмежує й пояснює допустиму зміну стану об'єкта.

В одному об'єкті є три різні колекції:

- словник `settings` має заздалегідь відомі ключі;
- порожній словник `statistics` отримує нові категорії під час роботи;
- список `achievements` зберігає впорядковану історію досягнень.

Читання за ключем

```
python
```

```
volume = self.settings["volume"]
```

Квадратні дужки повертають значення за точним ключем. Якщо ключа немає, Python показує `KeyError`.

Для необов'язкового ключа використовуй `get()`:

```
python
```

```
current = self.statistics.get(key, 0)
```

Якщо `key` існує, повернеться його значення. Якщо ні — запасне `0`. Без другого аргументу `get()` повертає `None`.

Не треба автоматично замінювати всі `[...]` на `.get()`. Коли ключ **зобов'язаний** існувати, `KeyError` чесно показує порушену структуру. `get()` доречний для справді необов'язкових даних або явного значення за замовчуванням.

Додавання й заміна мають однаковий запис

```
python
```

```
self.statistics["steps"] = 1  
self.statistics["steps"] = 2
```

Перший рядок додає нову пару, другий змінює значення наявного ключа.

Метод `update()` додає або замінює кілька пар:

```
python
```

```
self.settings.update({"volume": 40, "show_hints": False})
```

Видалення

```
python
```

```
del self.statistics["temporary"]
```

`del` потребує наявного ключа.

```
python
```

```
removed = self.statistics.pop("temporary", None)
```

`pop()` видаляє ключ і повертає значення. Другий аргумент є запасним результатом, якщо ключ відсутній.

ПЕРЕМКНИ ПІДКАЗКИ

Додай метод без аргументів:

```
python
```

```
def toggle_hints(self):  
    self.settings["show_hints"] = not self.settings["show_hints"]
```

Виклич його двічі й перевір `settings_text()`. Чому `not` зручно описує перемикач?

Перебирання словника

Пари через items()

```
dict_items.py

stats = {"health": 20, "energy": 8}

for key, value in stats.items():
    print(f"{key}: {value}")
```

Кожна пара розпаковується у key та value.

Лише ключі

```
python

for key in stats:
    print(key)
```

Це коротка форма `for key in stats.keys():`. Метод `keys()` корисний, коли хочеться явно підкреслити роботу з ключами.

Лише значення

```
python

for value in stats.values():
    print(value)
```

Значення можуть повторюватися. Якщо потрібні лише унікальні значення, їх можна передати до `set()`.

Визначений порядок показу

Словник пам'ятає порядок додавання, але іноді інтерфейс потребує алфавітного порядку:

```
sorted_dict.py

stats = {"health": 20, "score": 100, "energy": 8}

for key in sorted(stats):
    print(key)
```

`sorted(stats)` сортує ключі й повертає список.

Вкладені структури

Значенням словника може бути список, інший словник або об'єкт.

```
nested_profile.py

profile_data = {
    "name": "Mipa",
    "stats": {
        "score": 120,
        "level": 3,
    },
    "tags": ["дослідник", "помічник"],
}

print(f"{profile_data['name']}: {profile_data['stats']['score']} очок")
print(f"Мітки: {' '.join(profile_data['tags'])}")
```

Кожна пара дужок проходить на один рівень глибше. Надто глибоке вкладення важко читати:

```
python

data["players"][0]["inventory"]["tools"][2]
```

Якщо структура має власні правила й поведінку, виділи клас або хоча б проміжні імена.

Список словників

```
list_of_dicts.py

readings = [
    {"sensor": "A", "value": 21.5},
    {"sensor": "B", "value": 19.0},
]

for reading in readings:
    print(f"Датчик {reading['sensor']}: {reading['value']}")
```

Це зручно для простих зовнішніх даних: рядків таблиці, повідомлень API, записів JSON. Якщо кожен датчик має обчислення, перевірки й стан, об'єкти Sensor дадуть чіткішу модель.

Словник об'єктів

dict_of_objects.py

```
class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def status_text(self):
        return f"{self.name} має {self.energy} енергії."

robots = {
    "worker": Robot("Робі", 10),
    "scout": Robot("Іскра", 7),
}

print(robots["worker"].status_text())
```

Словник швидко знаходить роль "worker", а об'єкт зберігає поведінку конкретного робота. Колекції та ООП не конкурують — вони виконують різні роботи.

Коли клас, а коли словник

Обирай словник, коли:

- ключі можуть з'являтися динамічно;
- дані приходять або йдуть у формат на кшталт JSON;
- потрібне просте зіставлення «ключ → значення»;
- поведінка мінімальна й живе в іншому об'єкті.

Обирай клас, коли:

- структура має обов'язкові поля;
- зміни підкоряються правилам;
- дані мають власні дії;
- важливі зрозумілі методи на кшталт `move()`, `unlock()`, `calculate_total()`.

Нормальний варіант — клас **містить** словник, як `PlayerProfile.settings`. Клас захищає правила, словник дає гнучке зберігання пар.

НЕ ПЕРЕТВОРЮЙ УСЕ НА КЛАСИ

ООП-first не означає «клас для кожного числа». Локальний словник налаштувань може бути найпростішим рішенням. Клас виправданий, коли дає назву сутності й збирає разом її стан та поведінку.

Безпечний параметр-словник

Не використовуй порожній словник як значення параметра за замовчуванням:

shared_default_dict.py

```
class Profile:
    def __init__(self, settings={}):
        self.settings = settings

first = Profile()
second = Profile()
first.settings["volume"] = 20
print(second.settings)
```

Результат

```
{'volume': 20}
```

Зміна через `first` з'явилась у `second`: словник створився один раз під час визначення класу й став спільним для багатьох об'єктів.

Безпечний шаблон:

safe_default_dict.py

```
class Profile:
    def __init__(self, settings=None):
        if settings is None:
            settings = {}
        self.settings = settings.copy()

first = Profile()
second = Profile()

first.settings["volume"] = 20

print(first.settings)
print(second.settings)
```

`None` не змінюється. Новий словник створюється окремо для кожного виклику. `.copy()` також відділяє внутрішній словник від переданого зовнішнього словника.

Для перевірки саме `None` використовують `is None`, а не `== None`. `is` перевіряє тотожність одному спеціальному об'єкту `None`.

Множина зберігає унікальні значення

Множина `set` не зберігає повтори:

```
unique_tags.py

tags = {"python", "oop", "python", "practice"}

print(len(tags))
print("oop" in tags)
```

Порядок множини не слід вважати стабільним для показу. Якщо потрібен передбачуваний порядок, використовуй `sorted(tags)`.

Порожню множину створюють лише через `set()`:

```
python

tags = set()
```

`{}` створює порожній словник.

Основні зміни:

```
python

tags.add("testing")
tags.remove("oop")
tags.discard("missing")
```

`remove()` дає `KeyError`, якщо значення відсутнє; `discard()` мовчки лишає множину без змін.

Операції над множинами:

```
set_operations.py

learned = {"python", "oop", "loops"}
required = {"python", "testing"}

print(sorted(learned | required))
print(sorted(learned & required))
```

- `|` — об'єднання всіх значень;
- `&` — перетин спільних;
- `-` — різниця;
- `<=` — перевірка, чи є одна множина підмножиною іншої.

Множина доречна для унікальних тегів, відвіданих кімнат, дозволів або швидкої перевірки належності.

ДОПОВНИ БЕЗПЕЧНЕ ОНОВЛЕННЯ СТАТИСТИКИ

Заповни пропуски так, щоб перший виклик для нового ключа починав відлік з нуля.

```
python

def add_stat(self, key, amount=1):
    current = self.statistics.__(key, __)
    self.statistics[___] = current + amount
```

Перевір двома викликами `add_stat("rooms")`: спочатку з типовим кроком, потім з `amount=3`.

Типова помилка

Відсутній ключ

```
missing_key.py

settings = {"volume": 70}
print(settings["language"])
```

Результат

```
Traceback (most recent call last):
  File "...\\missing_key.py", line 2, in <module>
    print(settings["language"])
    ~~~~~^~~~~~
KeyError: 'language'
```

Останній рядок називає відсутній ключ. Виріши, що це означає для моделі:

- структура зламана — нехай помилка лишиться видимою;
- ключ необов'язковий — використай `settings.get("language", "uk")`;
- ключ треба створити раніше — виправ ініціалізацію.

Зміна розміру словника під час перебору

```
dict_size.py

statistics = {"wins": 3, "draws": 0, "losses": 1}

for key in statistics:
    if statistics[key] == 0:
        del statistics[key]
```

Результат

```
Traceback (most recent call last):
  File "...dict_size.py", line 3, in <module>
    for key in statistics:
          ^^^^^^^^^^^
RuntimeError: dictionary changed size during iteration
```

Python зупинився на наступному кроці циклу, бо набір ключів уже змінився. Перебирай копію `for key in list(statistics):` або створи новий словник.

Ключ і значення переплутані

`for key, value in data.items()` має два імені. Назви `for value, key ...` синтаксично працюватимуть, але введуть читача в оману й швидко спричинять логічну помилку.

ТИПОВА ПОМИЛКА

Не ховай помилку структури через нескінченний ланцюжок `.get(..., {})`. Якщо поле обов'язкове, краще отримати точну помилку біля джерела, ніж `None` у далекому обчисленні.

Швидка перевірка

Визнач стан обох профілів після змін:

```
profile_copy.py

class Profile:
    def __init__(self, settings):
        self.settings = settings.copy()

defaults = {"theme": "light", "volume": 20}
first = Profile(defaults)
second = Profile(defaults)

first.settings["volume"] = 80
second.settings["theme"] = "dark"

print(f"Перший: {first.settings}")
print(f"Другий: {second.settings}")
```

ПЕРЕВІР СЕБЕ

Як без помилки прочитати необов'язковий ключ `score` і отримати 0, якщо його немає?

- `data["score", 0]`
- `data.get("score")`
- `data.get("score", 0)`

Відповідь: `data.get("score", 0)`

Метод `get()` приймає ключ і запасне значення. Квадратні дужки потребують наявного точного ключа.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без підглядання поясни різницю між ключем, значенням і парою словника. Потім назви ситуацію, де клас доречніший за звичайний словник.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай профіль налаштування `difficulty` і метод, який приймає лише значення `easy`, `normal`, `hard`, а невідоме значення відхиляє.
- Створи клас `WordCounter` зі словником частот. Метод `add(word)` має збільшувати лічильник через `.get(word, 0)`.
- Побудуй словник об'єктів `Robot`, де ключ — унікальний ідентифікатор. Додай безпечний метод пошуку, який повертає об'єкт або `None`.
- Створи список із трьох словників зовнішніх даних, перетвори кожен на об'єкт `Sensor` і порівняй, де зручніше розмістити перевірку значення.
- Перебери словник у порядку ключів і надрукуй таблицю. Потім перебери тільки унікальні значення через множину.
- Створи дві множини навичок: уже вивчені та потрібні. Знайди спільні, відсутні й повний набір.
- Відтвори проблему зі спільним словником у параметрі `settings={}`, а потім виправ її через `None` і копію.
- Навмисно змieni розмір словника під час циклу, прочитай `RuntimeError`, а потім реалізуй безпечне очищення через копію ключів або словникове включення.

КОРОТКА ІСТОРИЧНА ПАУЗА

Назва «словник» добре описує ідею: слово веде до пояснення, а ключ — до значення. В інших мовах подібну структуру можуть називати `map`, `hash map`, `associative array` або `object`. Назви різняться, а головна модель одна: знайти значення не за позицією, а за ключем.

Підсумок

- Словник зберігає пари ключ: значення; ключі унікальні.
- `data[key]` потребує наявного ключа, `data.get(key, default)` підтримує необов'язкові дані.
- Присвоєння за ключем додає або змінює пару; `del` і `pop()` видаляють.
- `items()`, `keys()` і `values()` дають різні способи перебору.
- Значення можуть містити вкладені словники, списки або об'єкти, але надмірна глибина погіршує модель.
- Клас потрібен для сутності з правилами й поведінкою; словник — для гнучкого зіставлення та зовнішніх даних.
- Для змінної колекції за замовчуванням використовуй `None`, а нову колекцію створюй усередині `__init__`.
- Множина зберігає унікальні значення й підтримує об'єднання, перетин та різницю.

РОЗДІЛ 09

Функції й модулі: інструменти поза об'єктами

Метод належить об'єкту. Але не кожна дія природно належить герою, дверям або профілю. Перетворити хвилини на текст, перевірити формат імені або запустити програму — це можуть бути окремі **функції**. Модуль збирає пов'язаний код у .py-файлі.

Що зробимо

Відділимо клас Task від допоміжних функцій форматування й запуску. Потім навчимося імпортувати модулі без копіювання коду між файлами.

РЕЗУЛЬТАТ

Ти зможеш обрати: метод, якщо дія потребує стану конкретного об'єкта; функція, якщо отримує все потрібне через аргументи; модуль, якщо пов'язаному коду потрібен власний файл.

ЗАРАЗ У ФОКУСІ

Одна дія: простеж шлях від аргументу функції через локальні обчислення до значення return.

Якщо відволікся, повернися до `format_duration(95)` і знайди, під яким ім'ям число 95 живе всередині функції.

Функція має ім'я, параметри й результат

СПОЧАТКУ ПЕРЕДБАЧ

Розділи 95 хвилин на повні години й залишок хвилин. Запиши майбутній рядок text, а потім перевір запуском.

first_function.py

```
def format_duration(total_minutes):  
    hours = total_minutes // 60  
    minutes = total_minutes % 60  
    return f"{hours} год {minutes} хв"
```

```
text = format_duration(95)  
print(text)
```


ФУНКЦІЯ ПЕРЕТВОРЮЄ ОДНЕ ЗНАЧЕННЯ НА ІНШЕ

ДО

Виклик має аргумент 95; змінних hours, minutes і text ще немає.

→

ПІСЛЯ

text == "1 год 35 хв"; локальні hours і minutes більше не доступні зовні.

1	Параметр	total_minutes = 95	Аргумент виклику стає значенням параметра функції.
2	Повні години	hours = total_minutes // 60	Цілочисельне ділення дає 1.
3	Залишок	minutes = total_minutes % 60	Остача від ділення дає 35.
4	Повернення	return f"{hours} год {minutes} хв"	Функція створює і повертає рядок 1 год 35 хв.
5	Отримання	text = format_duration(95)	Ім'я text зберігає повернений результат.

Що тут важливо: Параметр приймає аргумент, локальні змінні допомагають обчислити результат, а return передає його до місця виклику.

Заголовок починається з def, далі йдуть ім'я, параметри, дужки та двокрапка. Вкладений блок — тіло функції.

Виклик format_duration(95) передає аргумент, функція створює локальні імена hours і minutes, а return віддає готовий рядок.

Функцію називають дією: format_duration, calculate_total, is_valid_name. Для функції, що повертає логічне значення, природні префікси is_, has_, can_.

Майстерня

Створи спочатку однофайлову версію `tasks_app.py`:

`tasks_app.py`

```
class Task:
    def __init__(self, title, minutes):
        self.title = title
        self.minutes = minutes
        self.is_done = False

    def complete(self):
        self.is_done = True

def format_task(task):
    marker = "x" if task.is_done else " "
    return f"[{marker}] {task.title} – {task.minutes} хв"

def make_task(title, minutes=30):
    clean_title = title.strip()
    return Task(clean_title, minutes)

task = make_task(" Підготувати приклад ", minutes=45)
print(format_task(task))
```

`tasks_app.py` · продовження

```
task.complete()
print(format_task(task))
```

Розподіл відповідальності:

- `Task` зберігає стан завдання й уміє завершувати себе;
- `make_task()` готує вхідні дані й створює об'єкт;
- `format_task()` отримує об'єкт і створює текстове представлення;
- нижні рядки описують конкретний сценарій.

Можна було зробити `task.format_text()`. Це теж правильне рішення, якщо текст є стабільною частиною моделі. Окрема функція доречна, коли форматів може бути кілька або вони належать інтерфейсу, а не самому завданню.

ФУНКЦІЇ НЕ Є «НЕ-ООП»

ООП допомагає моделювати сутності зі станом і поведінкою. Функції допомагають називати незалежні перетворення й координувати об'єкти. Хороша Python-програма природно використовує обидва інструменти.

Параметри працюють так само, як у методах

Функції підтримують позиційні, іменовані та стандартні аргументи:

```
function_arguments.py

def hero_text(name, level=1):
    return f"{name} – рівень {level}"

print(hero_text("Mipa"))
print(hero_text(name="Лея", level=4))
```

Параметр зі значенням за замовчуванням має стояти після обов'язкових параметрів.

Функція може повертати кілька значень як кортеж:

```
min_max.py

def find_bounds(numbers):
    return min(numbers), max(numbers)

lowest, highest = find_bounds([8, 3, 12, 5])
print(lowest)
print(highest)
```

Насправді `return min(...), max(...)` створює кортеж, а розпакування дає два імені.

Довільна кількість позиційних аргументів: `*args`

```
tags.py

def join_tags(*tags):
    return " | ".join(tags)

print(join_tags("python", "oop", "practice"))
```

Зірочка збирає всі додаткові позиційні аргументи в кортеж `tags`. Назва `args` поширена, але змістовне `tags`, `numbers`, `items` краще.

Звичайні позиційні параметри ставлять перед `*args`:

```
python

def build_message(prefix, *parts):
    return prefix + ": " + ", ".join(parts)
```

Довільні іменовані аргументи: ****kwargs**

```
profile_fields.py

def collect_fields(**fields):
    return fields

profile = collect_fields(role="mentor", active=True)
print(profile)
```

Дві зірочки збирають додаткові іменовані аргументи у словник. **kwargs** означає *keyword arguments*, але ім'я **fields**, **settings**, **options** краще пояснює дані.

Не приймай ***args** і ****kwargs** «про всяк випадок». Явні параметри краще документують очікування. Гнучкі форми потрібні, коли кількість значень справді змінна або функція свідомо передає опції далі.

Розпакування під час виклику

Зірочки працюють і в протилежний бік:

```
unpack_call.py

def position_text(x, y, name):
    return f"{name}: {x}, {y}"

position = (3, 8)
options = {"name": "Mipa"}

print(position_text(*position, **options))
```

position** розкладає кортеж на позиційні аргументи, *options** — словник на іменовані.

Область видимості

Ім'я, створене всередині функції, зазвичай **локальне**:

```
local_scope.py

value = 100

def add(a, b):
    value = a + b
    print(f"Внутрішня сума: {value}")
    return value

add(3, 4)
print(f"Зовнішнє значення: {value}")
```

Два імені **value** живуть у різних областях. Локальне зникає після завершення виклику, якщо результат ніде не збережено.

Функція може читати ім'я з зовнішнього модуля, але прихована залежність ускладнює перевірку:

```
python

tax_rate = 0.2

def total_with_tax(price):
    return price * (1 + tax_rate)
```

Для невеликої сталої це прийнятно, але змінні дані краще передавати параметром або зберігати в об'єкті. Ключове слово `global` дозволяє переприв'язати глобальне ім'я, та в початкових програмах воно майже завжди сигналізує, що стан варто організувати ясніше.

Змінний аргумент може змінитися

```
mutable_argument.py

def add_key(items):
    items.append("ключ")

backpack_items = ["карта"]
add_key(backpack_items)
print(backpack_items)
```

Функція отримала посилання на той самий список. Якщо зміна не повинна виходити назовні, створи копію або поверни новий список:

```
python

def with_key(items):
    result = items.copy()
    result.append("ключ")
    return result
```

Назва й документація мають підказувати, чи функція змінює переданий об'єкт.

Документаційний рядок

Перший рядок у тілі функції може коротко пояснити контракт:

```
python

def format_duration(total_minutes):
    """Повернути тривалість у формі «N год M хв»."""
    hours = total_minutes // 60
    minutes = total_minutes % 60
    return f"{hours} год {minutes} хв"
```

Це **docstring**, документаційний рядок. VS Code показує його в підказках. Він потрібен не для переказу кожного рядка, а для призначення, важливих аргументів, результату й неочевидних обмежень.

Для публічної функції корисно відповісти:

- що вона робить;
- що очікує;
- що повертає;
- чи змінює передані об'єкти;
- які помилки може спричинити.

Модуль — звичайний .py-файл

Винесемо код у два файли:

```
text

tasks_app/
├─ main.py
└─ task_tools.py
```

task_tools.py:

```
task_tools.py

def format_duration(total_minutes):
    hours = total_minutes // 60
    minutes = total_minutes % 60
    return f"{hours} год {minutes} хв"
```

main.py:

```
main.py

import task_tools

print(task_tools.format_duration(95))
```

Коли main.py виконує `import task_tools`, Python знаходить `task_tools.py`, запускає його верхньорівневий код один раз і створює об'єкт модуля. Крапка обирає ім'я всередині модуля.

Способи імпорту

```
python

import task_tools
task_tools.format_duration(95)
```

Цей варіант явно показує походження імені.

```
python

from task_tools import format_duration
format_duration(95)
```

Імпортує конкретне ім'я у поточний модуль.

```
python

import task_tools as tools
from task_tools import format_duration as duration_text
```

Псевдонім `as` доречний для довгої назви або загальноприйнятого скорочення.

Уникай:

```
python

from task_tools import *
```

Цей рядок синтаксично правильний і може не спричинити винятку. Дефект інший: зірочка приховує походження імен, може непомітно перезаписати наявне ім'я й ускладнює підказки редактора.

Код для запуску й код для імпорту

Модуль іноді має демонстрацію:

```
name_guard.py

def greet(name):
    return f"Привіт, {name}!"

if __name__ == "__main__":
    print("Модуль запущено напряму.")
```

Коли файл запускають напряму, спеціальне ім'я `__name__` дорівнює `"__main__"`. Під час імпорту воно містить назву модуля, тому захищений блок не запускається.

Цей захист потрібен для демонстрації, командного запуску або маленької перевірки. Визначення класів і функцій лишаються доступними для імпорту.

РОЗКЛАДИ ЗАВДАННЯ ПО ФАЙЛАХ

Створи `task.py` з класом `Task`, `task_tools.py` з `format_task()` і `main.py`, який імпортує обидві частини. Запускай з кореня папки командою `python main.py` на Windows або `python3 main.py` на macOS/Linux.

Імпорт зі стандартної бібліотеки

Python постачається з великою стандартною бібліотекою:

```
math_import.py

from math import hypot

distance = hypot(3, 4)
print(distance)
```

`math` не треба встановлювати окремо. Сторонні пакети, на відміну від нього, додають через менеджер пакетів у віртуальне середовище. Детальніше — у наступних розділах.

ЗАВЕРШИ ФУНКЦІЮ БЕЗ ДРУКУ ВСЕРЕДИНІ

Заповни пропуски так, щоб функція повертала число, а повідомлення формувалося після виклику.

```
python

def coins_for_days(days, coins_per_day):
    total = days * coins_per_day
    ____ total

coins = coins_for_days(4, 7)
print(f"Монет: {____}")
```

Типова помилка

Функцію визначено, але не викликано

```
python

def greet():
    print("Привіт")
```

Це лише створює функцію. Для виконання потрібні дужки: `greet()`.

Ім'я файлу затінює бібліотеку

Якщо назвати власний файл `random.py`, `json.py` або `math.py`, команда `import random` може підхопити твій файл замість стандартного модуля. Дай власному модулю конкретну назву: `random_robot.py`, `json_storage.py`.

Циклічний імпорт

`a.py` імпортує `b.py`, а `b.py` одразу імпортує `a.py`. Один модуль ще не встиг повністю створити свої імена, і другий їх не знаходить. Часто це означає, що спільний код треба винести у третій модуль або переглянути межі відповідальності.

ТИПОВА ПОМИЛКА

Не перетворюй довгий метод на функцію, яка все одно таємно читає десяток глобальних змінних. Хороша функція отримує залежності явно через параметри й повертає зрозумілий результат.

Швидка перевірка

Визнач, яка частина змінює об'єкт, а яка лише формує текст:

```
function_or_method.py

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def move(self, steps):
        self.energy -= steps

def status_line(robot):
    return f"{robot.name}: {robot.energy}"

robot = Robot("Робі", 10)
robot.move(3)
print(status_line(robot))
```

ПЕРЕВІР СЕБЕ

Коли дія найприродніше є методом?

- Коли вона використовує або змінює стан конкретного об'єкта.
- Коли в ній є більше одного рядка.
- Коли її треба імпортувати з іншого файлу.

Відповідь: Коли вона використовує або змінює стан конкретного об'єкта.

Метод виражає поведінку об'єкта через `self`. Незалежне перетворення, яке отримує все аргументами, часто краще оформити функцією.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий код і назви чотири частини функції: ім'я, параметри, тіло та повернене значення. Потім поясни, коли дія належить функції, а коли — методу об'єкта.

Самостійна робота

САМОСТІЙНА РОБОТА

- Напиши функції `celsius_to_fahrenheit()` і `fahrenheit_to_celsius()`. Вони мають повертати числа, а приклад використання — формувати текст.
- Створи функцію `build_robot(name, **settings)`, яка повертає об'єкт `Robot` і копіює дозволені налаштування. Невідомі ключі не ігноруй мовчки.
- Напиши функцію з `*scores`, яка повертає мінімум, максимум і середнє як кортеж. Окремо виріши, що робити без аргументів.
- Передай список у функцію, навмисно зміни його, а потім створи версію, яка повертає незалежну копію. Поясни контракт обох.
- Розклади маленьку програму на `model.py`, `formatters.py` і `main.py`. Не дублюй визначення між файлами.
- Перевір різницю між `import module` і `from module import name`. Для обох варіантів поясни, звідки видно походження функції.
- Створи модуль з демонстраційним кодом під `if __name__ == "__main__":`. Запусти його напряму, потім імпортуй і переконайся, що демонстрація не виконується.

КОРОТКА ІСТОРИЧНА ПАУЗА

У Python функції є повноцінними об'єктами: їх можна зберігати в змінних, передавати аргументами й повертати. Ми вже торкнулися цього в `sorted(..., key=str.lower)`. Це не суперечить ООП — сама мова дозволяє об'єктам і функціям співпрацювати.

Підсумок

- Функція називає незалежну дію; метод є функцією, пов'язаною з класом і об'єктом.
- Параметри, аргументи, значення за замовчуванням і `return` працюють у функціях так само, як у методах.
- `*args` збирає довільні позиційні аргументи в кортеж, `**kwargs` — іменовані у словник.
- Локальні імена живуть усередині виклику; явні параметри кращі за приховану залежність від глобального стану.
- Змінний аргумент може змінитися; контракт має це показувати.
- Docstring пояснює призначення й важливі умови використання.
- Модуль — `.py`-файл, який можна імпортувати; імпорти зі зірочкою приховують походження імен.
- Захист `if __name__ == "__main__":` відділяє прямий запуск від імпорту.

РОЗДІЛ 10

Структура OOP-проєкту у VS Code

Коли програма виростає, проблема вже не в тому, чи працює окремий `if`. Треба швидко знайти клас, зрозуміти точку запуску й не загубитися в імпортах. Структура папок — це карта відповідальності, а не прикраса.

Що зробимо

Розкладемо консольну станцію роботів на `main.py`, окремі модулі класів, файл налаштувань і пакет `objects`. Запустимо проєкт із кореня та розберемо, чому місце запуску впливає на імпорти й шляхи.

РЕЗУЛЬТАТ

Відкривши проєкт у VS Code, інша людина одразу побачить: де запуск, де об'єкти, де сталі дані й де перевірки. `main.py` координуватиме готові частини, а не міститиме всю логіку.

ЗАРАЗ У ФОКУСІ

Одна дія: для кожного файла сформулюй одну відповідальність і запускай проєкт тільки з його кореня.

Якщо відволікся, повернися до карти папок і спочатку знайди `main.py` — єдину точку запуску.

Почни з маленької карти

Для навчального проєкту достатня така структура:

```
text

robot_station/
├─ main.py
├─ settings.py
├─ objects/
│  ├── __init__.py
│  ├── robot.py
│  └── station.py
└─ tests/
   └─ test_station.py
```

- `main.py` — точка входу: створити об'єкти й запустити сценарій;
- `settings.py` — сталі налаштування;
- `objects/robot.py` — клас `Robot`;
- `objects/station.py` — клас `Station`, який керує роботами;

- `objects/__init__.py` — позначає звичайний пакет;
- `tests/` — перевірки, з якими детально працюватимемо пізніше.

Один навчальний клас на файл робить маршрут очевидним: `Robot` шукаємо в `robot.py`. Це не закон Python. Маленькі тісно пов'язані класи іноді доречно тримати разом, але спершу ясність важливіша за економію файлів.

Майстерня

1. Сталі налаштування

`settings.py`:

```
settings.py

DEFAULT_ENERGY = 20
STATION_NAME = "Орбіта"
```

Назви значень, які програма не планує змінювати, пишуть `UPPER_SNAKE_CASE`. Python технічно не забороняє переприсвоєння, але стиль повідомляє: це **константа за домовленістю**.

Не збирай у `settings.py` будь-яке число. `DEFAULT_ENERGY` — налаштування. Локальний крок `index + 1` не потребує глобальної назви.

2. Один клас — один зрозумілий модуль

`objects/robot.py`:

```
objects/robot.py

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def charge(self, amount):
        if amount <= 0:
            return False
        self.energy += amount
        return True

    def status_text(self):
        return f"{self.name}: {self.energy} енергії"
```

objects/station.py:

```
objects/station.py

class Station:
    def __init__(self, name):
        self.name = name
        self.robots = []

    def register(self, robot):
        self.robots.append(robot)

    def status_lines(self):
        return [robot.status_text() for robot in self.robots]
```

Station не імпортує Robot, бо не створює його й не перевіряє точний тип. Вона працює з будь-яким об'єктом, який має status_text(). Менше зайвих залежностей — простіші модулі.

3. main.py лише складає частини

main.py:

```
main.py

from objects.robot import Robot
from objects.station import Station
from settings import DEFAULT_ENERGY, STATION_NAME

def main():
    station = Station(STATION_NAME)
    station.register(Robot("Робі", DEFAULT_ENERGY))
    station.register(Robot("Іскра", 12))

    for line in station.status_lines():
        print(line)

if __name__ == "__main__":
    main()
```

Функція main() є явним сценарієм запуску. Захист унизу не дає сценарію виконатися, якщо хтось імпортує main.py для перевірки.

Однофайловий еквівалент, який можна запустити для швидкої перевірки:

СПОЧАТКУ ПЕРЕДБАЧ

В однофайловому еквіваленті знайди порядок створення станції та двох роботів. Запиши три майбутні рядки виведення до запуску.

station_single_file.py

```
DEFAULT_ENERGY = 20
STATION_NAME = "Орбіта"

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def status_text(self):
        return f"{self.name}: {self.energy} енергії"

class Station:
    def __init__(self, name):
        self.name = name
```

station_single_file.py · продовження

```
        self.robots = []

    def register(self, robot):
        self.robots.append(robot)

    def status_lines(self):
        return [robot.status_text() for robot in self.robots]

def main():
    station = Station(STATION_NAME)
    station.register(Robot("Робі", DEFAULT_ENERGY))
    station.register(Robot("Іскра", 12))

    print(f"Станція {station.name}")
    for line in station.status_lines():
        print(line)

if __name__ == "__main__":
    main()
```

ЯК ОБ'ЄКТИ СКЛАДАЮТЬСЯ У СЦЕНАРІЙ

ДО

Створено `Station("Орбіта");` її список `robots` порожній.

→

ПІСЛЯ

`station.robots` містить двох роботів; надруковано назву станції та два стани.

1	Перший об'єкт	<code>Robot("Робі", DEFAULT_ENERGY)</code>	Створюється робот з енергією 20.
2	Реєстрація	<code>station.register(...)</code>	Метод станції додає робота до власного списку.
3	Другий об'єкт	<code>Robot("Іскра", 12)</code>	Створюється й реєструється ще один робот.
4	Читання стану	<code>station.status_lines()</code>	Станція просить кожного робота підготувати власний текст стану.
5	Точка запуску	<code>main()</code>	Головна функція координує готові об'єкти, але не забирає їхню поведінку.

Що тут важливо: Модулі розділяють відповідальності у файлах, а `main.py` лише збирає об'єкти в один сценарій.

Розкладання по файлах не змінює модель. Воно лише дає кожній частині адресу.

Пакет і файл `__init__.py`

Папка `objects` містить модулі й має `__init__.py`, тому її можна імпортувати як пакет.

На початку файл може бути порожнім:

```
objects/__init__.py
```

```
"""Об'єкти станції роботів."""
```

Сучасний Python підтримує пакети без `__init__.py` у спеціальних сценаріях, але явний файл робить звичайний навчальний пакет зрозумілим інструментам і читачеві.

Можна відкрити зручні імпорти на рівні пакета:

```
objects/__init__.py
```

```
from .robot import Robot
from .station import Station
```

Тоді `main.py` може написати:

```
python
```

```
from objects import Robot, Station
```

Крапка в `.robot` означає «модуль `robot` у цьому самому пакеті». Не наповнюй `__init__.py` складною логікою: імпорт пакета не повинен несподівано запускати програму.

Абсолютні й відносні імпорти

Усередині пакета можливі два стилі:

```
python

from objects.robot import Robot
```

Це абсолютний імпорт від кореня проєкту.

```
python

from .robot import Robot
```

Це відносний імпорт від поточного пакета. Він працює, коли модуль завантажений як частина пакета, а не як випадковий окремий файл.

Для навчального проєкту обери один послідовний стиль. У `main.py` на корені природні абсолютні імпорти. Усередині `objects` короткі відносні імпорти показують внутрішній зв'язок.

ПРОЧИТАЙ АДРЕСУ ІМПОРТУ

У рядку `from objects.robot import Robot` назви по черзі пакет, модуль і клас. Потім поясни, чому `main.py` може використовувати абсолютний імпорт, а всередині `objects` доречний запис `from .robot import Robot`.

Запускай з кореня проєкту

Відкрий у VS Code саме папку `robot_station`, а не лише `objects`. Вбудований термінал має показувати корінь:

```
text

robot_station/
```

ЗАПУСК У РІЗНИХ СИСТЕМАХ

Windows PowerShell або Command Prompt

```
powershell

python main.py
```


Якщо налаштований launcher, також може працювати:

```
powershell  
  
py main.py
```

macOS i Linux

```
bash  
  
python3 main.py
```

Самі імпорти в коді не змінюються між системами. Відрізняється переважно команда інтерпретатора й запис шляхів у терміналі.

Python додає папку запущеного скрипту до шляху пошуку модулів. Якщо запустити `python objects/robot.py`, кореневі імпорти можуть не працювати, а головний сценарій взагалі оминається. Запускай точку входу з кореня.

Для пакета з `app/main.py` іноді використовують модульний запуск:

```
powershell  
  
python -m app.main
```

або:

```
bash  
  
python3 -m app.main
```

Ключ `-m` просить Python знайти модуль через систему імпортів. Це корисно, коли сама точка входу лежить усередині пакета.

ОБЕРИ ОДНУ ТОЧКУ ЗАПУСКУ

Для структури з корневим `main.py` запиши одну команду саме для своєї системи. Для структури з `app/main.py` запиши модульну команду. Не запускай окремий файл класу: спершу вкажи, який сценарій має зібрати об'єкти.

Поточна папка й шляхи до файлів

Імпорт і відкриття даних — різні механізми, але обидва залежать від контексту запуску.

```
python  
  
open("data/message.txt", encoding="utf-8")
```

Відносний шлях зазвичай рахується від **поточної робочої папки термінала**, а не автоматично від файлу з цим рядком. Тому однакова команда запуску з кореня робить поведінку передбачуваною.

У розділі про файли ми побудуємо шлях від `Path(__file__)`, коли ресурс повинен знаходитися поруч із модулем незалежно від робочої папки.

Віртуальне середовище

Стандартний модуль `venv` створює для проєкту окреме середовище пакетів. Це не встановлення редактора й не заміна системного Python. Середовище допомагає двом проєктам мати різні версії залежностей.

Створення в корені проєкту:

СТВОРЕННЯ `.venv`

Windows

```
powershell  
  
python -m venv .venv
```

macOS i Linux

```
bash  
  
python3 -m venv .venv
```

Активация змінює команду `python` у поточному терміналі:

АКТИВАЦІЯ `.venv`

Windows PowerShell

```
powershell  
  
.\.venv\Scripts\Activate.ps1
```

Windows Command Prompt

```
bat  
  
.\.venv\Scripts\activate.bat
```

macOS i Linux

```
bash

source .venv/bin/activate
```

Після активації на початку запрошення часто з'являється (`.venv`). Для виходу в усіх системах:

```
text

deactivate
```

У VS Code команда **Python: Select Interpreter** дозволяє вибрати інтерпретатор із `.venv`. Після вибору нові термінали зазвичай активують його автоматично.

Папку `.venv` не зберігають у Git. Додай до `.gitignore`:

```
text

.venv/
__pycache__/
*.pyc
```

ПЕРЕВІР ТРИ ОРІЄНТИРИ

Перед встановленням будь-якої залежності покажи: корінь проєкту у VS Code, активний інтерпретатор і папку `.venv` у `.gitignore`. Якщо один орієнтир не можеш знайти, не копіюй наступну команду навімання.

Пакети встановлюють через активний інтерпретатор

```
powershell

python -m pip install package_name
```

Форма `python -m pip` явно використовує `pip` того самого інтерпретатора, яким запускається програма. Не копіюй `package_name`: підстав точну назву потрібного стабільного пакета.

Список залежностей можна зафіксувати:

```
powershell

python -m pip freeze > requirements.txt
```

і відтворити в іншому середовищі:

```
powershell

python -m pip install -r requirements.txt
```

Для простого навчального проєкту цього достатньо. Не додавай бібліотеку лише тому, що вона популярна: спершу визнач, яку конкретну проблему вона розв'язує.

СТАНДАРТНА БІБЛІОТЕКА НЕ ПОТРЕБУЄ PIP

Модулі `json`, `random`, `pathlib`, `datetime`, `math` і `unittest` постачаються з Python. Команда на кшталт `pip install json` не потрібна й може встановити зовсім інший сторонній пакет.

ПОТРІБЕН PIP ЧИ НІ

Розклади назви у дві групи: `json`, `pathlib`, `pytest`, `random`. Для кожної назви скажи, чи вона входить до стандартної бібліотеки, чи є окремою залежністю проєкту.

Кеші й службові файли

Після імпорту Python може створити `__pycache__` і файли `.pyc`. Це кеш байткоду, який прискорює наступні завантаження модулів. Це не нові вихідні файли й не місце для власного коду.

Їх можна не додавати до Git. Якщо кеш зникне, Python створить його знову.

Циклічні залежності як сигнал структури

Уявімо:

- `robot.py` імпортує `Station`;
- `station.py` імпортує `Robot`.

Два модулі завантажуються одночасно й чекають один на одного. Часто один із імпортів просто не потрібен: `Station` може приймати об'єкт без перевірки класу. Або спільний контракт треба винести в третій модуль.

Малюй стрілки залежностей у один бік:

```
text
```

```
main.py -> Station -> об'єкти з потрібними методами
        -> Robot
        -> settings
```

`main.py` знає конкретні класи й складає їх. Нижчі модулі не повинні імпортувати `main.py`.

РОЗКЛАДИ ВІДПОВІДАЛЬНОСТІ ПО ФАЙЛАХ

Зістав кожен фрагмент з одним файлом: `settings.py`, `objects/robot.py`, `objects/station.py`, `main.py`.

- `DEFAULT_ENERGY = 20`
- `class Robot: ...`
- `class Station: ...`
- створення станції, реєстрація роботів і друк результату

Після розподілу поясни, який із цих файлів запускають і чому інші переважно імпортують.

Типова помилка

«Працює лише через кнопку в одному файлі»

Якщо запуск окремого `objects/robot.py` працює, а `main.py` — ні, або навпаки, перевір:

- 1 Яка папка відкрита у VS Code?
- 2 Яка поточна папка термінала?
- 3 Яку точну команду запущено?
- 4 Чи не названий власний файл як стандартний модуль?

Не додавай випадкові `..` до імпортів і не змінюй `sys.path` навмання. Спершу віднови єдину точку запуску з кореня.

Уся логіка лишилася в `main.py`

Окремі файли не допомагають, якщо `main.py` усе ще містить сотні рядків умов, змінює атрибути напрямую й лише формально створює порожні класи. Перенось до класу правила, що належать його стану.

ТИПОВА ПОМИЛКА

Не створюй папку для кожного маленького файла. Структура має зменшувати час пошуку. Для п'яти модулів достатньо кількох зрозумілих папок; глибоке дерево без потреби лише додає імпортів.

Швидка перевірка

Цей приклад доводить, що `main()` координує готові об'єкти:

```
clean_main.py

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    def charge(self, amount):
        self.energy += amount
        return f"{self.name} заряджено до {self.energy}."

def main():
    robot = Robot("Робі", 10)
    print(robot.charge(5))

if __name__ == "__main__":
    main()
```

У багатофайловій версії зміниться лише місце визначення `Robot` та рядок імпорту, а сценарій `main()` залишиться таким самим.

ПЕРЕВІР СЕБЕ

Звідки найкраще запускати команду `python main.py` у наведеній структурі?

- З будь-якої папки — поточна папка ніколи не впливає на Python.
- З кореня `robot_station`, де лежить `main.py`.
- З папки `objects`, бо там лежать класи.

Відповідь: З кореня `robot_station`, де лежить `main.py`.

Єдина точка запуску з кореня робить локальні імпорти й відносні шляхи передбачуваними.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Не дивлячись на дерево, намалюй мінімальний OOP-проект з `main.py`, двома файлами класів і папкою тестів. Підпиши стрілками лише реальні напрямки імпорту.

Самостійна робота

САМОСТІЙНА РОБОТА

- Розклади RobotConsole на `robot.py`, `console.py`, `settings.py` і короткий `main.py`. Запусти лише точку входу з кореня.
- Створи пакет `objects` з порожнім `__init__.py`, а потім відкрий у ньому зручні імпорти Robot і Station. Порівняй два стилі імпорту в `main.py`.
- Навмисно запусти внутрішній модуль із неправильної папки, зафіксуй помилку імпорту, а потім виправ маршрут без редагування `sys.path`.
- Створи `settings.py` з трьома справжніми налаштуваннями. Не винось туди локальні числа, зміст яких зрозумілий лише в одному методі.
- Створи `.venv`, активуй її відповідною командою своєї системи, перевір шлях через `python -c "import sys; print(sys.executable)"`, а потім виконай `deactivate`.
- Додай `.gitignore` для `.venv`, `__pycache__` і `.рус`. Поясни, чому вихідний код лишається, а ці файли можна відтворити.
- Намалюй стрілки імпортів для свого маленького проєкту. Прибери зайву двосторонню залежність або поясни, чому її поки неможливо уникнути.

КОРОТКА ІСТОРИЧНА ПАУЗА

Пакет у Python — це насамперед спосіб організувати простір імен. Крапки в `objects.robot` нагадують шлях по папках, але імпорт є роботою самої мови: вона знаходить модуль, створює його об'єкт і кешує завантаження.

Підсумок

- Структура проєкту показує відповідальності: точка входу, моделі, налаштування, тести.
- Один клас на файл — корисне навчальне правило, а не абсолютна вимога мови.
- `main.py` створює й поєднує об'єкти; правила стану мають жити у відповідних класах.
- `__init__.py` робить звичайний пакет явним і може відкривати зручні імпорти.
- Запуск із кореня проєкту робить імпорти та відносні шляхи передбачуваними.
- `python -m package.module` запускає модуль через систему імпортів.
- `.venv` ізолює сторонні пакети; команди активації відрізняються у Windows та macOS/Linux.
- Стандартну бібліотеку не встановлюють через `pip`, а службові кеші не зберігають у Git.
- Циклічний імпорт часто вказує на зайву залежність або нечітку межу модулів.

Офіційні орієнтири: [віртуальні середовища venv](#), [модулі та пакети](#), [вибір інтерпретатора у VS Code](#).

РОЗДІЛ 11

Пов'язані об'єкти: успадкування, композиція й властивості

Класи рідко живуть окремо. Електросамокат є видом транспорту, а його батарея — окремою частиною. Перше відношення іноді описує **успадкування**, друге — **композиція**. Вибір впливає на те, наскільки легко змінювати програму.

Що зробимо

Створимо базовий `Vehicle`, спеціалізований `ElectricScooter` і окремий об'єкт `Battery`. Потім закриємо небезпечну зміну заряду за властивістю `energy`.

РЕЗУЛЬТАТ

Ти зможеш перевірити відношення простими фразами: «X є різновидом Y» для успадкування та «X має Y» для композиції. Перевагу отримає простіший зв'язок, а не найбільше дерево класів.

ЗАРАЗ У ФОКУСІ

Одна дія: для кожного зв'язку став одне питання: об'єкт **є різновидом** іншого чи **має всередині** інший об'єкт?

Якщо відволікся, повернися до пари `Bicycle(Vehicle)` і прочитай її як «велосипед є транспортом».

Успадкування передає спільний опис

```
python

class Vehicle:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def move_text(self):
        return f"{self.name} рухається зі швидкістю {self.speed}."

class Bicycle(Vehicle):
    pass
```


`Bicycle(Vehicle)` означає: `Bicycle` успадковує доступні методи й атрибути, підготовлені `Vehicle`. Порожній клас із `pass` уже може створювати об'єкти:

СПОЧАТКУ ПЕРЕДБАЧ

У класі `Bicycle` написано лише `pass`. Визнач, чи зможе його об'єкт викликати `move_text()`, звідки візьметься цей метод і який рядок буде надруковано.

```
first_inheritance.py

class Vehicle:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def move_text(self):
        return f"{self.name} рухається зі швидкістю {self.speed}."

class Bicycle(Vehicle):
    pass

bicycle = Bicycle("Ластівка", 18)
print(bicycle.move_text())
```

ЯК ДОЧІРНІЙ КЛАС ЗНАХОДИТЬ УСПАДКОВАНИЙ МЕТОД

ДО
Клас `Bicycle` не має власного
`__init__()` або `move_text()`.

→

ПІСЛЯ
Об'єкт `Bicycle` має стан `Ластівка`,
18 і повертає успадкований текст
руху.

1	Зв'язок класів	<code>class Bicycle(Vehicle):</code>	<code>Vehicle</code> записаний як батьківський клас.
2	Створення	<code>Bicycle("Ластівка", 18)</code>	Python не знаходить <code>__init__</code> у <code>Bicycle</code> і використовує <code>Vehicle.__init__</code> .
3	Збереження стану	<code>self.name, self.speed</code>	У новому об'єкті з'являються атрибути, описані батьківським класом.
4	Пошук поведінки	<code>bicycle.move_text()</code>	Python не знаходить метод у <code>Bicycle</code> і бере <code>Vehicle.move_text</code> .

Що тут важливо: Успадкування дозволяє спеціалізованому класу повторно використати спільний стан і поведінку батьківського класу.

`Vehicle` називають базовим або батьківським класом, `Bicycle` — похідним або дочірнім.

Успадкування доречне, якщо об'єкт похідного класу справді можна використовувати всюди, де очікують базовий. Велосипед є транспортом і підтримує основні обіцянки транспорту.

Майстерня

Створи `electric_scooter.py`:

```
electric_scooter.py

class Battery:
    def __init__(self, capacity, charge=None):
        self.capacity = capacity
        self._charge = capacity if charge is None else charge

    @property
    def charge(self):
        return self._charge

    def use(self, amount):
        if amount < 0 or amount > self._charge:
            return False
        self._charge -= amount
        return True

    def status_text(self):
        return f"Заряд: {self._charge}/{self.capacity}."
```

electric_scooter.py · продовження

```
        self.speed = speed

    def move_text(self):
        return f"{self.name} рухається зі швидкістю {self.speed}."

class ElectricScooter(Vehicle):
    def __init__(self, name, speed, battery):
        super().__init__(name, speed)
        self.battery = battery
        self.distance = 0

    def ride(self, kilometers):
        if kilometers <= 0:
            return "Відстань має бути додатною."
        if not self.battery.use(kilometers):
            return "Недостатньо заряду."
        self.distance += kilometers
        return f"Самокат проїхав {kilometers} км."

battery = Battery(capacity=40, charge=30)
```

electric_scooter.py · продовження

```
scooter = ElectricScooter("Іскра", speed=22, battery=battery)

print(scooter.move_text())
print(scooter.battery.status_text())
print(scooter.ride(8))
print(scooter.battery.status_text())
```

Тут обидва зв'язки мають чітку причину:

- `ElectricScooter` є видом `Vehicle`;
- `ElectricScooter` має `Battery`.

`super()` викликає реалізацію базового класу

```
python

super().__init__(name, speed)
```

`super()` дає доступ до наступної реалізації в ланцюжку успадкування. Тут він просить `Vehicle` підготувати спільні атрибути `name` і `speed`.

Не дублюй:

```
python

self.name = name
self.speed = speed
```

у кожному похідному класі, якщо базовий уже має правильну ініціалізацію. Це не обов'язково впаде одразу, але дубльовані правила легко розійдуться.

`super()` краще за прямий виклик `Vehicle.__init__(self, ...)`, бо поважає порядок пошуку методів і підтримує складніші, хоча й рідші, схеми успадкування.

Перевизначення методу

Похідний клас може дати власну реалізацію методу з тим самим іменем:

```
override.py

class Vehicle:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def move_text(self):
        return f"{self.name} рухається зі швидкістю {self.speed}."

class Boat(Vehicle):
    def move_text(self):
        return f"Човен {self.name} рухається водою зі швидкістю {self.speed}."

boat = Boat("Плин", 12)
print(boat.move_text())
```

Це **перевизначення**. Виклик `boat.move_text()` знаходить найближчу реалізацію в `Boat`.

Якщо треба розширити, а не повністю замінити базовий результат:

```
python

def move_text(self):
    base_text = super().move_text()
    return base_text + " Двигун працює тихо."
```

Зберігай сумісний зміст: якщо базовий `move_text()` повертає рядок, похідний не повинен раптом видаляти об'єкт або повертати словник без дуже вагомої причини.

Композиція: об'єкт складається з інших об'єктів

```
python

self.battery = battery
```

Самокат не успадковує батарею. Він містить посилання на окремий об'єкт і делегує йому роботу:

```
python

if not self.battery.use(kilometers):
    return "Недостатньо заряду."
```

Переваги:

- Battery можна перевірити окремо;
- ту саму модель батареї можна використати в роботі або ліхтарі;
- батарею можна замінити іншим сумісним об'єктом;
- клас самоката не розростається правилами заряджання.

ЗАМІНИ БАТАРЕЮ

Створи другу батарею більшої місткості й присвой `scooter.battery = new_battery`. Перевір, що код `ride()` не треба змінювати: він очікує поведінку `use()` і `status_text()`, а не конкретну історію об'єкта.

Підкреслення позначає внутрішню деталь

```
python

self._charge = capacity
```

Одне початкове підкреслення — домовленість: «це внутрішня деталь класу; зовнішньому коду краще не змінювати напряду». Python не ставить фізичний замок:

```
python

battery._charge = -100
```

технічно можливе. Але такий код обходить перевірки й порушує контракт.

Подвійне підкреслення `__charge` вмикає механізм зміни імені, а не абсолютну приватність. Для початкових моделей достатньо одного `_` і зрозумілих публічних методів.

@property дає контрольоване читання

```
python

@property
def charge(self):
    return self._charge
```

Зовнішній код читає `battery.charge` без дужок, але насправді виконується метод. Властивість доречна для простого значення, яке виглядає як стан.

Без setter-властивості такий запис не дозволений:

```
read_only_property.py

class Battery:
    def __init__(self, charge):
        self._charge = charge

    @property
    def charge(self):
        return self._charge

battery = Battery(80)
battery.charge = -10
```

Результат

```
Traceback (most recent call last):
  File "...read_only_property.py", line 11, in <module>
    battery.charge = -10
    ^^^^^^^^^^^^^^^^^
AttributeError: property 'charge' of 'Battery' object has no setter
```

Останній рядок пояснює, що властивість доступна для читання, але не має setter-методу. Зміну треба робити через `use()` або окремий `charge_by()`, де діють правила.

Якщо пряме присвоєння справді є частиною зручного інтерфейсу, можна додати setter:

```
python

@charge.setter
def charge(self, value):
    if not 0 <= value <= self.capacity:
        raise ValueError("Заряд поза допустимими межами")
    self._charge = value
```

Але метод `recharge(amount)` часто краще називає дію, ніж приховане складне присвоєння.

ВЛАСТИВІСТЬ НЕ ПОТРІБНА ДЛЯ КОЖНОГО АТРИБУТА

Публічне `self.name` — нормальний Python. Додавай `_attribute` і `property`, коли є справжня умова цілісності, обчислення або потреба змінити реалізацію без зміни зовнішнього інтерфейсу.

Атрибут класу спільний для всіх об'єктів

```
class_attribute.py

class Robot:
    created_count = 0

    def __init__(self, name):
        self.name = name
        Robot.created_count += 1

first = Robot("Робі")
second = Robot("Іскра")

print(Robot.created_count)
print(second.created_count)
```

`created_count` належить класу й спільний. Читати його через `Robot.created_count` ясніше. Звичайні дані конкретного робота, як `name`, створюють через `self`.

Не використовуй змінний список атрибутом класу для особистих даних:

```
shared_class_list.py

class Backpack:
    items = []

first = Backpack()
second = Backpack()
first.items.append("ключ")
print(second.items)
```

Результат

```
['ключ']
```

Ключ додали через `first`, але його видно й у `second`: усі наплічники отримали той самий список класу. Створюй `self.items = []` у `__init__`.

@classmethod як альтернативний конструктор

Метод класу отримує клас у параметрі `cls`:

```
class_method.py

class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy

    @classmethod
    def fully_charged(cls, name):
        return cls(name, energy=25)

robot = Robot.fully_charged("Робі")
print(f"{robot.name}: {robot.energy}")
```

`fully_charged()` дає назву іншому способу створення. `cls(...)` підтримує похідні класи краще, ніж жорстко записане `Robot(...)`.

`@staticmethod` створює функцію в просторі імен класу без `self` і `cls`. Використовуй її рідко: якщо функція не пов'язана зі станом чи поняттям класу, звичайний модуль часто природніший.

Поліморфізм: одна команда для різних об'єктів

```
polymorphism.py

class WheelVehicle:
    def move_text(self):
        return "Колесо котиться."

class Boat:
    def move_text(self):
        return "Човен пливе."

vehicles = [WheelVehicle(), Boat()]

for vehicle in vehicles:
    print(vehicle.move_text())
```

Класи навіть не мають спільного базового класу, крім загального `object`. Python часто достатньо того, що об'єкт підтримує потрібний метод. Це називають **duck typing**: важлива поведінка, а не формальний ярлик.

Успадкування корисне для спільної реалізації й явного відношення, але не потрібне лише заради однакової назви методу.

Функції `isinstance(obj, Class)` і `issubclass(Child, Parent)` перевіряють формальний зв'язок. Не підмінюй ними нормальний поліморфізм великим ланцюжком перевірок типів.

ОБЕРИ ЗВ'ЯЗОК ЗА ЗМІСТОМ

Заміни пропуски одним із двох підходів: успадкуванням або композицією.

- `ElectricBicycle` ____ `Bicycle`, бо електровелосипед є різновидом велосипеда.
- `Bicycle` ____ об'єкт `Battery`, бо велосипед має батарею всередині.

Потім запиши по одному мінімальному заголовку класу для обох зв'язків.

Типова помилка

Успадкування заради повторного використання двох рядків

Якщо `Report` успадковує `Robot` лише через корисний метод форматування, твердження «звіт є роботом» хибне. Винеси функцію або окремий об'єкт формatera.

Забутий `super().__init__()`

missing_super.py

```
class Vehicle:
    def __init__(self, name, speed):
        self.name = name
        self.speed = speed

    def status(self):
        print(f"{self.name}: {self.speed}")

class ElectricScooter(Vehicle):
    def __init__(self, battery):
        self.battery = battery

scooter = ElectricScooter(80)
scooter.status()
```

Результат

```
Traceback (most recent call last):
  File "...\\missing_super.py", line 16, in <module>
    scooter.status()
  File "...\\missing_super.py", line 7, in status
    print(f"{self.name}: {self.speed}")
    ^^^^^^^^^
AttributeError: 'ElectricScooter' object has no attribute 'name'
```

Виклик успадкованого `status()` дійшов до читання `self.name`, якого похідний конструктор не створив. Або виклич `super().__init__(name, speed)`, або свідомо забезпеч усі обіцянки базового класу.

Похідний клас ламає контракт

Якщо базовий `move_text()` повертає рядок, а один похідний метод повертає `None` і лише друкує, спільний цикл не зможе однаково обробити об'єкти.

ТИПОВА ПОМИЛКА

Не створюй глибоку ієрархію наперед: `Entity -> Actor -> MovingActor -> LivingMovingActor -> Hero`. Спочатку потрібні конкретні два класи й реальна спільна поведінка. Композицію легше змінювати, тому за сумніву перевір варіант «має об'єкт».

Швидка перевірка

Визнач, де успадкування, а де композиція:

```
service_robot.py

class Robot:
    def __init__(self, name):
        self.name = name

class Tool:
    def __init__(self, title):
        self.title = title

class ServiceRobot(Robot):
    def __init__(self, name, tool):
        super().__init__(name)
        self.tool = tool

    def description(self):
        return f"{self.name}: інструмент – {self.tool.title}."

robot = ServiceRobot("Сервіс-1", Tool("ключ"))
print(robot.description())
```

ПЕРЕВІР СЕБЕ

Який зв'язок найкраще описує фразу «самокат має батарею»?

- Композиція: об'єкт `Battery` зберігається в атрибуті самоката.
- Успадкування: `ElectricScooter` повинен бути підкласом `Battery`.
- Глобальна змінна `battery` для всіх самокатів.

Відповідь: Композиція: об'єкт `Battery` зберігається в атрибуті самоката. Батарея є частиною самоката, але самокат не є різновидом батареї. Це відношення «має», тобто композиція.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без підглядання закінчи дві фрази: «Успадкування доречне, коли...» і «Композиція доречна, коли...». Наведи власний приклад для кожної.

Самостійна робота

САМОСТІЙНА РОБОТА

- Створи базовий `Notification` із методом `message_text()` і два похідні класи, які перевизначають формат, але завжди повертають рядок.
- Створи `Flashlight`, який має об'єкт `Battery`. Нехай ліхтар делегує витрату заряду батареї, а не змінює її внутрішній атрибут напругу.
- Додай батареї метод `recharge(amount)` з межами від 0 до `capacity`. Перевір від'ємне значення, переповнення й нормальний заряд.
- Перероби прямий публічний атрибут, для якого справді потрібна перевірка, на `_attribute` з `read-only` property і методами зміни.
- Створи атрибут класу, який рахує кількість створених об'єктів. Поясни, чому особистий стан не можна зберігати там само.
- Додай альтернативний конструктор `from_text()` через `@classmethod`, який створює об'єкт із простого рядка відомого формату.
- Знайди у власному прикладі хибне успадкування за тестом « $X \in Y$ » і заміни його композицією або функцією.
- Створи список різних об'єктів зі спільним методом `status_text()` без спільного класу й оброби його одним циклом.

КОРОТКА ІСТОРИЧНА ПАУЗА

Фраза «віддавай перевагу композиції перед успадкуванням» не забороняє успадкування. Вона нагадує: дерево типів важко перебудувати, а об'єкти-частини можна замінювати. Спершу перевір «має», а «є різновидом» залиш для справді стабільного відношення.

Підсумок

- Успадкування описує відношення «похідний клас є різновидом базового».
- `super()` викликає реалізацію наступного класу в ланцюжку й допомагає не дублювати ініціалізацію.
- Перевизначення дає похідному класу власну сумісну реалізацію методу.
- Композиція описує «об'єкт має інший об'єкт» і делегує йому відповідальність.

- Одне `_` позначає внутрішню деталь за домовленістю; `property` дає контрольоване читання або присвоєння.
- Атрибут класу спільний, атрибут `self` належить конкретному об'єкту.
- `@classmethod` зручний для альтернативного способу створення.
- Поліморфізм дозволяє однаково викликати сумісну поведінку різних об'єктів навіть без формального спільного предка.

РОЗДІЛ 12

Стандартна бібліотека й керована випадковість

Python постачається не лише із синтаксисом, а й із великою **стандартною бібліотекою**. Вона вже містить генератори випадкових чисел, статистику, дати, лічильники, переліки станів і багато інших перевірених інструментів.

Що зробимо

Створимо генератор тренувальних місій. Він використовуватиме окремий об'єкт випадковості, а перевірка зможе передати фіксоване зерно й отримати повторюваний результат.

РЕЗУЛЬТАТ

Випадковість залишиться залежністю, якою керує об'єкт, а не магією, розкиданою по методах. Ти навчишся відрізнати стандартний модуль, сторонній пакет і власний модуль.

ЗАРАЗ У ФОКУСІ

Одна дія: відокремлюй інструмент випадковості від об'єкта, який ним користується, щоб випадковий сценарій можна було повторити в перевірці.

Якщо відволікся, знайди параметр `random_source` у конструкторі генератора.

Три джерела імпортів

```
python

import random
from statistics import mean

from mission import Mission
```

У типовому файлі імпорти групують так:

- 1 стандартна бібліотека Python;
- 2 сторонні пакети, встановлені в середовище;
- 3 власні локальні модулі.

Між групами залишають порожній рядок. Це допомагає одразу побачити, що потребує `pip`, а що приходить разом із Python.

Стандартні random, statistics, collections, datetime, enum, json, pathlib, math окремо не встановлюють.

Майстерня

Створи mission_generator.py:

СПОЧАТКУ ПЕРЕДБАЧ

Два різні об'єкти Random отримують однакове зерно 7. Визнач, чи створять два генератори однакові перші місії і що надрукує останній print().

mission_generator.py

```
import random

class Mission:
    def __init__(self, start, destination, difficulty):
        self.start = start
        self.destination = destination
        self.difficulty = difficulty

    def description(self):
        return (
            f"Маршрут: {self.start} -> {self.destination}, "
            f"складність {self.difficulty}."
        )
```

mission_generator.py · продовження

```
class MissionGenerator:
    def __init__(self, places, random_source=None):
        self.places = places.copy()
        self.random_source = random_source or random.Random()

    def create(self):
        start, destination = self.random_source.sample(self.places, k=2)
        difficulty = self.random_source.randint(1, 5)
        return Mission(start, destination, difficulty)

places = ["Маяк", "Ліс", "Порт", "Вежа"]

first_generator = MissionGenerator(places, random.Random(7))
second_generator = MissionGenerator(places, random.Random(7))

first_description = first_generator.create().description()
second_description = second_generator.create().description()

print(first_description)
print(second_description)
print(f"Однаковий результат: {first_description == second_description}")
```

ЯК ВИПАДКОВІСТЬ СТАЄ КЕРОВАНОЮ

ДО

Є два окремі `Random(7)`; жодна місія ще не створена.

→

ПІСЛЯ

Створено дві окремі, але однакові за даними перші місії; результат перевірки — `True`.

1	Передавання залежності	<code>MissionGenerator(places, random.Random(7))</code>	Кожен генератор отримує власне джерело випадковості з однаковим початковим станом.
2	Вибір місць	<code>random_source.sample(.., k=2)</code>	Однаковий стан джерел дає однакову першу пару місць.
3	Вибір складності	<code>random_source.randint(1, 5)</code>	Наступне однакове рішення дає однакову складність.
4	Створення	<code>return Mission(...)</code>	Обидва генератори збирають рівні за даними місії.
5	Перевірка	<code>first_description == second_description</code>	Порівняння двох описів повертає <code>True</code> .

Що тут важливо: Зерно не прибирає випадковий алгоритм, а дає відтворювану послідовність; передана залежність робить це локальним і тестованим.

Два різні об'єкти `Random` отримали однакове зерно 7, тому створили однакову послідовність рішень. У звичайному запуску `random_source` не передається й використовується нове непередбачуване для користувача джерело.

Залежність передано в конструктор

python

```
def __init__(self, places, random_source=None):
    self.places = places.copy()
    self.random_source = random_source or random.Random()
```

Генератор не звертається до глобального `random.choice()` у випадкових місцях. Він зберігає один сумісний об'єкт. Це **впровадження залежності** (*dependency injection*): потрібний інструмент можна передати ззовні.

Назва звучить складно, але дія проста: не створюй незамінний інструмент глибоко всередині кожного методу, якщо перевірці або іншому сценарію може знадобитися контрольована версія.

Основні інструменти `random`

Випадковий елемент

random_choice.py

```
import random

rng = random.Random(3)
directions = ["північ", "південь", "схід", "захід"]
direction = rng.choice(directions)

print(f"Обрано: {direction}")
print(f"Обрано відомий напрямок: {direction in directions}")
```

`choice(sequence)` потребує непорожню послідовність і повертає один елемент.

Ціле число з обома межами

random_int.py

```
import random

rng = random.Random(5)
roll = rng.randint(1, 6)

print(f"Випало: {roll}")
print(f"Число в межах кубика: {1 <= roll <= 6}")
```

`randint(1, 6)` може повернути і 1, і 6. Це відрізняється від `range()`, де права межа не включається.

Еквівалент через `randrange()`:

python

```
rng.randrange(1, 7)
```

`randrange()` працює за правилами `range` й не включає `stop`.

Кілька різних елементів

random_sample.py

```
import random

rng = random.Random(2)
picked = rng.sample(["A", "B", "C", "D"], k=2)

print(picked)
print(f"Кількість: {len(picked)}")
print(f"Усі різні: {len(set(picked)) == len(picked)}")
```

`sample()` повертає новий список без повторного вибору того самого елемента. `k` не може бути більшим за кількість доступних значень.

Для вибору з можливими повторами є `choices(..., k=...)`.

Перемішування на місці

random_shuffle.py

```
import random

cards = [1, 2, 3, 4]
original = cards.copy()
rng = random.Random(4)
rng.shuffle(cards)

print(cards)
print(f"Ті самі карти: {sorted(cards) == sorted(original)}")
```

`shuffle()` змінює список і повертає `None`, як `list.sort()`. Якщо початковий порядок треба зберегти, перемішуй копію.

Дробове число

python

```
rng.random()
rng.uniform(1.5, 3.5)
```

`random()` дає `float` від 0 включно до 1 не включно. `uniform(a, b)` — дробове число між межами; точна поведінка крайньої межі залежить від округлення, тому не будуй на ній логіку точного включення.

Зерно не робить результат випадковішим

Псевдовипадковий генератор створює довгу послідовність за алгоритмом. Однаковий початковий стан — **seed**, або зерно — дає однакову послідовність.

Це корисно для:

- повторюваних тестів;
- відтворення помилки;
- однакової генерації навчального прикладу;
- збереження процедурного світу за кодом зерна.

Не став постійне зерно у звичайній програмі, якщо користувач очікує новий результат кожного запуску. Передавай його лише в перевірці або як усвідомлену функцію продукту.

НЕ ДЛЯ ПАРОЛІВ І ТОКЕНІВ

Модуль `random` не призначений для секретів. Його послідовність відтворювана. Для паролів, токенів і кодів безпеки використовують стандартний модуль `secrets`, наприклад `secrets.choice()` або `secrets.token_urlsafe()`.

Керована перевірка без вгадування

Можна передати не справжній генератор, а маленький передбачуваний об'єкт із потрібними методами:

fake_random.py

```
class FixedRandom:
    def sample(self, values, k):
        return ["Ліс", "Порт"]

    def randint(self, start, stop):
        return 4

class Mission:
    def __init__(self, start, destination, difficulty):
        self.start = start
        self.destination = destination
        self.difficulty = difficulty

    def description(self):
        return (
            f"Маршрут: {self.start} -> {self.destination}, "
            f"складність {self.difficulty}."
        )

class MissionGenerator:
```

fake_random.py · продовження

```
    def __init__(self, places, random_source):
        self.places = places
        self.random_source = random_source

    def create(self):
        start, destination = self.random_source.sample(self.places, k=2)
        difficulty = self.random_source.randint(1, 5)
        return Mission(start, destination, difficulty)

generator = MissionGenerator(["Ліс", "Порт"], FixedRandom())
print(generator.create().description())
```

MissionGenerator не питає, якого класу random_source. Йому потрібні сумісні методи sample() і randint(). Це практичний поліморфізм і основа майбутніх тестових підмін.

statistics: готові обчислення

statistics_example.py

```
from statistics import mean, median, mode

scores = [2, 4, 4, 6]

print(f"Середнє: {mean(scores):g}")
print(f"Медіана: {median(scores):g}")
print(f"Найчастіше: {mode(scores):g}")
```

- `mean()` — арифметичне середнє;
- `median()` — середнє позиційне значення після впорядкування;
- `mode()` — найчастіше значення.

Порожні дані або неоднозначні припущення треба обробляти відповідно до задачі. Читай документацію функції, а не вгадуй поведінку за назвою.

`collections.Counter`: **СЛОВНИК ЧАСТОТ**

`counter_example.py`

```
from collections import Counter

topics = ["python", "oop", "python", "loops", "python"]
counts = Counter(topics)

print(f"python: {counts['python']}")
print(f"loops: {counts['loops']}")
```

`Counter` поводитья як спеціалізований словник лічильників. Для відсутнього ключа повертає 0. Він також має `most_common()`:

`python`

```
top_two = counts.most_common(2)
```

Використання готового типу зменшує власний код, але варто розуміти базову словникову модель, яку він розширює.

Enum: названий набір станів

Рядки "new", "active", "done" легко написати з помилкою. Перелік Enum дає стани з автодоповненням:

```
enum_status.py

from enum import Enum, auto

class TaskStatus(Enum):
    NEW = auto()
    ACTIVE = auto()
    DONE = auto()

class Task:
    def __init__(self, title):
        self.title = title
        self.status = TaskStatus.NEW

    def start(self):
        self.status = TaskStatus.ACTIVE

    def status_text(self):
        if self.status is TaskStatus.ACTIVE:
            return "Завдання виконується."
        if self.status is TaskStatus.DONE:
            return "Завдання завершено."
```

```
enum_status.py · продовження

        return "Завдання нове."

task = Task("Практика")
task.start()
print(task.status_text())
```

Члени переліку порівнюють через `is` або `==`. `auto()` створює внутрішні значення, бо конкретні числа тут не важливі.

Не створюй Enum для двох станів, які природно виражає `bool`, наприклад `is_open`. Перелік корисний для трьох і більше взаємовиключних названих станів.

Дата й час

```
date_example.py

from datetime import date

lesson_day = date(2026, 8, 16)

print(lesson_day.isoformat())
print(lesson_day.year)
```

Об'єкти `date` краще за ручні рядки, коли треба порівнювати дні або додавати проміжки. Для поточної дати є `date.today()`, але приклади й тести з фіксованою датою повторюваніші.

Часові пояси — окрема важлива тема. Для реальних моментів часу не змішуй наївні `datetime` з даними різних поясів; використовуй `timezone-aware` об'єкти й чітко фіксуй зону.

Як читати документацію модуля

Не треба запам'ятовувати всю бібліотеку. Для нового інструмента перевір:

- 1 Чи модуль входить до стандартної бібліотеки потрібної версії Python?
- 2 Які типи приймає функція?
- 3 Що повертає і чи змінює аргумент?
- 4 Які винятки можливі?
- 5 Чи є приклад мінімального використання?
- 6 Чи підходить інструмент для безпеки й точності конкретної задачі?

У VS Code наведи курсор на ім'я або використай **Go to Definition**. Для повного контракту переходь до офіційної документації Python.

Сторонній пакет: коли він справді потрібен

Сторонній пакет не входить до стандартного Python. Перед додаванням:

- сформулюй проблему, яку він розв'язує;
- перевір, що пакет підтримується й має документацію;
- використовуй активну `.venv`;
- зафіксуй залежність;
- не підключай експериментальний інструмент до базового навчального матеріалу без причини.

Встановлення стабільного пакета в активне середовище:

```
powershell
```

```
python -m pip install package_name
```

`package_name` — заповнювач, а не справжня назва для копіювання. Підстав точну назву вибраного стабільного пакета з його офіційної документації. Після встановлення імпорт може мати іншу назву; її також треба взяти з документації, а не вгадувати.

ЗАЛИШ ВИПАДКОВІСТЬ ЗАМІННОЮ

Заповни пропуски так, щоб звичайний запуск створював власний Random, а тест міг передати підготовлений.

```
python

class Picker:
    def __init__(self, random_source=____):
        self.random_source = random_source or random.____()

    def choose(self, items):
        return self.random_source.____(items)
```

Типова помилка

Глобальне random.seed() змінює весь модуль

```
python

import random

random.seed(7)
```

Це змінює глобальний генератор модуля, яким можуть користуватися інші частини програми. Для ізолизованого компонента краще `rng = random.Random(7)` і передавання `rng` об'єкту.

Змішано choice() і sample()

Два окремі виклики `choice()` можуть вибрати те саме місце двічі. Якщо старт і фініш мають різнитися, `sample(places, k=2)` прямо виражає правило.

Встановлено назву стандартного модуля

Не запускай `pip install random` або `pip install statistics` для звичайного Python. Спершу спробуй імпорт у чистому файлі й перевір офіційну документацію.

ТИПОВА ПОМИЛКА

Не перевіряй випадковий код твердженням «результат має бути саме 4», якщо джерело не зафіксоване. Передай seeded Random або контрольовану підміну, тоді очікування стає частиною контракту.

Швидка перевірка

Два генератори з різними зернами повинні мати незалежний стан:

```
independent_random.py

import random

first = random.Random(1)
second = random.Random(2)

print([first.randint(1, 3) for _ in range(3)])
print([second.randint(1, 3) for _ in range(3)])
```

ПЕРЕВІР СЕБЕ

Навіщо передавати MissionGenerator окремий random_source?

- Щоб кожна місія завжди була однаковою для користувача.
- Щоб звичайний запуск мав випадковість, а перевірка могла підставити повторюване джерело.
- Щоб модуль random став стороннім пакетом.

Відповідь: Щоб звичайний запуск мав випадковість, а перевірка могла підставити повторюване джерело. Явна залежність дозволяє керувати випадковістю без зміни логіки генератора.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий приклад і поясни різницю між стандартним модулем, стороннім пакетом і власним модулем. Потім скажи, навіщо передавати random_source, якщо можна викликати глобальний random.

Самостійна робота

САМОСТІЙНА РОБОТА

- Створи генератор кидків кубика з переданим `random.Random`. Перевір однакові серії для однакових зерен і різні — для різних.
- Згенеруй випадкову команду з трьох різних імен через `sample()`. Оброби випадок, коли доступних імен менше трьох.
- Зроби об'єкт `Deck`, який зберігає карти, перемішує копію й роздає через `pop()`. Початковий список зовні не повинен змінюватися.
- Порахуй частоти слів через звичайний словник, а потім через `Counter`. Порівняй обсяг коду й структуру результату.
- Створи `Enum` для станів замовлення `NEW`, `PAID`, `SENT`, `DELIVERED`. Методи мають дозволяти лише логічні переходи.
- Передай генератору маленький `FixedRandom`, який повертає заготовлені значення. Перевір поведінку без справжньої випадковості.
- Знайди в офіційній документації один стандартний модуль для власної задачі. Запиши його вхід, результат, можливу помилку й мінімальний приклад.
- Поясни, чому `random` не підходить для коду відновлення пароля, і створи окремий безпечний приклад із `secrets.token_urlsafe()` без публікації отриманого токена.

КОРОТКА ІСТОРИЧНА ПАУЗА

Псевдовипадкова послідовність детермінована, але для людини виглядає хаотично. Саме повторюваність робить її корисною в моделюванні й тестах. Криптографічна випадковість має іншу мету й суворіші вимоги, тому Python розділяє `random` і `secrets`.

Підсумок

- Стандартна бібліотека постачається з Python; сторонні пакети встановлюють окремо у віртуальне середовище.
- `Random` можна створити як незалежний об'єкт і передати компоненту.
- Однакове зерно дає повторювану послідовність, що корисно для перевірок і відтворення помилок.
- `choice()`, `randint()`, `sample()`, `shuffle()` і `random()` мають різні контракти.
- `random` не підходить для секретів; для них існує `secrets`.
- `statistics`, `Counter`, `Enum` і `datetime` розв'язують поширені задачі без власного повторного винаходу.
- Перед додаванням залежності перевір призначення, підтримку, документацію й спосіб відтворення середовища.

- Випадкову залежність краще передавати явно, щоб об'єкт лишався керованим і тестованим.

Офіційні орієнтири: `random`, `secrets`, `statistics`, `collections.Counter`, `enum`.

РОЗДІЛ 13

Текстові файли, UTF-8 і шляхи

Стан у пам'яті зникає після завершення програми. Текстовий файл дозволяє зберегти нотатку, журнал або налаштування між запусками. Головна складність тут не `read()` чи `write()`, а точна відповідь: **який файл, у якій папці й у якому кодуванні**.

Що зробимо

Створимо `TextNotebook`, який отримує шлях ззовні, додає нотатки й читає їх. Він не вгадуватиме поточну папку та завжди використовуватиме UTF-8.

РЕЗУЛЬТАТ

Ти зможеш відрізнити шлях від вмісту файла, відносний шлях від абсолютного й поточну робочу папку від папки модуля.

ЗАРАЗ У ФОКУСІ

Одна дія: будуй шлях із частин через `Path`, а читання чи запис виконуй лише після того, як можеш назвати точний файл.

Якщо відволікся, повернися до `Path("notes") / "lesson.txt"` і розділи папку, ім'я та суфікс.

Path представляє шлях як об'єкт

СПОЧАТКУ ПЕРЕДБАЧ

Не запускаючи код, визнач три результати для `path.parent`, `path.name` і `path.suffix`. Поясни, чому жодна папка або файл від цих рядків ще не створюється.

```
path_basics.py

from pathlib import Path

path = Path("notes") / "lesson.txt"

print(path.parent)
print(path.name)
print(path.suffix)
```

ОБ'ЄКТ ШЛЯХУ ОПИСУЄ АДРЕСУ ЧАСТИНАМИ

ДО

Є лише два текстові фрагменти:
папка `notes` та ім'я `lesson.txt`.

→

ПІСЛЯ

Об'єкт `path` описує адресу; на диску
нічого не створено й не змінено.

1	Побудова	<code>Path("notes") / "lesson.txt"</code>	<code>Path</code> складає платформонезалежний об'єкт шляху; файл ще не відкривається.
2	Батьківська папка	<code>path.parent</code>	Результат описує <code>notes</code> .
3	Ім'я	<code>path.name</code>	Результат описує останню частину <code>lesson.txt</code> .
4	Суфікс	<code>path.suffix</code>	Результат описує розширення <code>.txt</code> .

Що тут важливо: `Path` спершу дає безпечний опис місця, а окрема операція читання або запису вже працює з файловою системою.

Оператор `/` у `pathlib` складає частини шляху. Це не ділення: ліва частина є об'єктом `Path`.

- `path.parent` — батьківська папка;
- `path.name` — повна назва файла;
- `path.stem` — назва без останнього розширення;
- `path.suffix` — останнє розширення;
- `path.parts` — кортеж частин.

`Path` сам використовує правила поточної операційної системи. Не треба вручну склеювати рядки через `"/"` або `"\\"`.

Майстерня

Створи `text_notebook.py`:

```
text_notebook.py

from pathlib import Path

class TextNotebook:
    def __init__(self, path):
        self.path = Path(path)

    def replace_all(self, notes):
        text = "\n".join(notes) + "\n"
        self.path.write_text(text, encoding="utf-8")

    def add(self, note):
        with self.path.open("a", encoding="utf-8") as file:
            file.write(note.rstrip() + "\n")

    def read_all(self):
        if not self.path.exists():
            return []
        text = self.path.read_text(encoding="utf-8")
        return text.splitlines()
```

```
text_notebook.py · продовження

notebook = TextNotebook("lesson_notes.txt")
notebook.replace_all(["Перша нотатка"])
notebook.add("Об'єкти тримають дані й поведінку разом.")

for note in notebook.read_all():
    print(note)
```

Перевірка запускає приклад у тимчасовій папці, тому він не зачіпає твої особисті файли. Під час ручної роботи `lesson_notes.txt` з'явиться в поточній папці терміналу.

Шлях передано об'єкту

```
python

def __init__(self, path):
    self.path = Path(path)
```

Клас не зашиває назву глобально. Один об'єкт може працювати з `lesson_notes.txt`, інший — із `ideas.txt`. Перевірка може передати шлях до окремої тимчасової папки.

`write_text()` перезаписує файл

```
python

self.path.write_text(text, encoding="utf-8")
```

Якщо файла немає, він створюється. Якщо є — попередній вміст **повністю замінюється**. Метод повертає кількість записаних символів, але часто цей результат не потрібен.

Перед записом у реальний важливий файл перевіряй шлях. Автоматичний запис не має мовчки обирати файл користувача лише за схожою назвою.

Режим "a" додає в кінець

```
python
```

```
with self.path.open("a", encoding="utf-8") as file:
    file.write(note.rstrip() + "\n")
```

Режими:

- "r" — читання; файл має існувати;
- "w" — запис із повною заміною;
- "a" — додавання в кінець;
- "x" — створення нового файла, помилка, якщо він уже є;
- додаткова "b" — двійковий режим, наприклад "rb".

with є **контекстним менеджером**. Після виходу з блоку файл закривається навіть за помилки. Не треба вручну викликати `file.close()`.

splitlines() прибирає символи кінця рядка

`read_text()` повертає один великий рядок. Метод `splitlines()` створює список логічних рядків без `\n` або `\r\n` наприкінці.

```
splitlines_example.py
```

```
text = "A\n\nB\n"
print(text.splitlines())
```

Порожній рядок усередині зберігається, а фінальний роздільник не створює зайвого елемента.

ПОРАХУЙ НЕПОРОЖНІ НОТАТКИ

Додай метод:

```
python
```

```
def count_nonempty(self):
    return sum(1 for note in self.read_all() if note.strip())
```

Перевір файл із порожнім рядком між двома нотатками.

UTF-8 робить текст передбачуваним

Комп'ютер зберігає байти. **Кодування** визначає, як перетворити символи на байти й назад. UTF-8 підтримує українські літери, латиницю, більшість писемностей і звичні символи.

Завжди вказуй:

```
python

encoding="utf-8"
```

для власних текстових форматів. Так поведінка не залежить від системних налаштувань Windows, Linux або macOS.

Якщо байти записані в одному кодуванні, а прочитані в іншому, можливий `UnicodeDecodeError` або спотворені символи. Правильне рішення — знати кодування джерела, а не пробувати випадкові варіанти до зникнення помилки.

Символ кінця рядка

Історично системи використовують різні послідовності:

- Windows часто `\r\n`;
- Linux і macOS — `\n`;
- старі версії macOS мали інший формат, який у сучасних проєктах майже не трапляється.

У текстовому режимі Python здебільшого нормалізує ці відмінності. `splitlines()` працює з різними завершеннями. Git може окремо налаштовувати перетворення кінців рядків.

Відносний і абсолютний шлях

```
absolute_path.py

from pathlib import Path

relative = Path("data") / "notes.txt"
absolute = relative.resolve()

print(absolute.is_absolute())
```

Відносний шлях не містить повного маршруту від кореня системи. Python тлумачить його від поточної робочої папки.

Абсолютний шлях містить повний маршрут:

```
text

C:\Users\Student\Project\data\notes.txt
/home/student/project/data/notes.txt
/Users/student/project/data/notes.txt
```

Не записуй особистий абсолютний шлях у код, який має працювати на іншому комп'ютері. Будуй маршрут від відомої основи.

Поточна робоча папка

```
current_folder.py

from pathlib import Path

current_folder = Path.cwd()
print(current_folder.is_dir())
```

`Path.cwd()` показує, звідки процес запущено. У VS Code це часто корінь відкритої проєктної папки, але не гарантується кнопкою чи кодом автоматично.

У терміналі перевір:

ПОТОЧНА ПАПКА Й СПИСОК ФАЙЛІВ

Windows PowerShell

```
powershell

pwd
Get-ChildItem
```

Короткі `ls` і `dir` у PowerShell також працюють як псевдоніми.

Windows Command Prompt

```
bat

cd
dir
```

macOS i Linux

```
bash

pwd
ls
```

Папка самого модуля

Коли ресурс лежить поруч із кодом незалежно від місця запуску:

```
python

from pathlib import Path

MODULE_DIR = Path(__file__).resolve().parent
DATA_FILE = MODULE_DIR / "data" / "defaults.txt"
```

`__file__` містить шлях поточного модуля під час звичайного запуску або імпорту. В інтерактивному REPL та деяких середовищах його може не бути, тому не використовуй цей шаблон без розуміння контексту.

Для даних усередині встановленого пакета є стандартний `importlib.resources`; просте припущення про сусідню папку не завжди працює після пакування.

Створення папок і перевірка типу шляху

```
make_folder.py

from pathlib import Path

folder = Path("output") / "notes"
folder.mkdir(parents=True, exist_ok=True)

file_path = folder / "summary.txt"
file_path.write_text("Готово\n", encoding="utf-8")

print(folder.is_dir())
print(file_path.is_file())
```

- `parents=True` дозволяє створити відсутні батьківські папки;
- `exist_ok=True` не вважає помилкою вже наявну папку;
- `exists()` перевіряє будь-який шлях;
- `is_file()` — звичайний файл;
- `is_dir()` — папку.

Не плутай «існує» з «безпечний для перезапису». Перед руйнівною заміною потрібне окреме рішення.

Читання великих файлів рядок за рядком

`read_text()` завантажує весь файл у пам'ять. Для великого журналу перебирай файловий об'єкт:

```
line_by_line.py

from pathlib import Path

path = Path("events.log")
path.write_text("START\n\nSTOP\n", encoding="utf-8")

nonempty_count = 0
with path.open("r", encoding="utf-8") as file:
    for line in file:
        if line.strip():
            nonempty_count += 1

print(nonempty_count)
```

На кожному кроці `line` зазвичай містить символ кінця рядка. `strip()` прибирає пробіли з обох країв; `rstrip("\n")` прибирає лише конкретний символ праворуч.

Двійкові файли — не текст

Зображення, PDF, аудіо й архіви читають у двійковому режимі:

```
python

data = Path("image.png").read_bytes()
```

або:

```
python

with Path("image.png").open("rb") as file:
    header = file.read(8)
```

Не декодуй довільні байти як UTF-8. Формат файла визначає, як їх тлумачити.

Різниця запису шляхів у системах

`Path("data") / "file.txt"` працює всюди. Але в терміналі та зовнішніх абсолютних шляхах є відмінності.

ПРИКЛАДИ ШЛЯХІВ

Windows

- диск має літеру: C:\Projects\Robot;
- роздільник традиційно \;
- зворотна риска в Python-рядку запускає escape-послідовність.

Якщо абсолютний Windows-шлях справді потрібен у коді, raw-рядок зручніший:

```
python

path = Path(r"C:\Projects\Robot\data.txt")
```

Не завершуй raw-рядок одинарною зворотною рискою. Ще краще — складай частини через Path.

Linux

- корінь починається з /;
- типовий домашній шлях: /home/name;
- назви файлів чутливі до регістру: Data.txt і data.txt різні.

macOS

- синтаксис шляхів Unix-подібний: /Users/name;
- файлова система часто поводить нечутливо до регістру за типовим налаштуванням, але це не гарантія для всіх дисків.

У переносному проєкті завжди вважай регістр важливим і точно повторюй назву файла.

Домашню папку можна отримати переносно:

```
python

home = Path.home()
```

Але не записуй туди дані мовчки. Шлях має бути очікуваним для користувача й відповідати призначенню програми.

ВІДНОВИ БЕЗПЕЧНИЙ ПОРЯДОК РОБОТИ З ФАЙЛОМ

Розташуй дії в логічному порядку: спочатку адреса й папка, потім запис, після нього читання.

```
python

text = path.read_text(encoding="utf-8")
path.parent.mkdir(parents=True, exist_ok=True)
path = Path("notes") / "lesson.txt"
path.write_text("Привіт", encoding="utf-8")
```

Поясни, який крок гарантує існування батьківської папки.

Типова помилка

Файл існує, але програма «не знаходить»

Найчастіше файл лежить не відносно поточної робочої папки. Надрукуй діагностику:

```
python

print(Path.cwd())
print(path.resolve())
print(path.exists())
```

Не переносись одразу до абсолютного особистого шляху. Виправ точку запуску або побудуй шлях від відомої основи.

Запис без encoding

Код може працювати на одному Windows-комп'ютері й зламатися на іншому середовищі. Для власного тексту явно вказуй UTF-8 при читанні й записі.

write_text() випадково стер попередній вміст

Метод замінює файл повністю. Для журналу використовуй режим "a", для безпечного створення нового — "x", для оновлення структури — спершу прочитай, зміни дані в пам'яті й усвідомлено запиши нову версію.

ТИПОВА ПОМИЛКА

Не перевіряй шлях лише через `exists()`, якщо далі очікуєш файл. На місці може бути папка. Використовуй `is_file()` або `is_dir()` відповідно до контракту.

Швидка перевірка

Спрогнозуй, що буде у файлі після двох записів:

```
file_modes.py

from pathlib import Path

path = Path("letters.txt")
path.write_text("A\n", encoding="utf-8")

with path.open("a", encoding="utf-8") as file:
    file.write("B\n")

print(path.read_text(encoding="utf-8"), end="")
```

ПЕРЕВІР СЕБЕ

Від якої папки зазвичай рахується відносний шлях Path("data/file.txt")?

- Завжди від домашньої папки користувача.
- Від поточної робочої папки процесу.
- Завжди від папки, де встановлено Python.

Відповідь: Від поточної робочої папки процесу.

Поточну робочу папку показує Path.cwd(). Для ресурсу біля модуля основу будують окремо через __file__.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без підглядання назви три властивості Path, які показують папку, ім'я і суфікс. Потім поясни різницю між відносним та абсолютним шляхом.

Самостійна робота

САМОСТІЙНА РОБОТА

- Створи TextNotebook для двох різних файлів і доведи, що кожен об'єкт читає лише свій шлях.
- Додай метод `replace_all()` із режимом безпечного створення: якщо файл уже є, метод повертає `False` і не перезаписує його.
- Створи папку `output/reports` через `mkdir(parents=True, exist_ok=True)` і запиши туди UTF-8 файл з українським текстом.
- Запусти один скрипт із кореня проєкту та з іншої папки. Порівняй `Path.cwd()`, відносний `.resolve()` і шлях, побудований від `__file__`.
- Прочитай файл із порожніми рядками двома способами: `read_text().splitlines()` та циклом по файловому об'єкту. Порівняй результат.
- Створи Windows-приклад шляху з raw-рядком і переносний еквівалент через `Path` та `/`. На Linux/macOS запиши відповідний абсолютний приклад, але не зашивай його в робочу логіку.
- Навмисно прочитай відсутній файл і збережи `traceback`. Не виправляй через випадковий абсолютний шлях — визнач правильну основу.
- Запиши чотири байти через `write_bytes()`, прочитай їх через `read_bytes()` і поясни, чому до них не застосовується `encoding`.

КОРОТКА ІСТОРИЧНА ПАУЗА

`pathlib` з'явився, щоб шлях був не крихким рядком, а об'єктом із діями: скласти, знайти батька, перевірити тип, прочитати. Це добре узгоджується з усією моделлю курсу: дані та пов'язана поведінка живуть разом.

Підсумок

- `Path` представляє шлях і переносно складає його частини через `/`.
- `read_text()` читає весь текст, `write_text()` повністю замінює файл, режим `"a"` додає.
- Контекстний менеджер `with` гарантує закриття файлового об'єкта.
- Для власних текстових файлів явно використовуй `encoding="utf-8"`.
- Відносний шлях рахується від `Path.cwd()`; шлях ресурсу біля модуля можна будувати від `__file__`.
- `mkdir()`, `exists()`, `is_file()` та `is_dir()` відповідають за різні перевірки.
- Windows, Linux і macOS мають різний вигляд абсолютних шляхів, але `pathlib` приховує більшість відмінностей.
- Запис може перезаписати дані, тому точний шлях і режим є частиною контракту.

Офіційні орієнтири: [pathlib](#), читання й запис файлів, Unicode у Python.

РОЗДІЛ 14

Винятки: передбачувані збої й корисні повідомлення

Файл може бути відсутній, користувач може ввести п'ять замість 5, а метод — отримати неможливе значення. Python повідомляє про такі ситуації через **винятки**. Завдання програми не приховати всі помилки, а відрізнити очікуваний збій від справжнього дефекту.

Що зробимо

Створимо EnergyReader, який читає число з файла. Нижчий рівень повідомлятиме точну проблему винятком, а зовнішній сценарій перетворюватиме очікувані збої на зрозумілий результат.

РЕЗУЛЬТАТ

Ти навчишся ловити лише ті винятки, які справді можеш обробити, залишати неочікувані дефекти видимими й додавати контекст без втрати першопричини.

ЗАРАЗ У ФОКУСІ

Одна дія: відрізняй передбачувану невдалу операцію від помилки у власній логіці й перехоплюй лише конкретний очікуваний виняток.

Якщо відволікся, повернися до рядка, який може підняти ValueError.

Traceback показує шлях до винятку

```
invalid_number.py  
  
number = int("п'ять")
```

Повний короткий traceback виглядає так; шлях скорочено:

```
Результат  
  
Traceback (most recent call last):  
  File "...invalid_number.py", line 1, in <module>  
    number = int("п'ять")  
              ^^^^^^^^^^^  
ValueError: invalid literal for int() with base 10: 'п'ять'
```

- ValueError — тип винятку;
- далі — повідомлення;

- вище — ланцюжок викликів і рядки коду.

Спочатку читай останній рядок, потім шукай **перший кадр свого коду**, який передав неправильне значення. Не редагуй рядки стандартної бібліотеки.

try і точний except

СПОЧАТКУ ПЕРЕДБАЧ

Рядок "п'ять" не є записом цілого числа. Перед запуском визнач, який рядок підніме виняток, який блок його перехопить і що побачить користувач.

safe_number.py

```
text = "п'ять"

try:
    number = int(text)
except ValueError:
    print("Потрібне ціле число.")
```

ВИНЯТОК ЗМІНЮЄ ЗВИЧАЙНИЙ МАРШРУТ ВИКОНАННЯ

ДО
text = "п'ять"; змінної number ще немає.

→

ПІСЛЯ
Програма завершується
контрольовано; number не створено,
надруковано корисне
повідомлення.

1	Спроба	number = int(text)	int() не може перетворити цей текст і піднімає ValueError.
2	Перехід	except ValueError:	Python пропускає решту блоку try і шукає відповідний обробник.
3	Корисне повідомлення	print("Потрібне ціле число.")	Обробник пояснює очікуваний формат без технічного traceback для користувача.

Що тут важливо: try не робить операцію безпомилковою — він описує окремий маршрут для конкретного передбачуваного збою.

Python виконує блок try. Якщо саме ValueError виникає до завершення блоку, виконання переходить у відповідний except.

Не клади в try десятки не пов'язаних рядків. Чим менший блок, тим ясніше, яка операція очікувано може зламатися.

Майстерня

Створи `energy_reader.py`:

`energy_reader.py`

```
from pathlib import Path

class EnergyDataError(ValueError):
    """Непридатні дані енергії у зовнішньому джерелі."""

class EnergyReader:
    def read(self, path):
        path = Path(path)

        try:
            text = path.read_text(encoding="utf-8")
        except FileNotFoundError as error:
            raise EnergyDataError("файл не знайдено") from error

        try:
            energy = int(text.strip())
        except ValueError as error:
            raise EnergyDataError("значення має бути цілим числом") from error

        if energy < 0:
```

`energy_reader.py` · продовження

```
        raise EnergyDataError("енергія не може бути від'ємною")

    return energy

def load_message(reader, path):
    try:
        energy = reader.read(path)
    except EnergyDataError as error:
        return f"Не вдалося прочитати енергію: {error}."
    else:
        return f"Зчитано енергію: {energy}."

valid_path = Path("valid_energy.txt")
invalid_path = Path("invalid_energy.txt")

valid_path.write_text("25\n", encoding="utf-8")
invalid_path.write_text("багато\n", encoding="utf-8")

reader = EnergyReader()
print(load_message(reader, valid_path))
```

`energy_reader.py` · продовження

```
print(load_message(reader, invalid_path))
print(load_message(reader, "missing_energy.txt"))
```

Тут два рівні мають різні ролі:

- `EnergyReader.read()` знає контракт даних і піднімає `EnergyDataError` із точною причиною;

- `load_message()` є межею інтерфейсу й вирішує, як показати очікувану проблему людині.

Власний виняток називає проблему предметної області

```
python

class EnergyDataError(ValueError):
    """Непридатні дані енергії у зовнішньому джерелі."""
```

Клас успадковує `ValueError`, бо проблема стосується непридатного значення. Суфікс `Error` є звичною домовленістю для винятків.

Власний тип дозволяє зовнішньому коду ловити саме проблеми енергетичних даних, не коштаючи будь-який `ValueError` із випадкового дефекту форматування в іншому місці.

raise піднімає виняток

```
python

if energy < 0:
    raise EnergyDataError("енергія не може бути від'ємною")
```

Метод відмовляється створювати некоректний результат. Виняток — частина контракту: або повертається коректне невід'ємне число, або піднімається `EnergyDataError`.

Не використовуй `raise` для звичайної гілки на кшталт «предмета немає в наплічнику», якщо це очікувана повсякденна відповідь методу. Для неї часто природні `False`, `None` або об'єкт результату. Виняток доречний, коли нормальне продовження операції неможливе.

raise ... from ... зберігає причину

```
python

except FileNotFoundError as error:
    raise EnergyDataError("файл не знайдено") from error
```

Новий виняток додає контекст, а `from error` зберігає першопричину в `traceback`. Під час налагодження видно обидва рівні.

Якщо навмисно хочеш приховати технічний ланцюжок від `traceback`, існує `raise NewError(...) from None`, але застосовуй це лише на межі з добре зрозумілою причиною.

else — шлях без винятку

python

```
try:
    energy = reader.read(path)
except EnergyDataError as error:
    return f"Помилка: {error}"
else:
    return f"Енергія: {energy}"
```

Блок `else` виконується, якщо `try` завершився без винятку. Він тримає успішну логіку поза `try`, тому випадковий збій у формуванні повідомлення не буде помилково сприйнятий як збій читання.

finally виконується завжди

finally_example.py

```
def work():
    try:
        print("Починаю роботу.")
        raise RuntimeError("збій")
    finally:
        print("Ресурс звільнено.")

try:
    work()
except RuntimeError:
    print("Помилку передано далі.")
```

`finally` виконується за успіху, винятку, `return` або `break`. Він потрібен для звільнення ресурсу, відновлення тимчасового стану чи завершального журналювання.

Для файлів використовуй `with`, бо контекстний менеджер уже правильно закриває ресурс. Не додавай `finally` там, де `with` виражає намір простіше.

ДОДАЙ ВЕРХНЮ МЕЖУ

Розшир `EnergyReader`: значення понад 100 теж має піднімати `EnergyDataError` ("енергія перевищує 100"). Перевір 100, 101 і від'ємне число як окремі межі.

Кілька очікуваних типів

```
multiple_exceptions.py

from pathlib import Path

def read_number(path):
    try:
        text = Path(path).read_text(encoding="utf-8")
        return int(text)
    except FileNotFoundError:
        return "Файл відсутній."
    except ValueError:
        return "Дані не є числом."

bad_path = Path("bad_number.txt")
bad_path.write_text("hi", encoding="utf-8")

print(read_number("missing.txt"))
print(read_number(bad_path))
```

Різні ехсепт дозволяють різні реакції.

Якщо реакція однакова, типи можна об'єднати кортежем:

```
python

except (FileNotFoundError, PermissionError) as error:
    return f"Файл недоступний: {error}"
```

Не об'єднуй типи лише заради короткості, якщо користувачеві потрібні різні наступні дії.

Ієрархія винятків і порядок ехсепт

Більш конкретні типи ставлять першими:

```
python

try:
    value = int(text)
except ValueError:
    print("Неправильне число")
except Exception:
    print("Інша неочікувана помилка")
```

Exception є широким базовим класом для багатьох звичайних винятків. Якщо поставити його першим, конкретний ValueError ніколи не дійде до свого блоку.

Широкий ехсепт Exception може бути доречний на зовнішній межі довготривалого сервісу, де помилку **журнують із traceback** і безпечно ізолюють одну задачу. Він не повинен перетворювати дефекти на мовчазне «щось пішло не так» всередині звичайного методу.

Уникай голого:

```
python

try:
    risky_work()
except:
    pass
```

Цей код може «успішно» завершитися без жодного рядка в терміналі. Саме це небезпечно: він може перехопити навіть сигнали завершення, приховує причину й залишає невідомий стан.

Поширені винятки початкових програм

- `SyntaxError` — Python не може розібрати код; `try` навколо вже нерозібраного файлу не допоможе.
- `IndentationError` — неправильна структура відступів; підклас `SyntaxError`.
- `NameError` — ім'я не визначено.
- `AttributeError` — об'єкт не має атрибута або методу.
- `TypeError` — операція чи виклик отримали несумісний тип або кількість аргументів.
- `ValueError` — тип придатний, але конкретне значення ні.
- `IndexError` — індекс послідовності за межами.
- `KeyError` — відсутній ключ словника.
- `FileNotFoundError` — файл не знайдено.
- `PermissionError` — системні права не дозволяють операцію.
- `ZeroDivisionError` — ділення на нуль.
- `ImportError/ModuleNotFoundError` — проблема імпорту.

Тип не є готовою відповіддю. Читай повідомлення й контекст виклику.

Перевірити наперед чи спробувати

Перевірити перед операцією

```
python

if path.exists():
    text = path.read_text(encoding="utf-8")
```

Спробувати операцію

```
python

try:
    text = path.read_text(encoding="utf-8")
except FileNotFoundError:
    ...
```

Перевірка `exists()` не гарантує успіх: між перевіркою й читанням файл може зникнути або бути недоступним. Коли сама операція є остаточною перевіркою, стиль «спробуй і оброби конкретний виняток» надійніший. Його часто називають EAFP: *easier to ask forgiveness than permission*.

Перевірки все одно потрібні для правил предметної області: `if amount <= 0` ясніше за штучне спричинення арифметичного винятку.

Виняток або значення результату

```
python

def find_robot(robot_id):
    return robots.get(robot_id)
```

Відсутній робот може бути нормальною відповіддю пошуку — `None` природний.

```
python

def require_robot(robot_id):
    try:
        return robots[robot_id]
    except KeyError as error:
        raise UnknownRobotError(robot_id) from error
```

Якщо операція обіцяє **обов'язково** повернути робота, відсутність порушує контракт і виняток природний. Назва `find_` проти `require_` допомагає показати різницю.

Не використовуй `assert` для введення

```
assert_input.py

energy = -1
assert energy >= 0, "Енергія не може бути від'ємною"
```

Результат

```
Traceback (most recent call last):
  File "...assert_input.py", line 2, in <module>
    assert energy >= 0, "Енергія не може бути від'ємною"
    ^^^^^^^^^^^^^
AssertionError: Енергія не може бути від'ємною
```

`assert` призначений для внутрішніх тверджень розробника й може бути вимкнений оптимізованим запуском Python. Для даних користувача або зовнішнього файла застосовуй звичайний `if` і піднімай відповідний виняток.

У тестах `assert` доречний: там він формулює очікувану поведінку, а `pytest` показує різницю.

СКЛАДИ ПОВНИЙ МАРШРУТ TRY / EXCEPT / ELSE

Розташуй блоки так, щоб помилка перетворення мала власне повідомлення, а успішне число друкувалося лише в `else`.

```
python

else:
    print(f"Число: {number}")
except ValueError:
    print("Потрібне ціле число")
try:
    number = int(text)
```

Типова помилка

`try` охоплює занадто багато

```
too_wide_try.py

text = "5"

def calculate(number):
    raise ValueError("Дефект у формулі")

def save(result):
    print(result)

try:
    number = int(text)
    result = calculate(number)
    save(result)
except ValueError:
    print("Неправильне число")
```

Результат

Неправильне число

Введення "5" правильне, але `calculate()` підняв `ValueError` через дефект формули. Широкий блок неправдиво пояснив проблему введенням. Зв'яжи `try` до перетворення, а успішну роботу перенеси в `else`.

Помилку надруковано й втрачено

```
python
```

```
except Exception as error:  
    print(error)
```

Це фрагмент обробника, не самостійна програма. Він друкує лише текст останнього рядка; файл, номер рядка та шлях викликів зникають. Програма продовжує роботу з невідомим станом. Або оброби конкретну очікувану ситуацію, або дай винятку піднятися, або журналюй його з повним контекстом на зовнішній межі.

Повторний raise написано неправильно

Усередині except просто raise повторно піднімає поточний виняток із початковим traceback. raise error може змінити вигляд ланцюжка. Для додавання контексту використовуй raise NewError(...) from error.

ТИПОВА ПОМИЛКА

Не перетворюй кожен програмну помилку на None. Якщо атрибут мав існувати, але не існує, AttributeError допомагає знайти дефект. Обробляй лише очікувані зовнішні умови, для яких маєш коректний наступний крок.

Швидка перевірка

Визнач, яка гілка спрацює для кожного значення:

```
parse_energy.py
```

```
def parse_energy(text):  
    try:  
        value = int(text)  
    except ValueError:  
        return "Потрібне ціле число."  
    else:  
        if not 0 <= value <= 100:  
            return "Енергія поза межами."  
        return value  
  
print(parse_energy("8"))  
print(parse_energy("viciu"))  
print(parse_energy("108"))
```

ПЕРЕВІР СЕБЕ

Чому `except ValueError` зазвичай кращий за голий `except` у перетворенні `int(text)`?

- Бо голий `except` працює лише для файлів.
- Бо `ValueError` автоматично виправляє введення.
- Він обробляє очікуваний неправильний запис числа й не приховує інші дефекти.

Відповідь: Він обробляє очікуваний неправильний запис числа й не приховує інші дефекти. Вузкий тип винятку показує конкретний план відновлення. Інші помилки залишаються видимими для налагодження.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий код і поясни полі `try`, `except`, `else` та `finally`. Потім назви, чому `except Exception` зазвичай гірший за конкретний тип винятку.

Самостійна робота

САМОСТІЙНА РОБОТА

- Розшир `EnergyReader` верхньою межею 100 і перевір значення -1, 0, 100, 101.
- Створи `SettingsReader`, який відрізняє відсутній файл, неправильне число та невідомий ключ. На зовнішній межі дай кожній причині окреме повідомлення.
- Перероби великий `try`, який охоплює читання, обчислення й показ, на маленький `try` з `else`. Навмисний дефект у обчисленні не повинен маскуватися як помилка файла.
- Створи власний `UnknownRobotError` і два методи: `find()` повертає `None`, `require()` піднімає виняток. Поясни контракти назв.
- Продемонструй `finally` на ресурсі або тимчасовому стані. Потім перевір, чи не може контекстний менеджер `with` виразити те саме ясніше.
- Підними новий виняток `from` першопричини й прочитай обидві частини `traceback`. Порівняй із повторним голим `raise`.
- Знайди голий `except` у навчальному прикладі або створи його навмисно. Покажи, який дефект він приховує, і заміни точним типом.
- Спробуй використати `assert` для перевірки введення, а потім заміни його звичайною умовою та `raise ValueError`. Поясни, чому другий контракт надійніший.

КОРОТКА ІСТОРИЧНА ПАУЗА

Виняток «піднімається» вгору стеком викликів, доки не знайде сумісний ехсерт. Це дозволяє низькому рівню повідомити точну проблему, а вищому — вирішити, чи повторити дію, показати повідомлення або завершити операцію.

Підсумок

- Виняток повідомляє, що операція не може нормально завершитися.
- `try` має охоплювати невелику очікувано ризикову операцію, ехсерт — конкретний тип.
- `else` виконує успішний шлях, `finally` — завершальну дію незалежно від результату.
- `raise` формує контракт відмови; власний клас винятку називає проблему предметної області.
- `raise NewError(...) from error` додає контекст і зберігає першопричину.
- Неочікуваний дефект краще лишити видимим, ніж перетворити на мовчазне `None`.
- Значення результату доречне для нормальної відсутності, виняток — для порушеної обіцянки операції.
- `assert` не замінює перевірку введення або зовнішніх даних.

Офіційний орієнтир: [помилки й винятки](#).

РОЗДІЛ 15

JSON: збереження стану й рефакторинг

Текстовий журнал зручний для людини, але програма має окремо вгадувати, де ім'я, число чи список. JSON зберігає просту структуру даних: словники, списки, рядки, числа, логічні значення та `null`.

Що зробимо

Збережемо профіль гравця у JSON через окремий `ProfileStore`. Сам клас `PlayerProfile` відповідатиме за правила стану, сховище — за файл, а точка запуску — за сценарій.

РЕЗУЛЬТАТ

Ти побачиш межу між живим об'єктом і його серіалізованими даними. Завантаження не повинно перетворювати весь код на вкладені словники.

ЗАРАЗ У ФОКУСІ

Одна дія: простеж дві межі перетворення: об'єкт Python → прості дані → JSON-текст і назад.

Якщо відволікся, визнач, де в прикладі живе словник, а де — лише текст із JSON-синтаксисом.

JSON має прості типи

Вигляд файла `profile.json`:

```
json

{
  "version": 1,
  "name": "Mira",
  "level": 3,
  "is_active": true,
  "skills": [
    "навігація",
    "ремонт"
  ]
}
```

Відмінності від запису Python:

- ключі JSON — рядки в подвійних лапках;
- логічні значення — `true` і `false` з малої літери;
- відсутнє значення — `null`, у Python воно стає `None`;

- коментарі й завершальні коми стандартний JSON не дозволяє.

JSON не зберігає методи класу. Він зберігає лише підтримувані дані, з яких програма може відновити новий об'єкт.

`dumps()` і `loads()` працюють із рядком

СПОЧАТКУ ПЕРЕДБАЧ

До запуску визнач типи `data`, `text` і `restored`, а також чи буде `restored` тим самим об'єктом або лише рівною за даними новою структурою.

```
json_string.py

import json

data = {
    "name": "Міра",
    "skills": ["навігація", "ремонт"],
}

text = json.dumps(data, ensure_ascii=False, indent=2)
restored = json.loads(text)

print(restored["name"])
print(restored == data)
```

ДАНІ ПРОХОДЯТЬ ЧЕРЕЗ JSON-ТЕКСТ І ПОВЕРТАЮТЬСЯ

ДО
`data` — словник Python з рядком та списком.

→

ПІСЛЯ
`data` і `restored` — два рівні за вмістом словники; `text` лишається рядком.

1	Серіалізація	<code>json.dumps(data, ...)</code>	Структура перетворюється на рядок <code>text</code> у JSON-форматі.
2	Межа формату	<code>type(text) is str</code>	У тексті немає методів словника або об'єкта класу.
3	Десеріалізація	<code>json.loads(text)</code>	JSON-рядок перетворюється на новий словник <code>restored</code> .
4	Читання	<code>restored["name"]</code>	За ключем отримуємо "Міра".
5	Порівняння	<code>restored == data</code>	Окремі словники мають рівний вміст, тому результат — <code>True</code> .

Що тут важливо: JSON зберігає прості дані, а поведінку класу треба відновити окремим конструктором на кшталт `from_data()`.

- `json.dumps(data)` повертає JSON-рядок; літера `s` нагадує *string*;

- `json.loads(text)` читає JSON-рядок і повертає Python-дані;
- `ensure_ascii=False` залишає українські літери читабельними;
- `indent=2` форматує файл для людини.

Без `ensure_ascii=False` дані не зіпсуються, але символи можуть виглядати як `\u041c...`

`dump()` і `load()` працюють із файловим об'єктом

json_file.py

```
import json
from pathlib import Path

path = Path("robot.json")
data = {"name": "Робі", "energy": 18}

with path.open("w", encoding="utf-8") as file:
    json.dump(data, file, ensure_ascii=False, indent=2)

with path.open("r", encoding="utf-8") as file:
    restored = json.load(file)

print(restored)
```

`dump()` пише у відкритий файл, `load()` читає з нього. У маленькому коді також можна поєднати `Path.write_text(json.dumps(...))`.

Запис у режимі "w" повністю замінює попередній файл. Це має бути усвідомленою частиною методу `save()`.

Майстерня

Створи `profile_storage.py`:

`profile_storage.py`

```
import json
from pathlib import Path

# 1. Зрозуміла помилка на межі даних
class ProfileDataError(ValueError):
    """JSON не відповідає контракту профілю."""

# 2. Живий об'єкт і правила його стану
class PlayerProfile:
    def __init__(self, name, level=1, skills=None):
        if not name.strip():
            raise ValueError("Ім'я не може бути порожнім")
        if level < 1:
            raise ValueError("Рівень має бути не меншим за 1")

        self.name = name.strip()
        self.level = level
        self.skills = list(skills) if skills is not None else []

    def level_up(self):
        self.level += 1
```

`profile_storage.py` · продовження

```
def learn(self, skill):
    if skill not in self.skills:
        self.skills.append(skill)

def to_data(self):
    return {
        "version": 1,
        "name": self.name,
        "level": self.level,
        "skills": self.skills.copy(),
    }

@classmethod
def from_data(cls, data):
    if not isinstance(data, dict):
        raise ProfileDataError("корінь має бути об'єктом JSON")
    if data.get("version") != 1:
        raise ProfileDataError("непідтримувана версія")

    try:
        return cls(
```

profile_storage.py · продовження

```

        name=data["name"],
        level=data["level"],
        skills=data.get("skills", []),
    )
except (KeyError, TypeError, ValueError) as error:
    raise ProfileDataError("неправильні поля профілю") from error

def summary(self):
    skills_text = ", ".join(self.skills) if self.skills else "немає"
    return f"{self.name}: рівень {self.level}; навички: {skills_text}."

# 3. Сховище знає файл і JSON
class ProfileStore:
    def __init__(self, path):
        self.path = Path(path)

    def save(self, profile):
        text = json.dumps(
            profile.to_data(),
            ensure_ascii=False,
            indent=2,
        )

```

profile_storage.py · продовження

```

        self.path.write_text(text + "\n", encoding="utf-8")

    def load(self):
        try:
            text = self.path.read_text(encoding="utf-8")
            data = json.loads(text)
        except FileNotFoundError:
            return None
        except json.JSONDecodeError as error:
            raise ProfileDataError("пошкоджений JSON") from error

        return PlayerProfile.from_data(data)

# 4. Сценарій з'єднує модель і сховище
store = ProfileStore("profile.json")

profile = PlayerProfile("Міпа", level=3)
profile.learn("навігація")
profile.learn("ремонт")
profile.level_up()

store.save(profile)

```

profile_storage.py · продовження

```

restored_profile = store.load()

print(restored_profile.summary())

```

to_data() створює просту форму

python

```
def to_data(self):
    return {
        "version": 1,
        "name": self.name,
        "level": self.level,
        "skills": self.skills.copy(),
    }
```

Метод не пише файл і не імпортує json. Він лише перетворює стан на підтримувані прості типи. Це дозволяє використати ті самі дані для файла, мережі або перевірки.

Назви to_dict()/from_dict() теж поширені. to_data() підкреслює, що повернена структура є зовнішнім представленням, а не весь внутрішній світ об'єкта.

from_data() відновлює об'єкт через конструктор

python

```
return cls(
    name=data["name"],
    level=data["level"],
    skills=data.get("skills", []),
)
```

Ми не створюємо порожній об'єкт і не вставляємо атрибути навімання. Звичайний __init__ знову перевіряє правила.

@classmethod використовує cls, тому альтернативний конструктор лишається сумісним із похідними класами.

Сховище не керує правилами профілю

ProfileStore знає Path, UTF-8 і JSON. Він просить профіль to_data() та передає прочитані дані PlayerProfile.from_data().

Завдяки цьому:

- модель можна перевірити без файла;
- формат збереження можна змінити окремо;
- інтерфейс не працює з сирими словниками;
- кожна помилка має зрозумілий рівень.

ЗАВАНТАЖ АБО СТВОРИ

Напиши функцію сценарію:

```
python

def load_or_create(store, default_name):
    profile = store.load()
    if profile is not None:
        return profile
    return PlayerProfile(default_name)
```

Відсутній файл тут є нормальним першим запуском, тому `load()` повертає `None`. Пошкоджений наявний JSON — інша ситуація, її не можна мовчки замінити новим профілем.

Перевірка типів після JSON

Синтаксично правильний JSON не обов'язково відповідає моделі:

```
json

{
  "version": 1,
  "name": 123,
  "level": "високий"
}
```

`json.loads()` успішно створить словник. Перевірка структури — відповідальність `from_data()` або окремого валідатора.

`isinstance()` доречний на межі зовнішніх даних:

```
python

if not isinstance(data.get("name"), str):
    raise ProfileDataError("name має бути рядком")
```

Не розкидай такі перевірки по всій програмі. Перетвори зовнішню структуру на надійний об'єкт один раз біля входу.

Які Python-значення змінюються в JSON

Стандартний JSON підтримує:

- `dict` із рядковими ключами → `object`;
- `list` і `tuple` → `array`, після читання це буде `list`;
- `str` → `string`;
- `int/float` → `number`;

- True/False → true/false;
- None → null.

Кортеж після кругової подорожі стає списком:

```
json_tuple.py

import json

text = json.dumps({"position": (3, 5)})
restored = json.loads(text)

print(type(restored["position"]).__name__)
print(restored["position"])
```

Об'єкт Path, date, set, Enum або власний клас стандартний encoder напямую не знає. Перетвори їх у просту явну форму: рядок, список, значення переліку, словник.

Ключі стають рядками

JSON object має рядкові ключі. Словник {1: "one"} після запису й читання повернеться як {"1": "one"}. Якщо тип ключа важливий, обери список записів або явне перетворення.

Версія формату й зміна структури

Поле:

```
python

"version": 1
```

допомагає відрізнити формати. Коли версія 2 перейменує level на rank, старі файли не повинні дивно ламатися в далекому методі.

Стратегії:

- підтримати читання кількох версій;
- написати явну міграцію v1 -> v2;
- відмовити з повідомленням, що версія не підтримується;
- для необов'язкового нового поля використати безпечне значення за замовчуванням.

Не змінюй значення старого поля мовчки, якщо це може втратити дані.

```
python
```

```
def migrate_v1_to_v2(data):
    return {
        "version": 2,
        "name": data["name"],
        "rank": data["level"],
        "skills": data.get("skills", []),
    }
```

Міграція є чистою функцією: отримує старі дані, повертає нові, не перезаписує файл самостійно.

Обережний запис через тимчасовий файл

Звичайний `write_text()` може лишити неповний файл, якщо процес аварійно завершиться посеред запису. Для важливого локального стану спочатку пишуть сусідній тимчасовий файл, а потім замінюють ціль:

```
python
```

```
def save(self, profile):
    text = json.dumps(profile.to_data(), ensure_ascii=False, indent=2) + "\n"
    temporary_path = self.path.with_suffix(self.path.suffix + ".tmp")
    temporary_path.write_text(text, encoding="utf-8")
    temporary_path.replace(self.path)
```

`replace()` замінює наявний цільовий файл. Це **руйнівна дія щодо попередньої версії**, тому шлях має бути точно контрольований, а для цінних даних потрібна резервна копія або історія версій.

Гарантії атомарності залежать від файлової системи й того, чи обидва шляхи на одному томі. Для початкового локального застосунку цей шаблон зменшує ризик часткового запису, але не замінює продумане резервування.

Рефакторинг: змінюємо будову, зберігаємо поведінку

Початкова програма часто виглядає так:

```
python
```

```
data = json.loads(Path("profile.json").read_text())
data["level"] += 1
Path("profile.json").write_text(json.dumps(data))
print(data["name"], data["level"])
```

Це синтаксично робочий код, а не виняток. Проблема архітектурна: читання, зміна моделі, запис і показ результату змішані. Рефакторинг розкладає їх:

```
text
```

```
main -> ProfileStore.load() -> PlayerProfile.from_data()  
      -> profile.level_up()  
      -> ProfileStore.save(profile)  
      -> profile.summary()
```

Безпечний порядок:

- 1 Зафіксуй поточний очікуваний результат маленькою перевіркою.
- 2 Виділи одну відповідальність без зміни зовнішнього результату.
- 3 Запусти перевірку.
- 4 Лише потім роби наступний крок.

Не поєднуй рефакторинг і нову функцію у величезній зміні, якщо можна розділити. Інакше важко зрозуміти причину регресії.

JSON не є безпечним лише тому, що текстовий

`json.load()` не виконує код, що робить JSON безпечнішим за формати на кшталт `pickle` для неперевіреного джерела. Але дані все одно можуть бути:

- надто великими;
- неправильної структури;
- з небезпечними шляхами або командами як звичайним текстом;
- навмисно створеними для перевантаження логіки.

Вміст JSON — дані, не інструкції. Перевіряй межі й ніколи не виконуй рядок із файла через `eval()` або оболонку.

`pickle` може створювати довільні Python-об'єкти й **не підходить для недовірених файлів**, бо завантаження може виконати код.

ВІДНОВИ МЕЖУ МІЖ ОБ'ЄКТОМ І ДАНИМИ

Заповни пропуски так, щоб один метод створював словник, а інший відновлював об'єкт через конструктор.

```
python

class Hero:
    def to_data(self):
        return {"name": self.name, "health": self.health}

    @classmethod
    def from_data(cls, data):
        return cls(name=data[____], health=data[____])
```

Типова помилка

Спроба записати об'єкт напряму

```
json_object.py

import json

class PlayerProfile:
    pass

profile = PlayerProfile()
json.dumps(profile)
```

Результат

```
Traceback (most recent call last):
  File "...json_object.py", line 9, in <module>
    json.dumps(profile)
    ...
TypeError: Object of type PlayerProfile is not JSON serializable
```

Останній рядок називає тип, для якого JSON не знає формату. Перетвори об'єкт через `profile.to_data()`.

Відсутній файл і пошкоджений файл обробляються однаково

Якщо `load()` повертає `None` і для `FileNotFoundError`, і для `JSONDecodeError`, програма може затерти пошкоджені дані новим порожнім профілем. Перший запуск — нормальний, пошкодження — проблема, яку треба показати або відновити з копії.

Зовнішній словник стає внутрішнім станом без копії

shared_json_list.py

```
class Profile:
    def __init__(self, data):
        self.skills = data["skills"]

data = {"skills": ["python"]}
profile = Profile(data)
data["skills"].append("git")
print(profile.skills)
```

Результат

```
['python', 'git']
```

profile ніхто не змінював напряму, але його стан уже містить git: список спільний із зовнішнім словником. Конструктор має зробити `list(data["skills"])` і встановити власну колекцію.

ТИПОВА ПОМИЛКА

Не використовуй `default=str` лише для того, щоб «усе якось записалось». Він мовчки перетворює невідомі об'єкти на рядки, з яких неможливо надійно відновити тип. Визнач явний формат для кожної важливої сутності.

Швидка перевірка

Перевір кругову подорож «об'єкт → дані → JSON → дані → новий об'єкт»:

json_round_trip.py

```
import json

class Profile:
    def __init__(self, name, level, skills):
        self.name = name
        self.level = level
        self.skills = list(skills)

    def to_data(self):
        return {
            "name": self.name,
            "level": self.level,
            "skills": self.skills.copy(),
        }

    @classmethod
    def from_data(cls, data):
        return cls(data["name"], data["level"], data["skills"])

original = Profile("Лея", 2, ["python"])
```

json_round_trip.py · продовження

```
text = json.dumps(original.to_data(), ensure_ascii=False)
restored = Profile.from_data(json.loads(text))

print(f'{restored.name}: {restored.level}; {restored.skills}')
```

ПЕРЕВІР СЕБЕ

Чому ProfileStore не повинен сам змінювати level профілю?

- Бо методи класу не можуть імпортувати json.
- Його відповідальність — зберігати й читати дані, а правила рівня належать PlayerProfile.
- Бо JSON не підтримує числа.

Відповідь: Його відповідальність — зберігати й читати дані, а правила рівня належать PlayerProfile. Розділення відповідальності дозволяє перевіряти модель без файла й змінювати формат сховища без перенесення правил профілю.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Без підглядання назви чотири етапи: об'єкт, прості дані, JSON-текст, відновлені дані. Поясни, на якому етапі повертається поведінка класу.

Самостійна робота

САМОСТІЙНА РОБОТА

- Додай профілю поле `settings` зі словником і забезпеч копію під час створення, `to_data()` та `from_data()`.
- Створи окремі перевірки для відсутнього файлу, пошкодженого JSON, відсутнього ключа й неправильного типу поля. Не повертай для всіх випадків однакове `None`.
- Реалізуй `Robot.to_data()/from_data()` і `RobotStore`. Методи руху й заряду мають лишитися в `Robot`, не у сховищі.
- Продемонструй, як кортеж і ключ-число змінюються після JSON round trip. Обери явне представлення, якщо початковий тип треба відновити.
- Додай `version: 1`, створи приклад старих даних і чисту функцію міграції у версію 2 без запису файла.
- Реалізуй запис через сусідній `.tmp` і `replace()` лише в окремій навчальній папці. Перед запуском надрукуй обидва абсолютні шляхи й переконайся, що вони в межах проекту.
- Візьми монолітний скрипт зі словником і JSON та розділи на модель, сховище і `main()`. Після кожного кроку перевіряй той самий видимий результат.
- Спробуй серіалізувати `Path`, `set` або `Enum`, прочитай `TypeError`, а потім визнач явну просту форму й зворотне відновлення.

КОРОТКА ІСТОРИЧНА ПАУЗА

JSON походить із синтаксису об'єктів JavaScript, але давно є незалежним форматом обміну. Його сила — невеликий спільний набір типів. Обмеження змушує явно вирішити, які дані справді потрібні для відновлення об'єкта.

Підсумок

- JSON зберігає прості структури, а не методи й довільні Python-об'єкти.
- `dumps()/loads()` працюють із рядками, `dump()/load()` — із файловими об'єктами.
- `ensure_ascii=False`, `indent` і UTF-8 роблять український файл читабельним.
- `to_data()` створює серіалізовану форму, `from_data()` перевіряє її й відновлює об'єкт.
- Сховище відповідає за шлях та формат; модель — за правила стану й поведінку.
- Відсутній файл, пошкоджений JSON і неправильна схема — різні ситуації.
- Версія формату та явні міграції допомагають безпечно змінювати структуру.
- Тимчасовий запис із `replace()` зменшує ризик часткового файла, але є заміною даних і потребує точного шляху.
- Рефакторинг змінює будову маленькими кроками, зберігаючи перевірену поведінку.

Офіційний орієнтир: [json](#).

РОЗДІЛ 16

Автоматичні тести з pytest

Ручний запуск показує, що один сценарій щойно спрацював. Автоматичний тест зберігає очікування у коді й повторює його після кожної зміни. Тест не доводить відсутність усіх помилок, але захищає конкретну важливу поведінку.

Що зробимо

Перевіримо клас `Robot`: початковий стан, успішний рух, відмову без пошкодження стану, кілька неправильних аргументів і JSON-сховище в тимчасовій папці.

РЕЗУЛЬТАТ

Ти зможеш перетворити вимогу на тест у формі «підготуй → виконай → перевір», прочитати звіт падіння й виправити причину, а не сам тест навмання.

ЗАРАЗ У ФОКУСІ

Одна дія: перед тестом сформулюй очікувану поведінку, виконай одну дію й перевір видимий стан, а не внутрішні дрібниці реалізації.

Якщо відволікся, повернися до трійки «підготуй → виконай → перевір».

pytest — стабільний сторонній пакет

pytest не входить до стандартної бібліотеки, тому його встановлюють у активне віртуальне середовище:

ВСТАНОВЛЕННЯ Й ЗАПУСК

Після активації .venv у Windows, macOS або Linux

```
powershell

python -m pip install pytest
python -m pytest
```

У активованому середовищі команда `python` вказує на локальний інтерпретатор проєкту в усіх системах.

Без активації у Windows PowerShell

```
powershell

.\.venv\Scripts\python.exe -m pytest
```

Без активації у macOS або Linux

```
bash

.venv/bin/python -m pytest
```

Форма `python -m pytest` гарантує, що `pytest` запускає той самий інтерпретатор, у якому пакет встановлено.

У цьому репозиторії точна перевірена залежність зафіксована в `requirements-dev.txt`. Для відтворення середовища:

```
powershell

python -m pip install -r requirements-dev.txt
```

Не встановлюй випадкові плагіни до `pytest`, доки немає конкретної потреби. Базового пакета достатньо для всього розділу.

Як pytest знаходить тести

Типова структура:

```
text

project/
├─ robot.py
├─ storage.py
└─ tests/
    ├─ test_robot.py
    └─ test_storage.py
```

За замовчуванням `pytest` знаходить файли на кшталт `test_*.py` і функції `test_*`. Імена мають описувати поведінку:

```
python

def test_move_changes_energy_and_distance():
    ...
```

Назва `test_1()` не допомагає зрозуміти, що зламалося.

Запускай із кореня проєкту, де імпорти моделі працюють так само, як у застосунку:

```
powershell  
  
python -m pytest
```

Корисні варіанти:

```
powershell  
  
python -m pytest -q  
python -m pytest -v  
python -m pytest tests/test_robot.py  
python -m pytest tests/test_robot.py::test_move_changes_energy_and_distance  
python -m pytest -k move  
python -m pytest -x
```

- `-q` — короткий звіт;
- `-v` — докладні назви тестів;
- шлях або node id запускає точну частину;
- `-k move` фільтрує за виразом у назві;
- `-x` зупиняється після першого падіння.

Перший тест: звичайний assert

Модель `robot.py`:

```
robot.py  
  
class Robot:  
    def __init__(self, name, energy=10):  
        self.name = name  
        self.energy = energy  
        self.distance = 0  
  
    def move(self, steps):  
        self.energy -= steps  
        self.distance += steps  
        return self.distance
```

Тест `test_robot.py`:

```
test_robot.py

from robot import Robot

def test_new_robot_has_expected_state():
    robot = Robot("Робі", energy=10)

    assert robot.name == "Робі"
    assert robot.energy == 10
    assert robot.distance == 0
```

pytest перехоплює стандартний `assert` і за падіння показує значення обох сторін. Не треба вчити окремі методи на кшталт `assertEqual()`.

Підготуй → виконай → перевір

Поширена структура тесту:

```
python

def test_move_changes_energy_and_distance():
    # Підготовка
    robot = Robot("Робі", energy=10)

    # Дія
    result = robot.move(3)

    # Перевірка
    assert result == 3
    assert robot.energy == 7
    assert robot.distance == 3
```

Англійською її називають Arrange–Act–Assert, або AAA. Коментарі не обов'язкові, якщо порожні рядки й назви вже ясно відділяють три частини.

Один тест може мати кілька `assert`, коли всі вони описують один видимий результат однієї дії. Не збирай в одному тесті десять незалежних сценаріїв.

Майстерня

У репозиторії є повністю виконуваний приклад:

```
text

examples/pytest_demo/
├─ robot.py
├─ storage.py
├─ test_robot.py
└─ test_storage.py
```

Його тестовий набір запускається з цієї папки:

```
powershell  
  
python -m pytest -q
```

Очікуваний підсумок для зафіксованої версії прикладу:

```
text  
  
7 passed
```

Один параметризований тест запускається тричі, тому фізичних функцій менше, ніж зібраних тестових випадків.

Щоб сам розділ також мав незалежну швидку перевірку без стороннього runner, звичайні твердження Python працюють так:

СПОЧАТКУ ПЕРЕДБАЧ

У робота 10 енергії та 0 пройденої відстані. Перед запуском визнач обидва значення після `move(3)` і виріши, чи побачиш повідомлення, якщо всі `assert` успішні.

```
plain_assert_check.py  
  
class Robot:  
    def __init__(self, energy):  
        self.energy = energy  
        self.distance = 0  
  
    def move(self, steps):  
        self.energy -= steps  
        self.distance += steps  
  
robot = Robot(10)  
robot.move(3)  
  
assert robot.energy == 7  
assert robot.distance == 3  
print("Базові твердження виконано.")
```

ОДИН ТЕСТ ЯК КОНТРОЛЬОВАНА ЗМІНА СТАНУ

ДО
`robot.energy = 10, robot.distance = 0.`

→

ПІСЛЯ
`robot.energy = 7, robot.distance = 3; обидві перевірки пройдено.`

1	Підготовка	<code>robot = Robot(10)</code>	Створюється відомий початковий стан.
2	Дія	<code>robot.move(3)</code>	Метод віднімає 3 від енергії та додає 3 до відстані.
3	Перша перевірка	<code>assert robot.energy == 7</code>	Якщо енергія інша, виконання зупиниться з <code>AssertionError</code> .
4	Друга перевірка	<code>assert robot.distance == 3</code>	Перевіряється друга видима частина поведінки.
5	Сигнал успіху	<code>print("Базові твердження виконано.")</code>	Рядок виконується лише після двох успішних тверджень.

Що тут важливо: Тест фіксує початковий стан, одну дію та спостережуваний результат; він не повинен залежати від порядку запуску інших тестів.

Для повного набору все одно використовуй `pytest`: він знаходить тести, ізолює контекст, показує діагностику й надає fixtures.

Спочатку побач падіння

Якщо новий тест одразу зелений, перевір, чи він справді здатен знайти дефект.

Тимчасово напиши неправильне очікування:

```
python
```

```
assert robot.energy == 8
```

`pytest` покаже приблизно:

```
text
```

```
E      assert 7 == 8
E      +   where 7 = robot.energy
```

Потім поверни правильне очікування. Цей короткий червоний крок доводить, що тест виконує потрібну гілку.

Коли справжній тест падає:

- 1 Прочитай назву тесту.
- 2 Знайди перший рядок із `E` та значення.

- 3 Визнач: неправильний код, неправильне очікування чи неправильна підготовка?
- 4 Виправ одну причину.
- 5 Запусти спочатку точний тест, потім весь набір.

Не змінюй очікування лише для того, щоб отримати зелений колір. Воно має походити з вимоги.

Перевірка винятку через `pytest.raises`

Модель відмовляє за надмірної відстані:

```
robot.py

class EnergyError(ValueError):
    pass

def move(self, steps):
    if steps > self.energy:
        raise EnergyError("Не вистачає енергії")
    self.energy -= steps
    self.distance += steps
```

Тест:

```
test_robot.py

import pytest

from robot import EnergyError, Robot

def test_failed_move_preserves_state():
    robot = Robot("Робі", energy=10)

    with pytest.raises(EnergyError, match="Не вистачає"):
        robot.move(11)

    assert robot.energy == 10
    assert robot.distance == 0
```

Контекстний менеджер перевіряє, що блок справді підняв `EnergyError`. `match` перевіряє повідомлення як регулярний вираз.

Два останні твердження особливо важливі: операція не лише відмовила, а й **не пошкодила стан**.

Якщо виняток не виникне або тип буде інший, тест впаде.

Fixture готує надійний контекст

Однаковий новий робот потрібен багатьом тестам:

```
test_robot.py

import pytest

from robot import Robot

@pytest.fixture
def robot():
    return Robot("Робі", energy=10)

def test_new_robot_has_expected_state(robot):
    assert robot.energy == 10

def test_move_changes_energy(robot):
    robot.move(3)
    assert robot.energy == 7
```

pytest бачить параметр `robot`, знаходить fixture з такою назвою й викликає її перед тестом.

За замовчуванням кожен тест отримує **новий** результат fixture. Зміна робота в одному тесті не протікає в інший. Це одна з причин не використовувати спільний глобальний об'єкт.

Fixture має готувати контекст, а не приховувати саму дію, яку тест перевіряє. Якщо тест називається `test_move...`, виклик `move()` краще лишити видимим у тесті.

Спільні fixtures для кількох файлів кладуть у `conftest.py`; його не імпортують вручну — pytest знаходить його за деревом папок.

Параметризація перевіряє таблицю випадків

```
test_robot.py

import pytest

@pytest.mark.parametrize("steps", [0, -1, -20])
def test_move_rejects_nonpositive_steps(robot, steps):
    with pytest.raises(ValueError, match="додатною"):
        robot.move(steps)
```

Одна функція запускається тричі. У звіті кожен набір має окремий test id.

Коли результат теж змінюється:

```
python

@pytest.mark.parametrize(
    "start, steps, expected_energy",
    [
        (10, 1, 9),
        (10, 10, 0),
        (3, 2, 1),
    ],
)
def test_move_uses_energy(start, steps, expected_energy):
    robot = Robot("Робі", energy=start)
    robot.move(steps)
    assert robot.energy == expected_energy
```

Не мутуй список або словник із параметрів, якщо очікуєш чисте значення для наступного запуску: pytest передає параметри як є, без автоматичного копіювання.

ПЕРЕВІР МЕЖІ

Додай випадки для `steps = 1`, `steps = energy`, `steps = energy + 1`. Це значення на межі, безпосередньо до неї та після неї.

tmp_path дає окрему тимчасову папку

Тест сховища не повинен писати `profile.json` у робочу папку або торкатися справжніх даних.

```
test_storage.py

def test_store_round_trip(tmp_path):
    path = tmp_path / "robot.json"
    store = RobotStore(path)
    robot = Robot("Іскра", energy=8)
    robot.move(3)

    store.save(robot)
    restored = store.load()

    assert restored.to_data() == robot.to_data()
    assert path.read_text(encoding="utf-8").endswith("\n")
```

`tmp_path` — вбудована fixture pytest, яка повертає окремий `pathlib.Path` для тесту. Не підставляй `--basetemp` на важливу папку: pytest може очистити вказану базову директорію перед запуском.

Перевірка round trip підтверджує: збережений і відновлений об'єкти мають однакові дані. Окреме твердження про фінальний `\n` фіксує формат файла.

Перевірка виведення через capsys

Краще повертати значення, але іноді треба перевірити справжній CLI-вивід:

```
python

def greet(name):
    print(f"Привіт, {name}!")

def test_greet_prints_message(capsys):
    greet("Mipa")

    captured = capsys.readouterr()
    assert captured.out == "Привіт, Mipa!\n"
    assert captured.err == ""
```

`capsys` перехоплює стандартні потоки виведення. Не тестуй кожен пробіл інтерфейсу, якщо формат не є частиною контракту.

Що саме тестувати в об'єкті

Пріоритет:

- публічний результат методу;
- важлива зміна стану;
- відмова й незмінність стану;
- межові значення;
- перетворення `to_data()/from_data()`;
- взаємодія складених об'єктів через публічний інтерфейс.

Не прив'язуй тест до внутрішнього `_charge`, якщо контракт дає `battery.charge`. Інакше безпечний рефакторинг внутрішньої реалізації зламає тести без зміни поведінки.

Один тест має бути:

- повторюваним;
- незалежним від порядку інших тестів;
- швидким для свого рівня;
- зрозумілим за назвою;
- здатним упасти з однієї чіткої причини.

Набір тестів і регресія

Регресія — поведінка, яка працювала, але зламалася після зміни. Коли знаходиш дефект:

- 1 Напиши тест, який відтворює його й падає.
- 2 Виправ мінімальну причину.
- 3 Побач зелений новий тест.
- 4 Запусти весь набір.

Тест лишається як сигналізація проти повторення того самого дефекту.

Тести не замінюють перевірку інтерфейсу людиною, аналіз безпеки або розуміння вимог. Вони є виконуваною частиною доказу.

Конфігурація в `pyproject.toml`

Коли проєкту потрібні сталі опції:

```
toml

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-ra"
```

- `testpaths` обмежує звичний пошук;
- `-ra` додає короткий підсумок неочевидних результатів.

Не приховуй попередження або падіння глобальними опціями лише заради чистого екрана. Кожне приглушення має конкретну причину.

Стандартний `unittest`

Python має вбудований модуль `unittest`, який не потребує встановлення. `pytest` може запускати багато наявних `unittest`-тестів і дає коротший синтаксис для нових початкових перевірок.

Вибір залежить від проєкту. Ця довідка використовує `pytest` через простий `assert`, `fixtures` і читабельну параметризацію. Не переписуй робочий набір іншого проєкту лише заради назви інструмента.

СКЛАДИ ПЕРЕВІРКУ ЗА СХЕМОЮ AAA

Розташуй рядки як Arrange — підготуй, Act — виконай, Assert — перевір.

```
python

assert robot.energy == 7
robot.move(3)
robot = Robot(10)
```

Після цього додай другу незалежну перевірку для `distance`.

Типова помилка

Тести залежать один від одного

```
test_shared_robot.py

robot = Robot("Робі", 10)

def test_move():
    robot.move(3)
    assert robot.energy == 7

def test_initial_energy():
    assert robot.energy == 10
```

За порядку, показаного у файлі, `pytest` повідомить:

```
Результат

.F
FAILED test_shared_robot.py::test_initial_energy - assert 7 == 10
1 failed, 1 passed
```

Перший тест змінив спільний об'єкт, тому другий отримав енергію 7. Створюй нового робота в кожному тесті або `function-scoped fixture`.

Перевірено реалізацію, а не поведінку

Тест вимагає, щоб метод використав рівно один цикл або конкретний приватний список. Рефакторинг ламає тест, хоча результат той самий. Перевір публічний результат і значущий стан.

Виняток очікується надто широко

```
python

with pytest.raises(Exception):
    robot.move(-1)
```

Якщо всередині `move()` випадково виникне `AttributeError`, звіт усе одно може бути зеленим:

```
Результат

1 passed
```

Це хибно позитивний тест. Очікуй точний `ValueError` або власний `EnergyError` і, за потреби, перевір повідомлення.

Тест містить умову

```
python

if result == 3:
    assert True
```

Навіть для `result = 999` умова просто не виконається, і `pytest` покаже:

```
Результат

1 passed
```

Тест завершився без жодного твердження й помилково вважається успішним. Пиши пряме `assert result == 3`.

ТИПОВА ПОМИЛКА

Не запускай тести на справжній файл користувача, домашню папку або спільну базу. Передавай тимчасову залежність (`tmp_path`, `fake`, тестовий об'єкт) і доводь межі шляху до операції.

Швидка перевірка

Який тест найкраще ловить регресію «невдала атака все одно відняла енергію»?

```
python

def test_failed_attack_preserves_energy():
    hero = Hero(energy=5)

    with pytest.raises(EnergyError):
        hero.attack(cost=8)

    assert hero.energy == 5
```

Тут назва, точний виняток і стан після відмови описують одну вимогу.

ПЕРЕВІР СЕБЕ

Навіщо тесту сховища fixture `tmp_path`?

- Щоб `pytest` автоматично опублікував файл у `Git`.
- Щоб `JSON` перетворився на двійковий формат.
- Щоб кожен тест працював у власній тимчасовій папці й не торкався реальних файлів.

Відповідь: Щоб кожен тест працював у власній тимчасовій папці й не торкався реальних файлів. `tmp_path` дає `pathlib.Path` для ізольованих тестових даних. Шлях передається сховищу як звичайна залежність.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Закрий приклад і розшифруй три `A` в `Arrange–Act–Assert`. Потім поясни, чому тест поведінки кращий за перевірку точного внутрішнього алгоритму.

Самостійна робота

САМОСТІЙНА РОБОТА

- Запусти готовий `examples/pytest_demo` і поясни, чому звіт має 7 випадків, хоча тестових функцій менше.
- Навмисно зміни правильне очікування на неправильне, прочитай `introspection pytest`, потім віднови вимогу й зелений результат.
- Додай тест успішного руху на точну межу енергії та тест відмови на один крок більше. В обох перевір стан.
- Створи fixture нового `PlayerProfile` і два незалежні тести, один з яких змінює навички. Доведи, що зміна не протікає.
- Параметризуй щонайменше чотири неправильні значення для методу. Не мутуй передані параметри між випадками.
- Перевір власний виняток через `pytest.raises(..., match=...)` і переконайся, що стан після відмови не змінився.
- Використай `tmp_path` для round trip `JSON`-сховища. Не пиши тестовий файл у корінь або домашню папку.
- Відтвори знайдений раніше дефект тестом, побач червоний результат, виправ мінімальну причину й запусти весь набір.

КОРОТКА ІСТОРИЧНА ПАУЗА

Автоматичний тест — це програма, яка запускає іншу частину програми й порівнює спостереження з очікуванням. Сильна діагностика `pytest` для звичайного `assert` зробила такі тести короткими: сама мова формулює твердження, `runner` додає пошук, ізоляцію та звіт.

Підсумок

- `pytest` — стабільний сторонній пакет; встановлюй його в `.venv` і запускай через `python -m pytest`.
- Файли й функції `test_*` `pytest` знаходить автоматично.
- Тест організовують як підготовку, одну дію й перевірку результату через `assert`.
- Спочатку побач, що новий тест здатен падати з правильної причини.
- `pytest.raises()` перевіряє точний виняток, `fixture` готує незалежний контекст.
- Параметризація запускає одну перевірку для таблиці випадків.
- `tmp_path` ізолює файлові тести, `capsys` перехоплює консольний вивід.
- Перевірай публічну поведінку та стан, а не випадкові деталі реалізації.
- Регресійний тест спершу відтворює дефект, потім захищає виправлення.
- Зелений набір — важливе свідчення, але не доказ, що поза перевіреними сценаріями помилок немає.

Офіційні орієнтири: [початок роботи з pytest](#), [твердження](#), [fixtures](#), [параметризація](#), [tmp_path](#).

РОЗДІЛ 17

Підтримуваний код: стиль, налагодження й карта синтаксису

Програма може мати правильний синтаксис і все одно давати неправильний результат. А робочий код може бути настільки заплутаним, що наступна зміна стане небезпечною. Завершальний розділ збирає звички, які допомагають читати, діагностувати й підтримувати початковий OOP-код.

Що зробимо

Розберемо логічний дефект у класі `Wallet`, знайдемо його через маленьке відтворення, `traceback`, спостереження стану й `breakpoint` у VS Code. Потім зафіксуємо компактну карту всієї мови, пройденої в довідці.

РЕЗУЛЬТАТ

Після повідомлення «не працює» ти матимеш маршрут: відтворити → класифікувати → звузити → перевірити припущення → виправити одну причину → запустити регресійний тест.

ЗАРАЗ У ФОКУСІ

Одна дія: зафіксуй очікування й фактичний стан до першого редагування, потім змінюй лише одну ймовірну причину.

Якщо відволікся, повернися до пари «очікували 70 — отримали 130».

Три великі види проблем

1. Синтаксична помилка

```
missing_colon.py
```

```
energy = 5

if energy > 0
    print("Pyx")
```

Результат

```
File "...\\missing_colon.py", line 3
    if energy > 0
        ^
SyntaxError: expected ':'
```

Python не може побудувати програму через відсутню двокрапку. Каретка стоїть у кінці умови, а останній рядок прямо називає очікуваний символ. Жоден рядок файлу не почне нормально виконуватися, доки синтаксис не виправлено.

2. Виняток під час виконання

```
empty_list.py
```

```
items = []  
print(items[0])
```

Результат

```
Traceback (most recent call last):  
  File "...\\empty_list.py", line 2, in <module>  
    print(items[0])  
          ~~~~~^^^  
IndexError: list index out of range
```

Код синтаксично правильний, але порожній список не має елемента з індексом 0. Traceback показує файл, вираз і тип винятку.

3. Логічний дефект

СПОЧАТКУ ПЕРЕДБАЧ

Для балансу 100 і платежу 30 запиши очікуваний результат. Потім прочитай оператор += буквально й окремо передбач фактичний результат програми.

```
logic_bug.py
```

```
balance = 100  
payment = 30  
  
balance += payment  
  
print(f"Очікували 70, отримали {balance}")
```


СПОСТЕРЕЖЕННЯ ЛОКАЛІЗУЄ ЛОГІЧНИЙ ДЕФЕКТ

ДО
`balance = 100, payment = 30;`
очікувана поведінка — зменшення балансу до 70.

→

ПІСЛЯ
Після виправлення баланс дорівнює 70; очікування і фактичний стан збігаються.

1	Фактична операція	<code>balance += payment</code>	Оператор додає 30, хоча предметна дія «платіж» вимагала віднімання.
2	Фактичний стан	<code>balance == 130</code>	Програма працює без винятку, але стан суперечить очікуванню.
3	Порівняння	Очікували 70, отримали 130	Два конкретні числа звужують пошук до операції зміни балансу.
4	Одна правка	<code>balance -= payment</code>	Змінюється одна причина, а не весь фрагмент одразу.
5	Повторна перевірка	<code>balance == 70</code>	Той самий сценарій доводить, що поведінка відновлена.

Що тут важливо: Логічний дефект знаходять через явне очікування, спостереження стану та одну контрольовану зміну, а не випадкові редагування.

Програма не падає, але оператор має неправильний знак. Логічні дефекти знаходять через точне очікування, проміжний стан і тести.

Класифікація визначає наступну дію. Без неї людина часто безладно переставляє відступи, додає try або переписує робочі частини.

Майстерня

Початкова версія гаманця має дефект:

```
wallet_bug.py

class Wallet:
    def __init__(self, balance=0):
        self.balance = balance

    def spend(self, amount):
        if amount > self.balance:
            return False
        self.balance += amount
        return True

wallet = Wallet(100)
result = wallet.spend(30)

print(result)
print(wallet.balance)
```

Результат

True
130

Вимога: `Wallet(100).spend(30)` має повернути `True` і лишити баланс 70.

Крок 1. Найменше відтворення

Виклик і два `print()` внизу файла — це найменше відтворення. Не запускай весь великий інтерфейс, якщо дефект видно в кількох рядках. Менше рухомих частин — менше гіпотез.

Крок 2. Запиши очікування до виправлення

python

```
assert wallet.spend(30) is True
assert wallet.balance == 70
```

Другий `assert` падає й показує фактичне 130. Тепер проблема конкретна: метод повідомляє успіх, але змінює баланс у протилежний бік.

Крок 3. Перевір проміжний стан

Тимчасова діагностика:

python

```
print(f"DEBUG до: balance={self.balance!r}, amount={amount!r}")
self.balance += amount
print(f"DEBUG після: balance={self.balance!r}")
```

!r у f-рядку використовує `repr()`, тому типово показує лапки рядка й escape-символи. Це корисно для прихованих пробілів і різниці між "30" та 30.

Крок 4. Виправ одну причину

wallet_fixed.py

```
class Wallet:
    def __init__(self, balance=0):
        self.balance = balance

    def spend(self, amount):
        if amount <= 0:
            raise ValueError("Сума має бути додатною")
        if amount > self.balance:
            return False

        self.balance -= amount
        return True

wallet = Wallet(100)

print(wallet.spend(30))
print(wallet.balance)
print(wallet.spend(100))
print(wallet.balance)
```

Ми змінили += на -= і додали окрему перевірку неможливої суми. Невдала велика покупка не змінює стан.

Крок 5. Лиш регресійний тест

python

```
def test_spend_reduces_balance():
    wallet = Wallet(100)

    assert wallet.spend(30) is True
    assert wallet.balance == 70

def test_failed_spend_preserves_balance():
    wallet = Wallet(70)

    assert wallet.spend(100) is False
    assert wallet.balance == 70
```

Тимчасові print("DEBUG...") після діагностики прибори, а тести лиши.

Як читати traceback

Умовний приклад:

```
text

Traceback (most recent call last):
  File "main.py", line 12, in <module>
    print(report.build())
  File "report.py", line 18, in build
    return self.wallet.summary()
AttributeError: 'NoneType' object has no attribute 'summary'
```

Маршрут:

- 1 Останній рядок: `AttributeError`; викликали `summary` у `None`.
- 2 Найнижчий кадр власного коду: `report.py`, рядок 18 — місце прояву.
- 3 Кадр вище: хто викликав `build()`.
- 4 Питання до стану: чому `self.wallet` дорівнює `None`?
- 5 Пошук місця створення або присвоєння, а не додавання випадкової перевірки в останній рядок.

Місце винятку не завжди є місцем причини. Неправильне значення могло з'явитися раніше.

ПОВІДОМЛЕННЯ ЯК ДОКАЗ, НЕ ІНСТРУКЦІЯ

Traceback є даними про виконання. Він не наказує видалити рядок чи загорнути все в `try`. Спочатку зістав тип, фактичне значення й контракт об'єкта.

Спостереження без здогадок

`repr()` показує точніший вигляд

```
repr_debug.py

raw_name = "  Роби\n"
clean_name = raw_name.strip()
escaped_newline = "\\n"

print(repr(raw_name))
print(repr(clean_name))
print(f"Прихований символ видно: {escaped_newline in repr(raw_name)}")
```

Звичайний `print(raw_name)` приховав би, де закінчується значення.

type() перевіряє тип

```
python
```

```
print(type(value).__name__)
```

Це корисно біля межі `input()`, JSON або зовнішнього модуля. Не будуй логіку з десятків `type()` — після перевірки на межі працюй із надійними об'єктами.

vars() показує атрибути простого об'єкта

```
vars_debug.py
```

```
class Robot:
    def __init__(self, name, energy):
        self.name = name
        self.energy = energy
```

```
robot = Robot("Робі", 7)
print(vars(robot))
```

Це діагностика, не публічний формат збереження. Класи зі `__slots__` або деякі вбудовані об'єкти можуть не мати звичайного `__dict__`.

dir() і help() для дослідження

```
python
```

```
print(dir(robot))
help(str.strip)
```

`dir()` дає багато внутрішніх імен; використовуй його для дослідження, а не як заміну документації. `help()` показує вбудовану довідку об'єкта.

Налагоджувач VS Code

Breakpoint зупиняє програму **до** виконання вибраного рядка й дозволяє побачити живий стан.

Базовий маршрут:

- 1 Відкрий потрібний `.py`-файл.
- 2 Натисни в полі ліворуч від номера рядка або F9 — з'явиться червона точка.
- 3 Запусти **Python Debugger: Debug Python File** або F5.
- 4 Коли виконання зупиниться, переглянь **Variables**, **Watch** і **Call Stack**.
- 5 Виконуй по одному кроку.

Основні дії:

- Continue (F5) — до наступного breakpoint;
- Step Over (F10) — виконати рядок, не заходячи всередину виклику;
- Step Into (F11) — зайти у функцію або метод;
- Step Out (Shift+F11) — завершити поточний виклик і повернутися вище;
- Restart — повторити з початку;
- Stop (Shift+F5) — завершити сеанс.

На ноутбучі може знадобитися клавіша Fn разом із F-клавішами. Кнопки в панелі Debug працюють незалежно від розкладки.

ЗУПИНИ ГАМАНЕЦЬ ПЕРЕД ЗМІНОЮ

Постав breakpoint на `self.balance -= amount`. До виконання рядка перевір `self.balance`, `amount` і результат умови. Зроби Step Over і побач новий баланс. Не змінюй значення через Debug Console під час цієї перевірки.

Умовний breakpoint

Коли цикл має 100 елементів, зупинка на кожному незручна. Правою кнопкою на breakpoint можна додати умову, наприклад:

```
text
member.energy < 0
```

Зупинка відбудеться лише за істинної умови. Умова сама має бути безпечною й не змінювати стан.

Logpoint

Logpoint друкує повідомлення без зупинки та без редагування вихідного коду. Він зручний для спостереження в циклі. Після діагностики прибері зайві точки, щоб наступний запуск не дивував.

Вбудований breakpoint()

```
python
breakpoint()
```

У звичайному терміналі він запускає стандартний debugger `pdb`; у налаштованому середовищі може інтегруватися з іншим налагоджувачем. Не залишай випадковий `breakpoint()` у готовому коді.

Мінімальне відтворення

Якщо проблема виникає у великій програмі:

- 1 Скопіюй не весь проєкт, а найменший сценарій у окремий тест або маленький файл.
- 2 Прибери мережу, GUI, випадковість і реальні файли, якщо дефект лишається.
- 3 Передай фіксовані дані.
- 4 Переконайся, що проблема все ще відтворюється.
- 5 Тепер змінюй одну гіпотезу.

Якщо після видалення частини дефект зник, це теж доказ: поверни останню частину й досліди її взаємодію.

Не переписуй систему з нуля до локалізації. Нова версія може приховати симптом і створити інші дефекти, не пояснивши першопричину.

Журналювання для довшої роботи

Тимчасовий `print()` добрий у маленькому скрипті. Для програми з модулями стандартний `logging` дає рівні й джерело повідомлення:

```
logging_example.py

import logging

logger = logging.getLogger(__name__)

class Counter:
    def __init__(self, value=0):
        self.value = value

    def add(self, amount):
        logger.debug("Додавання %s до %s", amount, self.value)
        self.value += amount
        return self.value

logging.basicConfig(level=logging.WARNING)
counter = Counter(5)
print(f"Результат: {counter.add(2)}")
```

Рівні від детального до критичного:

- `DEBUG` — діагностика розробника;
- `INFO` — нормальна важлива подія;
- `WARNING` — незвична ситуація, робота триває;
- `ERROR` — операція не виконана;
- `CRITICAL` — система не може нормально продовжувати.

Модуль створює `logger = logging.getLogger(__name__)`, а точка входу налаштовує формат, рівень і місце виведення. Бібліотечний модуль не повинен сам викликати `basicConfig()` при імпорті.

У експерт метод `logger.exception("...")` додає `traceback`. Не записуй у журнал паролі, токени, приватні дані або весь вміст особистих файлів.

Стиль — інтерфейс для читача

PEP 8 описує загальні домовленості, але чинний стиль конкретного проєкту має пріоритет, якщо він послідовний.

Відступи й пробіли

- чотири пробіли на рівень;
- не змішуй табуляцію й пробіли;
- пробіли навколо бінарних операторів: `energy = start - cost`;
- без пробілу перед дужкою виклику: `move(3)`, не `move (3)`;
- після коми — пробіл.

Імена

- класи: `RobotConsole`, `PlayerProfile`;
- функції, методи, змінні: `load_profile`, `energy_cost`;
- константи: `DEFAULT_ENERGY`;
- внутрішні деталі: `_charge`;
- винятки: `ProfileDataError`;
- `self` — перший параметр методу екземпляра;
- `cls` — перший параметр `classmethod`.

Ім'я має пояснювати роль, а не лише тип. `players` краще за `player_list`, якщо список уже видно з коду; `active_players` краще за обидва, коли відбір важливий.

Довгі рядки

PEP 8 консервативно використовує 79 символів для коду стандартної бібліотеки й дозволяє командам домовлятися до 99. Важливіше не точне магічне число, а читання поруч із деревом файлів, терміналом і `diff`.

Перенось у дужках:

```
python

message = (
    f"{profile.name}: рівень {profile.level}; "
    f"навички: {skills_text}."
)
```

Уникай зворотної ризки для переносу, якщо дужки виражають структуру природно.

Порожні рядки

- між верхньорівневими класами й функціями — зазвичай два;
- між методами — один;
- всередині довгого методу — рідко, щоб відділити логічні фази.

Не перетворюй файл на «паркан» із порожніх рядків. Формат має показувати блоки.

Імпорти

- на початку файла після module docstring;
- стандартна бібліотека, сторонні, локальні — окремими групами;
- по одному модулю на рядок: `import json, import logging`;
- конкретні імена з одного модуля можна групувати;
- не використовуй `import *`.

Коментарі й docstrings

Коментар пояснює причину або обмеження:

```
python

# Копія не дозволяє зовнішньому списку змінити внутрішній порядок.
self.items = items.copy()
```

Не повторює очевидне:

```
python

self.energy -= 1 # Віднімаємо один від енергії
```

Docstring публічного класу чи функції пояснює контракт. Актуальний короткий текст кращий за довгий застарілий.

Анотації типів допомагають редактору

type_hints.py

```
class Robot:
    def __init__(self, name: str, energy: int = 10) -> None:
        self.name = name
        self.energy = energy

    def move(self, steps: int) -> int:
        self.energy -= steps
        return self.energy

robot: Robot = Robot("Робі")
remaining: int = robot.move(3)
print(f"{robot.name}: {remaining}")
```

- `name: str` — очікуваний тип параметра;
- `-> None` — метод не повертає корисного значення;
- `-> int` — очікуваний тип результату;
- `robot: Robot` — анотація змінної.

Колекції:

python

```
def names(robots: list[Robot]) -> list[str]:
    return [robot.name for robot in robots]
```

Необов'язкове значення в сучасному Python:

python

```
def find(robot_id: str) -> Robot | None:
    ...
```

Анотація **не перевіряє** тип автоматично під час звичайного виконання:

python

```
robot.move("три")
```

VS Code або окремий інструмент перевірки типів (type checker) може попередити, але Python все одно спробує виконати виклик. Вхідні дані перевіряй під час виконання там, де це частина контракту.

Додавай анотації насамперед до публічних меж і не ускладнюй початковий код довгими типами без користі.

Маленькі методи й явний стан

Ознаки, що метод варто розділити:

- він читає введення, змінює модель, пише файл і друкує;
- має багато рівнів вкладення;
- назва містить кілька дій через «і»;
- для перевірки треба готувати половину програми;
- одна зміна вимагає редагування далеких гілок.

Розділяй за відповідальністю, не за кількістю рядків. П'ятирядковий метод може мати три різні причини для зміни; двадцятирядкове ясне перетворення може бути єдиною дією.

Ранній вихід зменшує вкладення:

```
python

def spend(self, amount):
    if amount <= 0:
        raise ValueError("Сума має бути додатною")
    if amount > self.balance:
        return False

    self.balance -= amount
    return True
```

Карта синтаксису

Це не заміна пояснень попередніх розділів, а короткий показчик.

Значення й присвоєння

```
python

name = "Робі"           # str
energy = 10              # int
speed = 2.5              # float
is_ready = True          # bool
missing = None           # відсутнє значення
```

Колекції

```
python

items = ["карта", "ключ"]
position = (3, 5)
settings = {"volume": 70}
tags = {"python", "oop"}
```

Умова

```
python

if energy <= 0:
    message = "Порожньо"
elif energy < 5:
    message = "Мало"
else:
    message = "Достатньо"
```

Повторення

```
python

for item in items:
    print(item)

while is_ready:
    is_ready = update()
```

Функція

```
python

def calculate_cost(amount, price=10):
    return amount * price
```

Клас і об'єкт

```
python

class Robot:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Привіт, я {self.name}."

robot = Robot("Робі")
print(robot.greet())
```

Успадкування й композиція

```
python

class ServiceRobot(Robot):
    def __init__(self, name, tool):
        super().__init__(name)
        self.tool = tool
```

Імпорт

```
python
```

```
import json
from pathlib import Path
from objects.robot import Robot
```

Файл

```
python
```

```
path = Path("data.txt")
path.write_text("Привіт\n", encoding="utf-8")
text = path.read_text(encoding="utf-8")
```

Виняток

```
python
```

```
try:
    value = int(text)
except ValueError as error:
    raise DataError("неправильне число") from error
else:
    use(value)
```

JSON

```
python
```

```
text = json.dumps(data, ensure_ascii=False, indent=2)
data = json.loads(text)
```

Тест

```
python
```

```
def test_move_reduces_energy():
    robot = Robot("Робі", energy=10)
    robot.move(3)
    assert robot.energy == 7
```

Контрольний маршрут перед завершенням зміни

- 1 **Результат:** що саме тепер має бачити користувач або отримувати інший об'єкт?
- 2 **Межі:** які значення на межі, порожні або неправильні?

- 3 **Стан:** що має змінитися, а що гарантовано лишитися?
- 4 **Помилка:** який виняток очікуваний, а який означає дефект?
- 5 **Перевірка:** чи є найменший ручний сценарій і автоматичний тест?
- 6 **Структура:** чи живе правило в правильному класі або функції?
- 7 **Імена:** чи можна прочитати код без перекладу `x`, `data2`, `do_it`?
- 8 **Шляхи:** чи точно відомо, які файли читаються або перезаписуються?
- 9 **Системи:** чи не зашито Windows-шлях у переносну логіку?
- 10 **Прибирання:** чи видалено тимчасові `prints`, `breakpoints` і мертвий код?

ВІДНОВИ ДИСЦИПЛІНУ НАЛАГОДЖЕННЯ

Розташуй дії в безпечному порядку.

- повторно запусти той самий сценарій;
- запиши очікуваний результат;
- виправ одну ймовірну причину;
- створи найменше відтворення;
- виведи проміжний стан;
- залиш регресійний тест.

Поясни, чому одночасне виправлення трьох місць руйнує доказ причини.

Типова помилка

Одночасно змінено все

Перейменовані класи, перенесені файли, нова логіка й новий формат JSON в одному кроці. Коли тест падає, причин десятки. Розділи зміну на незалежні перевірні кроки.

Налагодження випадковими редагуваннями

Додано `try`, потім `if value`, потім глобальну змінну — без повторюваного прикладу. Спочатку зафіксуй фактичний стан і одну гіпотезу.

«Стиль» змінює поведінку

Під час форматування випадково пересунуто відступ, змінено оператор або степто `return`. Після механічного форматування все одно запускай тести й переглядай `diff`.

Типова анотація сприйнята як перевірка

`amount: int` не захищає від рядка з `input()` або `JSON`. Перетворення й перевірка під час виконання на зовнішній межі лишаються потрібними.

ТИПОВА ПОМИЛКА

Не приховуй логічну проблему широким ехсепт, значенням за замовчуванням або додатковим `if None` біля місця падіння. Знайди, де контракт уперше порушився, і виправ джерело.

Швидка перевірка

Знайди дефект до запуску, потім виправ:

```
safe_transfer.py

class Wallet:
    def __init__(self, balance):
        self.balance = balance

    def transfer_to(self, other, amount):
        if amount <= 0:
            raise ValueError("Сума має бути додатною")
        if amount > self.balance:
            return False

        self.balance -= amount
        other.balance += amount
        return True

first = Wallet(100)
second = Wallet(50)

first.transfer_to(second, 30)

print(first.balance)
print(second.balance)
```

Для надійності тест має також перевірити, що невдале передавання не змінює **жоден** гаманець.

ПЕРЕВІР СЕБЕ

Який перший корисний крок для логічного дефекту у великій програмі?

- Загорнути всю програму в ехсепт `Exception`.
- Переписати всі класи з нуля.
- Створити найменший повторюваний сценарій із точним очікуваним і фактичним результатом.

Відповідь: Створити найменший повторюваний сценарій із точним очікуваним і фактичним результатом. Мінімальне відтворення зменшує кількість причин. Після нього можна перевіряти стан і одну гіпотезу за раз.

ЗГАДАЙ БЕЗ ПІДГЛЯДАННЯ

Не дивлячись у розділ, назви три великі види проблем: синтаксичну, виняток під час виконання та логічний дефект. Для кожної назви перший доказ, який треба прочитати.

Самостійна робота

САМОСТІЙНА РОБОТА

- Візьми логічний дефект із `+=` замість `-=`. Пройди маршрут: мінімальне відтворення, червоний тест, breakpoint, виправлення, весь зелений набір.
- Створи traceback із трьох власних викликів. Познач місце прояву й окремо місце, де неправильне значення вперше з'явилося.
- Досліди рядок із пробілами й `\n` через `print()`, `repr()` та `len()`. Поясни, яку інформацію приховує звичайний вигляд.
- Постав у VS Code звичайний та умовний breakpoint у циклі. Перевір Variables, Call Stack, Step Into і Step Over без редагування стану.
- Заміни тимчасові `print("DEBUG")` на модульний logger із рівнем DEBUG. Налаштування зроби тільки в `main.py` і не записуй приватні дані.
- Відформатуй навмисно неохайний файл за правилами про імена, відступи, імпорти й переноси. Потім переглянь diff і запусти тести.
- Додай анотації до публічного класу Robot, функції з колекцією та пошуку з результатом Robot | None. Покажи окремо, що під час виконання введення все одно треба перевіряти.
- Скороти великий метод ранніми виходами й виділенням однієї відповідальності. Публічна поведінка до й після має підтверджуватися тим самим тестом.
- Склади власну односторінкову карту синтаксису лише з конструкцій, які вже запускав. Для кожної додай один рядок «коли використовувати».
- Пройди десятипунктовий контрольний маршрут для будь-якої своєї маленької програми й зафіксуй дві конкретні зміни, які підвищили надійність.

КОРОТКА ІСТОРИЧНА ПАУЗА

PEP — Python Enhancement Proposal, тобто публічна пропозиція або довідковий документ про розвиток Python. PEP 8 став спільною мовою стилю, але його найважливіша думка практична: код набагато частіше читають, ніж пишуть.

Підсумок

- Синтаксична помилка, виняток під час виконання й логічний дефект потребують різних способів пошуку.
- Надійний маршрут: відтворити, звузити, перевірити стан, виправити одну причину, додати регресійний тест.
- Traceback веде до місця прояву, але причина може з'явитися раніше.
- `repr()`, `type()`, `vars()`, `breakpoint` і `debugger` дають факти замість здогадок.
- VS Code дозволяє зупинятися, переглядати змінні й проходити виклики крок за кроком.
- `logging` краще за постійні діагностичні `print()` у багатомодульній програмі.
- Послідовний стиль, точні імена й малі відповідальності зменшують вартість наступної зміни.
- Анотації типів допомагають редактору й читачеві, але не замінюють перевірку зовнішніх даних під час виконання.
- Карта синтаксису є показником; розуміння приходить через запуск, зміну й перевірку.
- Завершена зміна має не лише працювати один раз, а й залишати доказ: тест, ясний стан і відтворюваний запуск.

Офіційні орієнтири: [PEP 8](#), [налагодження Python у VS Code](#), [logging](#), [анотації типів](#).